

DOCUMENTACIÓN TÉCNICA OFICIAL

Manual Técnico *del Backend*

Sistema POS KAFFA · API REST Laravel 10 + PostgreSQL

Arquitectura, instalación, base de datos, seguridad, servicios internos, referencia completa de la API y operación en producción.

AUTORES

Ana Maria Joaqui Conejo
Santiago Salamanca Muñoz
Juan José Sandoval Chávez

INSTITUCIÓN

Servicio Nacional de Aprendizaje — SENA
Programa ADSO · Popayán, Cauca

Octubre de 2026

Control documental

Título	Manual Técnico del Backend — Sistema POS KAFFA
Versión	1.0
Fecha de emisión	Octubre de 2026
Código documental	KAFFA-MT-BE-001
Audiencia	Ingenieros de software, DevOps, QA y personal técnico ajeno al proyecto
Alcance	Backend Laravel 10 + PostgreSQL, API REST, base de datos, servicios externos, seguridad y operación
Fuente de verdad	Código del backend auditado y documentación de `docs/`
Commit auditado	`01fb00d` — `fix(turnos): fecha_fin opcional`
Medición de calidad	Commit `1ea1e76`, 2 de noviembre de 2026 (capítulo 32)
Marco de calidad	ISO 9001, ISO/IEC 25010, ISO/IEC 27001, ISO/IEC 29119, ISO 8601 (alineación evaluada, no certificación)
Estado	Verificado contra código; recomendaciones marcadas como tales

Resumen ejecutivo

KAFFA es un sistema POS para cafetería con una API REST en Laravel 10 y PostgreSQL. El backend concentra autenticación, catálogo, pedidos, pagos, facturación interna, caja, inventario, compras, gastos, mermas, turnos, chat, reportes, eventos y un asistente local. La interfaz web y la app móvil son clientes desacoplados que consumen JSON.

El sistema está diseñado para una operación de una o varias sedes pequeñas: usa transacciones y bloqueos pesimistas para evitar sobreventa, cierres dobles y facturas duplicadas; aplica RBAC con roles, abilities y policies; y separa el correo transaccional (Brevo/Mailpit) de las imágenes (Cloudinary o disco local).

Este manual permite a un ingeniero externo instalar el backend desde cero, entender el modelo de datos, operar la API, desplegarlo y mantenerlo. No es una certificación de seguridad ni un manual fiscal: las limitaciones se documentan de forma explícita.

Alcance y exclusiones

Incluido: instalación, configuración, base de datos, servicios externos, API completa, seguridad, pruebas, despliegue, operación y mantenimiento.

Excluido: manual de usuario final de la SPA, diseño visual del frontend, facturación electrónica certificada, pasarela de pagos en línea, aplicación móvil nativa y infraestructura como código (no presentes en el repositorio).

Cómo leer este manual

Este documento está pensado para que una persona que nunca ha visto KAFFA pueda instalar, entender, operar y extender el backend sin depender de sus autores. Se recomienda leer las partes I y II completas antes de tocar el código; después, usar la parte V como referencia de consulta.

Los capítulos combinan explicación, procedimiento y verificación. Cuando una afirmación no pudo comprobarse contra el código, se marca de forma explícita.

Convenciones y evidencias

- **Verificado**: comportamiento comprobado en rutas, controladores, servicios, migraciones o pruebas.
- **Recomendación**: práctica sugerida para producción, no necesariamente implementada.
- **[NO VERIFICADO]**: el repositorio no contiene evidencia suficiente.
- **[GAP]**: funcionalidad existente sin cobertura de pruebas o con documentación incompleta.

> NOTA
>
> **Nota.** Los ejemplos usan datos ficticios y nunca incluyen credenciales reales. Los importes se muestran en pesos colombianos (COP) salvo que se indique otra moneda.

Historial de revisiones

Versión	Fecha	Autor	Cambio
1.0	Octubre 2026	Equipo KAFFA	Creación del manual consolidado y verificado del backend
1.0 (rev. 2)	Noviembre 2026	Equipo KAFFA	Corrección de diagramas, numeración e índice de tablas, y evaluación de calidad frente a normas ISO

Soporte y mantenimiento

El manual se regenera con `node build/build-all.mjs` desde `docs/manual-tecnico/`. Las fuentes están en `src/` y los datos extraídos del backend en `data/`. Cualquier cambio de código debe reflejarse en el capítulo correspondiente y en la matriz de endpoints.

Índice general

Los números de página corresponden a la paginación del cuerpo del manual (la portada no se numera).

1. Contexto del sistema 21

1.1 Problema de negocio 21
1.2 Actores y responsabilidades 21
1.3 Módulos funcionales 22
1.4 Flujo general de una venta 22
1.5 Qué queda fuera del alcance 23
1.6 Fuentes de verdad 23
1.7 Cómo está organizado el manual 23

2. Arquitectura del sistema 25

2.1 Vista de contexto 25
2.2 Stack verificado 26
2.3 Capas del backend 27
2.4 Ciclo de una petición 28
2.5 Estructura de carpetas 28
2.6 Decisiones de diseño relevantes 29
2.7 Integración con los clientes 30
2.8 Topología de desarrollo y producción 30
2.9 Restricciones y riesgos arquitectónicos 31

3. Requisitos de instalación 32

3.1 Resumen ejecutivo 32
3.2 Sistema operativo y usuario 32
3.3 PHP y extensiones 33
3.4 PostgreSQL 34
3.5 Composer y dependencias PHP 35
3.6 Node.js, pnpm y frontend 35
3.7 Puertos y red 36
3.8 Dominio, DNS y TLS 36
3.9 Cuentas de servicios externos 36
3.10 Checklist previo a la instalación 37

3.11 Matriz de compatibilidad 37

4. Instalación paso a paso 38

4.1 Obtener el código 38
4.2 Instalar dependencias PHP 38
4.3 Crear el archivo de entorno 38
4.4 Configurar la base de datos 39
4.5 Migrar y sembrar 39
4.6 Enlazar el disco público 40
4.7 Permisos de escritura 40
4.8 Levantar el servidor de desarrollo 40
4.9 Probar el login 41
4.10 Usuarios de prueba 41
4.11 Frontend opcional 41
4.12 Base de pruebas y PHPUnit 42
4.13 Smoke test completo 42
4.14 Problemas comunes de instalación 42

5. Configuración del backend 44

5.1 Regla de oro 44
5.2 Variables de aplicación 44
5.3 Base de datos 45
5.4 Sanctum y sesión 45
5.5 CORS 46
5.6 OTP y verificación 46
5.7 Correo 47
5.8 Cloudinary 47
5.9 Caché, colas y sistema de archivos 48
5.10 Logging y errores 48
5.11 OpenAPI / Scramble 49
5.12 Variables reservadas 49
5.13 Plantilla mínima de producción 49
5.14 Checklist de configuración 50

6. Operación de la base de datos 51

6.1 Motor y conexión 51

6.2 Crear la base desde cero 51
6.3 Migraciones 51
6.4 Seeders 52
6.5 Respaldos 53
6.6 Mantenimiento 54
6.7 Consultas útiles de operación 54
6.8 Integridad y mantenimiento de datos 56
6.9 Checklist de operación 56

7. Servicios externos 57

7.1 Mapa de dependencias 57
7.2 Correo transaccional 58
7.3 Cloudinary 60
7.4 Disco público para eventos móviles 61
7.5 Gemini (reservado) 61
7.6 CDNs del frontend 62
7.7 Matriz de fallos y degradación 62
7.8 Checklist de servicios externos 62

8. Modelo de datos 63

8.0 Cómo leer el diccionario 63
8.1 Motor y convenciones 63
8.2 Inventario de tablas (34) 64
8.3 Diagrama entidad–relación 65
8.4 Diccionario de datos 65
8.5 Índices de rendimiento — 2026_07_31_000000_add_performance_indexes_to_tables.php 78
8.6 Seeders — backend-kaffa/database/seeders/ 78

9. Flujos de datos y consistencia 80

9.1 Alta diferida y verificación 80
9.2 Pedido: del carrito al inventario 81
9.3 Compra: aumento de stock y egreso 85
9.4 Gasto y merma 86
9.5 Caja y arqueo 86
9.6 Turnos 87
9.7 Asistente KAFFA 88

9.8 Invariantes del sistema 88

10. Organización del código 89

10.1 Mapa de responsabilidades 89

10.2 Rutas 89

10.3 Controladores 90

10.4 Form Requests 90

10.5 Servicios 91

10.6 Modelos y trait HasBasicQueries 91

10.7 Policies y gates 91

10.8 Middleware 92

10.9 Recursos y serialización 92

10.10 Eventos y listeners 92

10.11 Excepciones 93

10.12 Convenciones de código 93

11. Autenticación y seguridad 94

11.1 Modelo de autenticación 94

11.2 Registro y verificación por OTP 95

11.3 Sesión y revocación 95

11.4 Roles 95

11.5 Abilities 96

11.6 Matriz RBAC resumida 96

11.7 Policies y ownership 97

11.8 Middleware turno.activo 97

11.9 Throttling 97

11.10 CORS 98

11.11 Protección de datos 98

11.12 Hallazgos y estado 99

11.13 Checklist de seguridad 99

12. Servicios de negocio 100

12.1 TransactionService 100

12.2 PedidoService::crear 100

12.3 CompraService::crear 101

12.4 GastoService 102

12.5 MermaService 102
12.6 Caja 102
12.7 FacturaService::generarNumeroFactura 103
12.8 TurnoService 103
12.9 RegistrationService y OtpService 104
12.10 AsistenteService 104
12.11 Tabla resumen de servicios 105

13. Errores y códigos de negocio 106

13.1 Formato general 106
13.2 Códigos HTTP 106
13.3 Códigos de negocio 106
13.4 Dónde se lanza cada error 107
13.5 Errores de autenticación 107
13.6 Errores de validación frecuentes 108
13.7 Diagnóstico por síntoma 108
13.8 Logging 108
13.9 Recomendaciones para clientes 108

14. Concurrencia, transacciones y rendimiento 110

14.1 Transacciones 110
14.2 Bloqueos pesimistas 110
14.3 Deadlocks 110
14.4 Inventario atómico 110
14.5 Facturación sin carreras 111
14.6 Paginación y filtros 111
14.7 Índices 111
14.8 Caché 112
14.9 Capacidad y límites 112
14.10 Pruebas de carga recomendadas 112
14.11 Antipatrones prohibidos 113

15. Convenciones de la API 114

15.1 Base URL y versionado 114
15.2 Autenticación 114
15.3 Formato de datos 114

15.4 Paginación 114
15.5 Filtros 115
15.6 Ordenamiento 115
15.7 Códigos de estado y errores 116
15.8 Throttle 116
15.9 Ejemplo curl completo 116
15.10 Idempotencia y reintentos 117
15.11 Zona horaria y dinero 117
15.12 Content-Type y archivos 117
15.13 Encabezados recomendados 117
15.14 Versionado de OpenAPI 118

16. API de autenticación y sesión 119

16.1 Endpoints 119
16.2 POST /login 119
16.3 POST /registro 120
16.4 POST /email/verify-code 121
16.5 POST /email/verification-notification 121
16.6 POST /forgot-password 121
16.7 POST /reset-password 121
16.8 GET /me y GET /user 122
16.9 PUT /perfil 122
16.10 POST /logout 122
16.11 POST /auth/refresh 122
16.12 POST /auth/logout-all 123
16.13 GET /health 123
16.14 Flujo recomendado para clientes 123

17. Endpoints públicos 124

17.1 Listado 124
17.2 GET /equipo 124
17.3 GET /categorias 125
17.4 GET /productos 125
17.5 GET /eventos 125
17.6 Seguridad de los endpoints públicos 126
17.7 Ejemplos curl 126

18. API de catálogo 127

18.1 Matriz de acceso 127
18.2 Categorías 127
18.3 Productos 128
18.4 Insumos 128
18.5 Medios de pago 129
18.6 Imágenes de producto 129
18.7 Errores frecuentes 130
18.8 Ejemplo completo curl 130

19. API de pedidos, pagos y facturas 131

19.1 Endpoints 131
19.2 POST /pedidos 131
19.3 Autoría por rol 133
19.4 PUT /pedidos/{id} 133
19.5 DELETE /pedidos/{id} 133
19.6 Pagos de pedido 133
19.7 Facturas 134
19.8 Errores frecuentes 134
19.9 Ejemplo curl 135
19.10 Recomendaciones de integración 135

20. API de caja y movimientos 136

20.1 Endpoints 136
20.2 Apertura 136
20.3 Cierre 137
20.4 Movimientos 137
20.5 Reglas 138
20.6 Arqueo 138
20.7 Errores 138
20.8 Ejemplo curl 138

21. API de compras, proveedores, gastos y mermas 140

21.1 Endpoints 140
21.2 Compras 140
21.3 Proveedores 141

21.4 Gastos 141
21.5 Mermas 141
21.6 Stock: reglas de oro 142
21.7 Caja en compras/gastos 142
21.8 Errores 142
21.9 Ejemplo curl 143
21.10 Reportería 143

22. API de turnos 144

22.1 Endpoints 144
22.2 Crear 144
22.3 Horarios 144
22.4 Turno activo 145
22.5 mis-turnos 145
22.6 Reglas de negocio 145
22.7 Errores 146
22.8 Ejemplo curl 146
22.9 Operación diaria 146

23. API de chat y reportes 147

23.1 Chat: endpoints 147
23.2 Bandeja 147
23.3 Enviar 147
23.4 Marcar leídos 148
23.5 Contactos sugeridos 148
23.6 Reportes 148
23.7 Sondeo 149
23.8 Errores 149
23.9 Ejemplo curl 149

24. API de usuarios y roles 150

24.1 Endpoints 150
24.2 Crear usuario 150
24.3 Actualizar 150
24.4 Filtrar por rol 151
24.5 Roles 151

24.6 Pivote usuario-rol 151
24.7 Eliminar 151
24.8 Seguridad 151
24.9 Errores 152
24.10 Ejemplo curl 152

25. Asistente KAFFA 153

25.1 Endpoints 153
25.2 Enviar mensaje 153
25.3 Historial 153
25.4 Reiniciar 154
25.5 Capacidades por rol 154
25.6 Cómo detecta la intención 155
25.7 Privacidad 155
25.8 Limitaciones 155
25.9 Ejemplos curl 156
25.10 Errores 156

26. Matriz completa de endpoints 157

26.1 Resumen por módulo 157
26.2 Detalle por módulo 157
26.3 Cómo leer la matriz 164

27. Despliegue en producción 165

27.1 Topología recomendada 165
27.2 Preparar el servidor 165
27.3 Desplegar el backend 166
27.4 Nginx 166
27.5 Apache 167
27.6 PHP-FPM 167
27.7 Compilar la SPA 168
27.8 Runbook de despliegue 168
27.9 Rollback 168
27.10 Variables de producción 168
27.11 Consideraciones de escalado 169
27.12 Checklist de despliegue 169

28. Observabilidad y operación 170

28.1 Health checks 170
28.2 Logs 170
28.3 Métricas recomendadas 170
28.4 Alertas mínimas 171
28.5 Backups y recuperación 171
28.6 Runbook: API caída 171
28.7 Runbook: correos no llegan 172
28.8 Runbook: caja descuadrada 172
28.9 Runbook: stock negativo o inconsistente 172
28.10 Capacidad y crecimiento 172
28.11 Documentación operativa 172

29. Seguridad operativa 174

29.1 Superficie de exposición 174
29.2 Hardening del servidor 174
29.3 Gestión de secretos 175
29.4 Seguridad de la base de datos 175
29.5 Seguridad de la API 175
29.6 Seguridad del frontend 175
29.7 Respuesta a incidentes 176
29.8 Checklist de auditoría trimestral 176

30. Pruebas y calidad 177

30.1 Configuración 177
30.2 Inventario de pruebas 177
30.3 Qué cubre la suite 178
30.4 Gaps 178
30.5 Cómo añadir una prueba 178
30.6 Pruebas manuales de aceptación 179
30.7 Calidad de código 179
30.8 CI recomendado 180
30.9 Criterios de aceptación de un cambio 180

31. Mantenimiento y contribución 181

31.1 Añadir un endpoint 181

31.2 Añadir una tabla o columna 181
31.3 Regenerar OpenAPI 182
31.4 Mantener el manual 182
31.5 Convenciones de documentación 182
31.6 Deuda técnica conocida 183
31.7 Roadmap sugerido 183
31.8 Contacto y soporte 183

32. Calidad del sistema frente a normas ISO 184

32.1 Método de evaluación 184
32.2 Correspondencia entre las normas y el manual 185
32.3 ISO/IEC 25010:2023 — calidad de producto 186
32.4 ISO/IEC 27001:2022 — controles de seguridad 188
32.5 ISO 9001:2015 — gestión de la calidad 189
32.6 ISO/IEC 29119 — proceso de pruebas 190
32.7 ISO/IEC/IEEE 12207 y 42010 — ciclo de vida y arquitectura 190
32.8 ISO 8601 y PDF/UA — formatos y accesibilidad 191
32.9 Resultado global 191
32.10 Brechas y plan de mejora 191
32.11 Limitaciones de la evaluación 192

90. Anexos 193

A. Plantilla de entorno comentada 193
B. Catálogo OpenAPI 194
C. SQL útil 194
D. Configuraciones de servidor 195
E. Catálogo de errores 195
F. Trazabilidad de pruebas 196
G. Correspondencia documento-código 196
H. Glosario 197

91. Referencias y control de versiones 198

91.1 Referencias técnicas 198
91.2 Documentos internos 198
91.3 Control de versiones del manual 199
91.4 Cómo mantenerlo 199

Fin del índice

Índice de tablas

Las tablas se numeran en orden de lectura. La página indicada es la del cuerpo del manual.

Capítulo 1. Contexto del sistema

Tabla 1. Actores del sistema y sus permisos 21
Tabla 2. Módulos funcionales del backend 22
Tabla 3. Fuentes de verdad por dominio 23

Capítulo 2. Arquitectura del sistema

Tabla 4. Stack tecnológico verificado 26
Tabla 5. Capas del backend y responsabilidades 27
Tabla 6. Integración del backend con la SPA y la app móvil 30
Tabla 7. Riesgos arquitectónicos y mitigaciones 31

Capítulo 3. Requisitos de instalación

Tabla 8. Requisitos mínimos y recomendados por categoría 32
Tabla 9. Extensiones PHP requeridas y opcionales 33
Tabla 10. Dependencias PHP declaradas en composer.json 35
Tabla 11. Herramientas de Node.js para el frontend 35
Tabla 12. Puertos y exposición de red 36
Tabla 13. Cuentas de servicios externos necesarias 36
Tabla 14. Matriz de compatibilidad de versiones 37

Capítulo 4. Instalación paso a paso

Tabla 15. Usuarios de prueba creados por los seeders 41
Tabla 16. Smoke test posterior a la instalación 42
Tabla 17. Problemas comunes de instalación y solución 42

Capítulo 5. Configuración del backend

Tabla 18. Gestión de secretos por entorno 44
Tabla 19. Variables de aplicación 44
Tabla 20. Variables de conexión a la base de datos 45
Tabla 21. Variables de Sanctum y sesión 45
Tabla 22. Variables del servicio OTP 46
Tabla 23. Variables de Cloudinary 48
Tabla 24. Variables de caché, colas y almacenamiento 48
Tabla 25. Variables reservadas o sin uso 49

Capítulo 6. Operación de la base de datos

Tabla 26. Parámetros de conexión a PostgreSQL 51
Tabla 27. Seeders de la base de datos 52
Tabla 28. Elementos a incluir en el respaldo 53

Capítulo 7. Servicios externos

Tabla 29. Mapa de dependencias externas 57
Tabla 30. Reglas y valores por defecto del código OTP 58
Tabla 31. Fallos típicos del correo transaccional 59
Tabla 32. Matriz de fallos y degradación por servicio 62

Capítulo 8. Modelo de datos

Tabla 33. Inventario: tablas de infraestructura de Laravel 64
Tabla 34. Inventario: tablas del núcleo de negocio 64
Tabla 35. Inventario: tablas de los módulos de características 65
Tabla 36. Diccionario de la tabla users 65
Tabla 37. Diccionario de la tabla rols 66
Tabla 38. Diccionario de la tabla usuario__rols 66
Tabla 39. Diccionario de la tabla categorias 66
Tabla 40. Diccionario de la tabla productos 67
Tabla 41. Diccionario de la tabla recetas 67
Tabla 42. Diccionario de la tabla insumos 67
Tabla 43. Diccionario de la tabla proveedors 68
Tabla 44. Diccionario de la tabla medio__pagos 68
Tabla 45. Diccionario de la tabla pedidos 68
Tabla 46. Diccionario de la tabla detalle__pedidos 69
Tabla 47. Diccionario de la tabla pago__pedidos 69
Tabla 48. Diccionario de la tabla factura__ventas 69
Tabla 49. Diccionario de la tabla factura_contadores 70
Tabla 50. Diccionario de la tabla compras 70
Tabla 51. Diccionario de la tabla detalle__compras 71
Tabla 52. Diccionario de la tabla compra__pagos 71
Tabla 53. Diccionario de la tabla gastos 71
Tabla 54. Diccionario de la tabla gasto__pagos 72
Tabla 55. Diccionario de la tabla mermas 72
Tabla 56. Diccionario de la tabla cajas 72
Tabla 57. Diccionario de la tabla movimiento__cajas 73
Tabla 58. Diccionario de la tabla turnos 74
Tabla 59. Diccionario de la tabla barista_turno 74
Tabla 60. Diccionario de la tabla reportes 74
Tabla 61. Diccionario de la tabla mensajes 75
Tabla 62. Diccionario de la tabla asistente_mensajes 75
Tabla 63. Diccionario de la tabla personal_access_tokens 75
Tabla 64. Diccionario de la tabla password_reset_tokens 76
Tabla 65. Diccionario de la tabla codigos_otp 76
Tabla 66. Diccionario de la tabla registros_pendientes 77
Tabla 67. Diccionario de la tabla eventos 77
Tabla 68. Diccionario de la tabla evento_imagenes 78
Tabla 69. Índices de rendimiento añadidos por tabla 78
Tabla 70. Seeders del proyecto (orden de ejecución) 79

Capítulo 9. Flujos de datos y consistencia

Tabla 71. Pasos del alta diferida de clientes 80
Tabla 72. Autoría de un pedido según rol 83
Tabla 73. Efecto de cada operación sobre stock, caja y autoría 86
Tabla 74. Horarios de los tipos de turno 87
Tabla 75. Invariantes del sistema y dónde se garantizan 88

Capítulo 10. Organización del código

Tabla 76. Mapa de responsabilidades por necesidad 89
Tabla 77. Servicios de negocio y su responsabilidad 91
Tabla 78. Gates de autorización y su uso 92
Tabla 79. Middleware registrado y su efecto 92

Capítulo 11. Autenticación y seguridad

Tabla 80. Flujo de registro y verificación por OTP 95
Tabla 81. Endpoints de sesión y revocación de tokens 95
Tabla 82. Roles del sistema 95
Tabla 83. Abilities asignadas a cada rol 96
Tabla 84. Matriz RBAC resumida 96
Tabla 85. Respuestas del middleware de turno activo 97
Tabla 86. Límites de throttling por grupo 97
Tabla 87. Hallazgos de seguridad y estado 99

Capítulo 12. Servicios de negocio

Tabla 88. Errores de negocio de PedidoService 101
Tabla 89. Horarios de TurnoService 103
Tabla 90. Intenciones detectadas por el AsistenteService 105
Tabla 91. Resumen transaccional de los servicios 105

Capítulo 13. Errores y códigos de negocio

Tabla 92. Uso de los códigos HTTP 106
Tabla 93. Catálogo de códigos de negocio 106
Tabla 94. Origen de cada error 107
Tabla 95. Errores de autenticación y su respuesta 107
Tabla 96. Errores de validación frecuentes 108
Tabla 97. Diagnóstico de errores por síntoma 108

Capítulo 14. Concurrencia y rendimiento

Tabla 98. Bloqueos pesimistas por recurso 110
Tabla 99. Límites de capacidad y cuellos de botella 112

Capítulo 15. Convenciones de la API

Tabla 100. Base URL por entorno 114
Tabla 101. Parámetros de paginación 115
Tabla 102. Códigos de estado y su significado 116
Tabla 103. Límites de throttle de la API 116

Capítulo 16. API de autenticación y sesión

Tabla 104. Endpoints de autenticación y sesión 119
Tabla 105. Respuestas de POST /login 120

Capítulo 17. Endpoints públicos

Tabla 106. Endpoints públicos 124
Tabla 107. Parámetros de GET /equipo 124

Capítulo 18. API de catálogo

Tabla 108. Matriz de acceso al catálogo 127
Tabla 109. Medios de pago y su carácter virtual 129
Tabla 110. Errores frecuentes del catálogo 130

Capítulo 19. API de pedidos, pagos y facturas

Tabla 111. Endpoints de pedidos, pagos y facturas 131
Tabla 112. Autoría y comportamiento por rol 133
Tabla 113. Errores frecuentes de pedidos 134

Capítulo 20. API de caja y movimientos

Tabla 114. Endpoints de caja y movimientos 136
Tabla 115. Campos de un movimiento de caja 137
Tabla 116. Interpretación de la diferencia de arqueo 138
Tabla 117. Errores frecuentes de caja 138

Capítulo 21. API de compras, proveedores, gastos y mermas

Tabla 118. Endpoints de compras, proveedores, gastos y mermas 140
Tabla 119. Reglas de stock por acción 142
Tabla 120. Errores frecuentes de compras y mermas 142

Capítulo 22. API de turnos

Tabla 121. Endpoints de turnos 144
Tabla 122. Horarios por tipo de turno 144
Tabla 123. Errores frecuentes de turnos 146

Capítulo 23. API de chat y reportes

Tabla 124. Endpoints de chat 147
Tabla 125. Endpoints de reportes 148
Tabla 126. Errores frecuentes de chat y reportes 149

Capítulo 24. API de usuarios y roles

Tabla 127. Endpoints de usuarios y roles 150
Tabla 128. Errores frecuentes de usuarios 152

Capítulo 25. Asistente KAFFA

Tabla 129. Endpoints del Asistente KAFFA 153
Tabla 130. Intenciones y ejemplos de pregunta 155

Tabla 131. Errores frecuentes del asistente 156

Capítulo 26. Matriz completa de endpoints

Tabla 132. Resumen de operaciones por módulo 157
Tabla 133. Matriz de endpoints: autenticación y sesión 157
Tabla 134. Matriz de endpoints: sitio público y eventos 158
Tabla 135. Matriz de endpoints: catálogo 159
Tabla 136. Matriz de endpoints: pedidos, pagos y facturas 160
Tabla 137. Matriz de endpoints: caja y movimientos 160
Tabla 138. Matriz de endpoints: compras y proveedores 161
Tabla 139. Matriz de endpoints: gastos y mermas 162
Tabla 140. Matriz de endpoints: turnos 162
Tabla 141. Matriz de endpoints: chat y reportes 162
Tabla 142. Matriz de endpoints: usuarios y roles 163
Tabla 143. Matriz de endpoints: asistente KAFFA 164

Capítulo 27. Despliegue en producción

Tabla 144. Componentes de la topología de producción 165

Capítulo 28. Observabilidad y operación

Tabla 145. Health checks por nivel 170
Tabla 146. Métricas recomendadas y alertas 170
Tabla 147. Política de respaldos por activo 171

Capítulo 29. Seguridad operativa

Tabla 148. Superficie de exposición y riesgo 174
Tabla 149. Gestión y rotación de secretos 175

Capítulo 30. Pruebas y calidad

Tabla 150. Inventario de archivos de prueba 177
Tabla 151. Brechas de cobertura conocidas 178
Tabla 152. Pruebas manuales de aceptación 179

Capítulo 31. Mantenimiento y contribución

Tabla 153. Deuda técnica conocida 183

Capítulo 32. Calidad del sistema frente a normas ISO

Tabla 154. Escala de madurez usada en la evaluación 184
Tabla 155. Mediciones ejecutadas y comando reproducible 184
Tabla 156. Normas ISO consideradas y lugar donde se aplican 185
Tabla 157. Medición de la adecuación funcional 186
Tabla 158. Medición de la eficiencia de desempeño 186
Tabla 159. Medición de la compatibilidad 186
Tabla 160. Medición de la capacidad de interacción 187
Tabla 161. Medición de la fiabilidad 187
Tabla 162. Medición de la seguridad 187
Tabla 163. Medición de la mantenibilidad 188

Tabla 164. Medición de la flexibilidad 188
Tabla 165. Medición de la seguridad funcional 188
Tabla 166. Cobertura de controles de ISO/IEC 27001:2022 189
Tabla 167. Alineación con ISO 9001:2015 190
Tabla 168. Métricas del proceso de prueba 190
Tabla 169. Alineación con los procesos de ciclo de vida y arquitectura 190
Tabla 170. Resumen de la evaluación ISO/IEC 25010:2023 191
Tabla 171. Brechas priorizadas y acciones de mejora 191

Capítulo 90. Anexos

Tabla 172. Catálogo OpenAPI por grupo de rutas 194
Tabla 173. Catálogo de errores de negocio 195
Tabla 174. Trazabilidad entre requisitos y pruebas 196
Tabla 175. Correspondencia entre documentos y código 196
Tabla 176. Glosario de términos 197

Capítulo 91. Referencias y control de versiones

Tabla 177. Documentos internos del proyecto 198
Tabla 178. Control de versiones del manual 199

Fin del índice de tablas

1. Contexto del sistema

KAFFA es un sistema de punto de venta (POS) para una cafetería. Su backend es una API REST en Laravel 10 que concentra las reglas de negocio y la persistencia; la interfaz web y la aplicación móvil son clientes desacoplados que consumen JSON.

Este capítulo explica **qué hace el sistema, quién lo usa y dónde termina su alcance**. Es la base para interpretar el resto del manual: si una decisión de código no encaja con el contexto descrito aquí, conviene revisarla antes de modificarla.

1.1 Problema de negocio

Una cafetería necesita operar de forma coordinada cuatro frentes que normalmente viven en hojas de cálculo separadas:

1. **Venta y cobro**: pedidos de mostrador y pedidos creados por clientes, con varios medios de pago y comprobantes.
2. **Inventario**: consumo de insumos por receta, compras a proveedores, mermas y stock mínimo.
3. **Caja**: apertura con base, movimientos automáticos por pagos y cierre con arqueo.
4. **Operación del personal**: turnos, permisos por rol, chat con clientes y reportes de novedades.

KAFFA integra esos frentes en una sola base de datos transaccional. El backend no “dibuja” pantallas: expone recursos y validaciones para que cualquier cliente (SPA, app móvil o integración futura) opere con las mismas reglas.

1.2 Actores y responsabilidades

Tabla 1. *Actores del sistema y sus permisos*

Actor	Rol técnico	Qué puede hacer	Restricciones principales
Visitante	Sin autenticación	Ver menú, categorías, eventos publicados y equipo visible	Solo lectura; no ve insumos, recetas, pagos ni métricas
Cliente	`cliente`	Registrarse, verificar correo, iniciar sesión, crear pedidos propios, consultarlos, chatear y usar el Asistente	Solo ve sus pedidos; no puede cambiar estados ni operar caja
Barista	`barista`	POS de mostrador, cobros, compras, gastos, mermas, caja, chat y reportes	Requiere **turno activo** para mutar recursos operativos; solo opera su propia caja
Administrador	`admin`	Catálogo, usuarios, roles, turnos, eventos, facturas, supervisión y reportes	Exento de turno; token con ability `*`; no puede saltarse validaciones de negocio

Un usuario puede tener **varios roles a la vez** (`usuario_rols`). Esto permite, por ejemplo, que una persona sea barista y cliente sin duplicar cuentas.

1.3 Módulos funcionales

Tabla 2. *Módulos funcionales del backend*

Módulo	Propósito	Entidades principales
Autenticación	Registro público, verificación por OTP, login, refresh y recuperación	`users`, `registros_pendientes`, `codigos_otp`, `personal_access_tokens`
Catálogo	Menú vendible, composición por insumos y medios de pago	`categorias`, `productos`, `insumos`, `recetas`, `medio__pagos`
Pedidos y pagos	Ciclo de venta, cobro, comprobantes y factura	`pedidos`, `detalle__pedidos`, `pago__pedidos`, `factura__ventas`
Caja	Sesión de caja, movimientos y arqueo	`cajas`, `movimiento__cajas`
Compras	Abastecimiento y aumento de stock	`compras`, `detalle__compras`, `compra__pagos`, `proveedors`
Gastos	Egresos operativos	`gastos`, `gasto__pagos`
Mermas	Pérdidas de insumos y ajuste de stock	`mermas`, `insumos`
Turnos	Asignación de baristas y control de horario	`turnos`, `barista_turno`
Chat y reportes	Comunicación cliente ↔ personal y novedades	`mensajes`, `reportes`
Eventos	Contenido de la landing y Home móvil	`eventos`, `evento_imagenes`
Asistente KAFFA	Chatbot local con respuestas por rol	`asistente_mensajes`

1.4 Flujo general de una venta

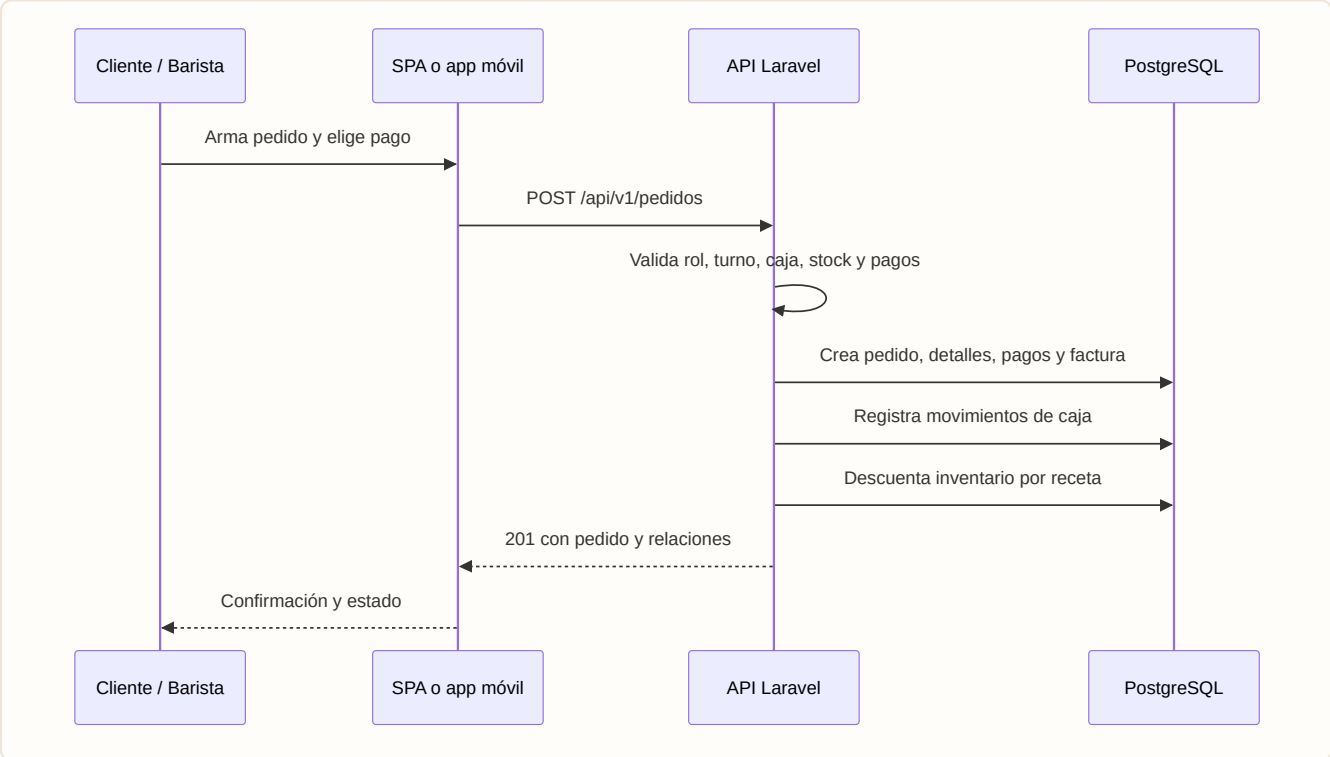


Figura 1. Flujo resumido de una venta. El detalle transaccional está en el capítulo 12.

1.5 Qué queda fuera del alcance

Para evitar expectativas incorrectas, el código **no** implementa:

- Pagos en línea reales: los medios "virtuales" son transferencias, QR o pagos móviles acreditados con un **comprobante** subido a Cloudinary.
- Facturación electrónica fiscal colombiana: la numeración es interna (`F{AÑO}{NNNNNN}`), sin DIAN.
- Websockets o tiempo real: la SPA usa sondeo (5–60 s).
- App Android/iOS nativa dentro de este repositorio: existe una especificación derivada, pero no un proyecto móvil.
- Docker, CI/CD o infraestructura como código: el repositorio no los incluye.
- Multisesión por cookies: la API es *pure bearer token*.

> **RECOMENDACIÓN**
>
> **Recomendación.** Si el negocio necesita facturación electrónica o pagos en línea, conviene diseñarlos como módulos nuevos con idempotencia y conciliación, no como un parche sobre `PedidoService`.

1.6 Fuentes de verdad

Tabla 3. *Fuentes de verdad por dominio*

Dominio	Fuente canónica
Endpoints	`backend-kaffa/routes/api.php` + `docs/openapi.json`
Base de datos	Migraciones + `docs/02-base-datos.md`
Reglas de negocio	Servicios (`app/Services`) + `docs/04-modulos.md`
Seguridad	Middleware, policies y `docs/05-seguridad.md`
Operación	`.env.example`, `config/*`, `docs/07-despliegue.md`
Pruebas	`backend-kaffa/tests` + `docs/08-pruebas.md`

Cuando este manual y un README de aplicación discrepen, rige el código y la especificación OpenAPI. Los READMEs son resúmenes, no contratos.

1.7 Cómo está organizado el manual

- **Parte I (capítulos 1–2)**: contexto y arquitectura.
- **Parte II (3–7)**: requisitos, instalación, configuración, base de datos y servicios externos.
- **Parte III (8–9)**: modelo de datos y flujos de consistencia.
- **Parte IV (10–14)**: organización del código, seguridad, servicios, errores y concurrencia.
- **Parte V (15–26)**: referencia de la API y matriz completa de endpoints.
- **Parte VI (27–31)**: despliegue, operación, pruebas, troubleshooting y mantenimiento.
- **Parte VII (32)**: evaluación de la calidad del sistema frente a normas ISO.
- **Anexos**: plantilla de entorno, catálogo de endpoints, SQL, configuraciones de servidor, errores y referencias.

NOTA

Nota. Los ejemplos de este manual usan datos ficticios. Nunca se reproducen contraseñas, claves SMTP, secretos de Cloudinary ni credenciales de producción.

2. Arquitectura del sistema

KAFFA usa una arquitectura **desacoplada cliente-servidor**. El backend no renderiza la aplicación: expone una API JSON versionada en `/api/v1`; la SPA y la app móvil son consumidores independientes. Esta separación permite cambiar la interfaz sin tocar las reglas de negocio y probar la API con `curl`, Postman o cualquier cliente HTTP.

2.1 Vista de contexto

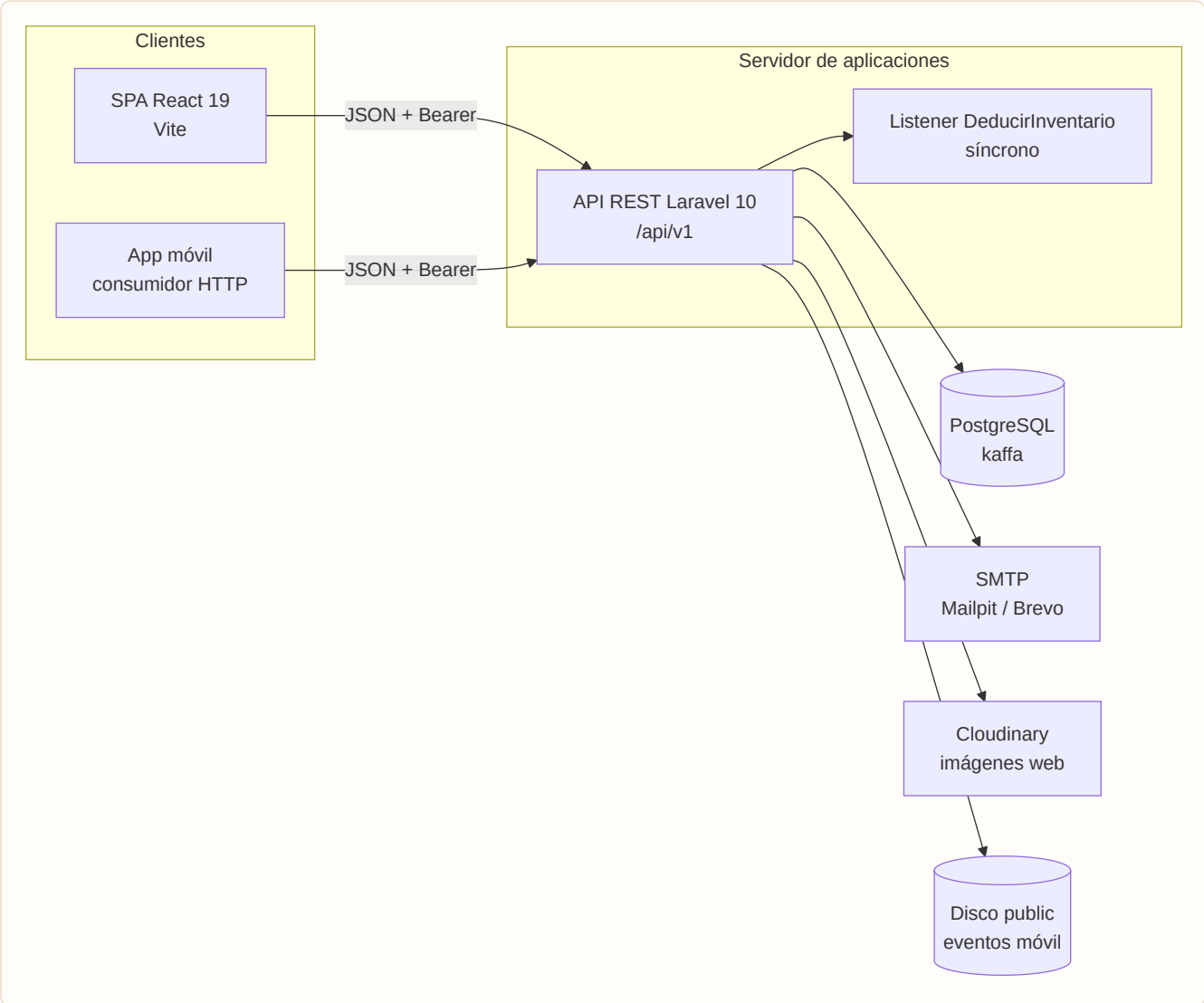


Figura 1. Contexto de despliegue lógico. Cloudinary recibe las imágenes desde el cliente web; el backend solo valida la URL. La excepción es `POST /eventos/imagen`, que guarda el archivo en el disco `public`.

2.2 Stack verificado

Tabla 4. *Stack tecnológico verificado*

Componente	Versión / requisito	Evidencia
PHP	`^8.1` (el entorno auditado usa 8.5.11)	`composer.json`
Laravel	`10.50.x`	`composer.lock`
Sanctum	`3.3.x`	`composer.json`
PostgreSQL	13+ recomendado; auditado con 18.6	`.env.example`, `phpunit.xml`
Composer	2.x	`composer.lock`
Node.js	20+ recomendado; auditado con 24.19	frontend
pnpm	gestor del frontend	`pnpm-lock.yaml`
PHPUnit	`^10.1`	`composer.json`
Scramble	`^0.13` (solo dev)	`composer.json`

El backend **no** usa Redis, colas externas ni WebSockets por defecto. `QUEUE_CONNECTION=sync` y `CACHE_DRIVER=file` son suficientes para desarrollo; producción puede escalar a Redis sin cambiar el código.

2.3 Capas del backend

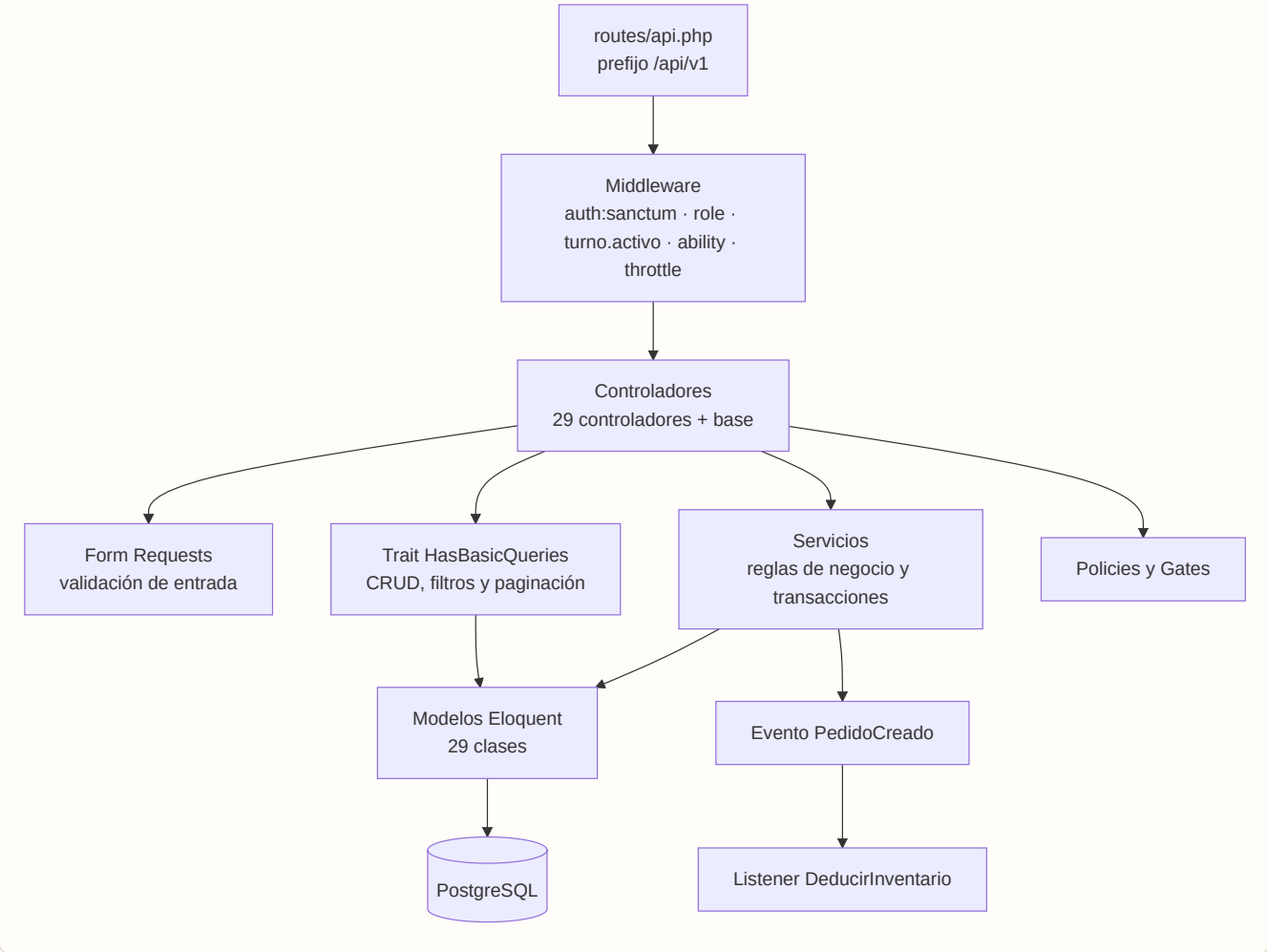


Figura 2. Capas del backend. La autorización se aplica en tres niveles: middleware de ruta, policies/gates y scopes de consulta.

Tabla 5. *Capas del backend y responsabilidades*

Capa	Responsabilidad	Ejemplo
Rutas	Declarar endpoint, método, middleware y nombre	`routes/api.php`
Middleware	Autenticación, rol, turno, ability y throttle	`CheckRole`, `EnsureActiveTurno`
Form Requests	Validar tipos, longitudes y reglas condicionales	`StorePedidoRequest`
Controladores	Orquestar entrada/salida HTTP; delegan reglas	`PedidoController`
Servicios	Transacciones, stock, caja, facturación	`PedidoService`
Modelos	Persistencia, relaciones, casts, scopes	`Pedido`, `Caja`
Policies/Gates	Autorización fina por registro	`PedidoPolicy`
Recursos	Serialización uniforme y ocultamiento de datos	`BaseResource`
Eventos/Listeners	Efectos secundarios desacoplados	`PedidoCreado` → `DeducirInventario`

2.4 Ciclo de una petición

1. El cliente envía `https://host/api/v1/recurso` con `Authorization: Bearer {token}`.
2. El grupo `api` aplica el throttle global de 60/min por usuario o IP.
3. `auth:sanctum` valida el token y carga el usuario.
4. `role:` comprueba que el usuario tenga alguno de los roles permitidos.
5. `turno.activo` se aplica solo a baristas y valida horario/asignación.
6. `ability:` comprueba que el token tenga el alcance requerido.
7. El Form Request valida el cuerpo; si falla, responde 422 con el *bag* de errores.
8. El controlador llama al servicio o al trait de consulta.
9. El servicio abre una transacción, bloquea filas críticas y aplica reglas.
10. El modelo persiste; los eventos síncronos ejecutan efectos (descuento de stock).
11. `BaseResource` serializa y oculta campos sensibles.
12. El `Handler` transforma excepciones en JSON uniforme.

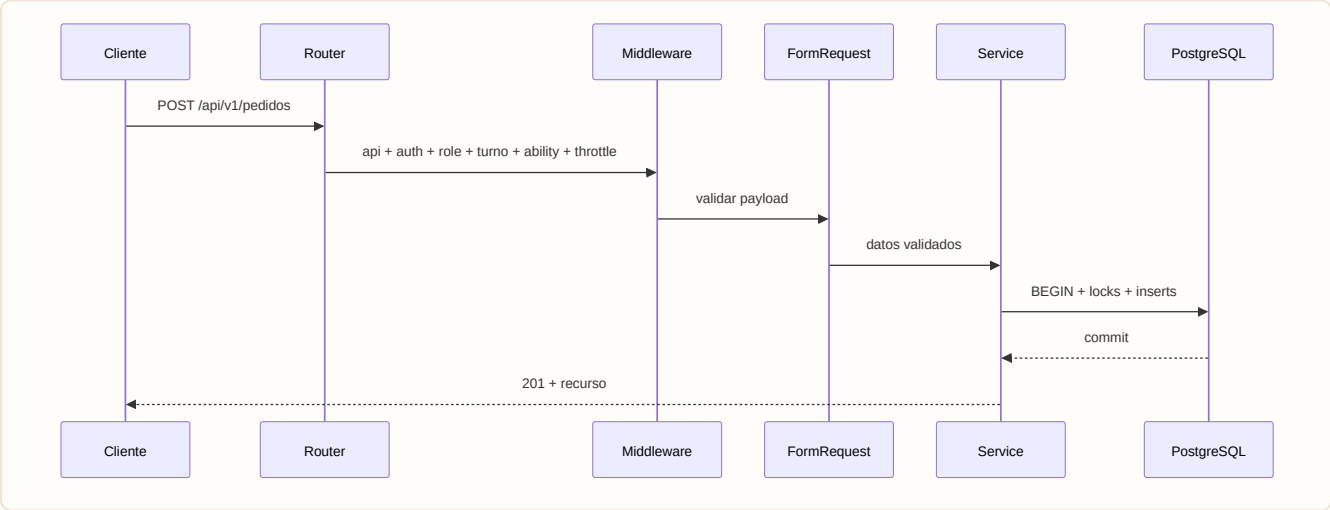


Figura 3. Ciclo de una petición de escritura.

2.5 Estructura de carpetas

```
backend-kaffa/
├── app/
│   ├── Events/               PedidoCreado
│   ├── Exceptions/           BusinessRuleException, Handler
│   ├── Http/
│   │   ├── Controllers/      29 controladores de dominio + `Controller` base
│   │   ├── Middleware/       CheckRole, EnsureActiveTurno, EnsureTokenCanAbility
│   │   ├── Requests/         Store*/Update* por recurso
│   │   └── Resources/        BaseResource
│   ├── Listeners/            DeducirInventario
│   ├── Mail/                 CodigoOtpMail
│   ├── Models/               29 modelos + Concerns/HasBasicQueries
│   ├── Policies/             17 policies de dominio + BasePolicy
│   ├── Providers/            AuthServiceProvider, RouteServiceProvider
│   ├── Rules/                CloudinaryUrl
│   └── Services/             10 servicios de negocio
├── config/                   app, auth, cors, database, mail, otp, sanctum, services
├── database/
```

```
│   ├── factories/              UserFactory
│   ├── migrations/             44 migraciones
│   └── seeders/                Rol, MedioPago, User, Turno, Catalog
├── routes/api.php              API versionada
├── storage/app/public/eventos  imágenes subidas desde móvil
├── tests/Feature/              13 archivos de pruebas
└── .env.example                plantilla de configuración
```

Figura 4. Estructura del backend.

2.6 Decisiones de diseño relevantes

2.6.1 API versionada por prefijo

El prefijo `api/v1` se declara en `RouteServiceProvider`. No hay negociación por cabecera ni versionado en el dominio. Para una ruptura incompatible, se recomienda crear `api/v2` y mantener `v1` durante una ventana de transición.

2.6.2 Token Bearer puro

La API no usa sesiones por cookies ni CSRF. `EnsureFrontendRequestsAreStateful` está comentado en `Kernel.php`; `supports_credentials` es `false`. Esto simplifica clientes móviles y evita depender de dominios stateful de Sanctum. La contrapartida es que el token debe protegerse en el cliente (almacenamiento seguro, HTTPS, expiración).

2.6.3 Lógica de negocio en servicios

Los controladores son delgados. Las operaciones que cruzan varias tablas (pedido, compra, gasto, merma, factura, turno) viven en `app/Services` y se ejecutan dentro de `TransactionService::run`, que reintenta ante deadlocks. Esto evita duplicar reglas entre web y móvil.

2.6.4 Autorización en profundidad

No basta con `role:`. Las rutas de escritura exigen `ability:` y los modelos aplican policies/gates. Los scopes (`Pedido::scopeByUser`, `CajaPolicy`, `FacturaVentaPolicy`) limitan los datos que un cliente o barista puede ver. Esta triple barrera está documentada en el capítulo 11.

2.6.5 Soft delete selectivo

Las entidades de negocio conservan `deleted_at`; las tablas de detalle y movimientos se borran físicamente cuando el servicio lo decide. El soft delete no restaura stock por sí solo: `PedidoService::eliminar` no repone inventario, mientras que `CompraService` y `MermaService` sí revierten sus efectos. Es una decisión deliberada que el operador debe conocer.

2.6.6 Imágenes: dos caminos

- **Web**: el navegador sube a Cloudinary con un preset *unsigned*; el backend recibe y valida la `secure_url` con `App\Rules\CloudinaryUrl`.
- **Móvil**: `POST /eventos/imagen` recibe multipart, guarda en `storage/app/public/eventos` y devuelve una URL absoluta.

No hay endpoint genérico de subida de comprobantes. Si se migra a subidas firmadas, el backend debe emitir la firma y nunca exponer el API secret.

2.7 Integración con los clientes

Tabla 6. *Integración del backend con la SPA y la app móvil*

Aspecto	SPA web	App móvil
Base URL	`/api/v1` relativa, proxy Vite o Apache	URL absoluta del backend
Autenticación	`Authorization: Bearer`	Igual
Persistencia del token	`localStorage` (`kaffaToken`)	Almacenamiento seguro del sistema
Actualización de datos	Sondeo 5–60 s	Puede usar sondeo o refresh manual
Catálogo público	`GET /productos` , `/categorias` sin sesión	Igual, para Home de invitado
Imágenes	Cloudinary directo	Cloudinary o `POST /eventos/imagen`
Errores	Lee `message` , `errors` y `code`	Igual; útil para `FUERA_HORARIO` , `TOKEN_SIN_ABILITY`

RECOMENDACIÓN

Recomendación. Ningún cliente debe replicar reglas de negocio críticas (stock, suma de pagos, arqueo). Puede anticiparlas para mejorar la experiencia, pero el backend siempre debe validarlas.

2.8 Topología de desarrollo y producción

Desarrollo verificado

```
:5173  Vite dev server (SPA)
:8000  php artisan serve (API)
:5432  PostgreSQL (kaffa)
:1025  Mailpit SMTP (opcional)
:8025  Mailpit UI (opcional)
```

Producción prevista

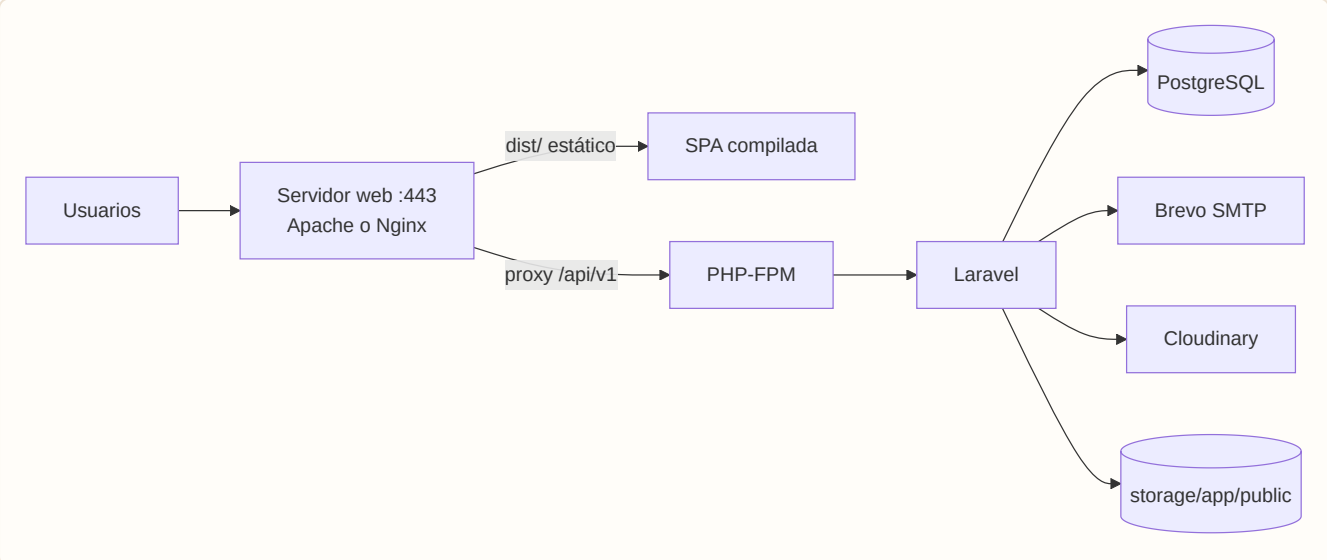


Figura 5. Topología de producción recomendada. No hay evidencia de un despliegue real en el repositorio; el capítulo 27 detalla los pasos.

2.9 Restricciones y riesgos arquitectónicos

Tabla 7. *Riesgos arquitectónicos y mitigaciones*

Riesgo	Impacto	Mitigación
Sin websockets	La UI puede mostrar datos con retraso	Sondeo configurable; no reducir por debajo del throttle
`QUEUE_CONNECTION=sync`	El envío de correo bloquea la petición	`MAIL_TIMEOUT=10`; en producción usar cola Redis
Cloudinary *unsigned*	Cualquiera con el preset puede subir archivos	Restringir preset, monitorear cuota, migrar a firma
Token en `localStorage`	Robo por XSS	CSP, sanitización, HTTPS, expiración corta y refresh
Sin idempotencia en pedidos	Un reintento de red puede duplicar una venta	Implementar `Idempotency-Key` en el futuro
Stock descontado por evento síncrono	Si el listener falla, la transacción se revierte	Mantener el listener dentro de la transacción y probar
Factura interna	No sirve como documento fiscal	Integrar proveedor autorizado si se requiere

ADVERTENCIA

Advertencia. El sistema es robusto para la operación de una cafetería, pero no es un ERP fiscal. No se debe prometer cumplimiento tributario sin un módulo de facturación electrónica certificado.

NOTA

Evaluación de calidad. La descripción arquitectónica de este capítulo se compara con ISO/IEC/IEEE 42010 en el capítulo 32 (§32.7).

3. Requisitos de instalación

Este capítulo enumera **todo lo que debe existir antes de ejecutar la primera migración**. La lista distingue entre requisitos verificados en el código y recomendaciones de operación. Si falta un componente, la instalación fallará en un punto identificable; la columna “síntoma” ayuda a diagnosticarlo.

3.1 Resumen ejecutivo

Tabla 8. *Requisitos mínimos y recomendados por categoría*

Categoría	Mínimo	Recomendado
Sistema operativo	Linux x86_64 (Ubuntu/Debian, Fedora, Arch)	Ubuntu 22.04/24.04 LTS
CPU	2 vCPU	4 vCPU
RAM	2 GB	4–8 GB
Disco	10 GB libres	40 GB SSD + respaldos
PHP	8.1	8.2 u 8.3 (estable)
PostgreSQL	13	15 o 16
Composer	2.x	última 2.x
Node.js (solo frontend)	18	20/22 LTS
pnpm (solo frontend)	8	9+
Servidor web	—	Nginx o Apache con PHP-FPM
TLS	—	Certificado válido (Let's Encrypt)

VERIFICADO

Verificado. El backend se ejecutó con PHP 8.5.11 y PostgreSQL 18.6 durante la auditoría. Sin embargo, para producción se recomienda una versión PHP con soporte activo y probada con Laravel 10 (8.2/8.3), no la última experimental.

3.2 Sistema operativo y usuario

El backend es una aplicación PHP estándar; no requiere extensiones de sistema exóticas. En Linux se recomienda:

- Crear un usuario de servicio sin privilegios (`kaffa`) para PHP-FPM.
- No ejecutar `php artisan` como `root`.
- Mantener el reloj sincronizado con NTP: la lógica de turnos y cierres depende de `America/Bogota`.
- Configurar `locale` UTF-8 (`es_CO.UTF-8` o `en_US.UTF-8`) para que PostgreSQL y los correos manejen tildes correctamente.

```
sudo adduser --system --group --home /var/www/kaffa kaffa
sudo timedatectl set-timezone America/Bogota
locale -a | grep -i utf
```

En Windows, el proyecto puede ejecutarse con Laragon, XAMPP o WSL2. Para producción no se recomienda Windows/IIS sin una validación específica de permisos y PHP-FPM.

3.3 PHP y extensiones

3.3.1 Versión

`composer.json` exige `"php": "^8.1"`. Laravel 10 es compatible con PHP 8.1–8.3 de forma estable. PHP 8.4/8.5 puede funcionar, pero genera avisos de deprecación en dependencias antiguas.

```
php -v
php -m
```

3.3.2 Extensiones necesarias

Tabla 9. *Extensiones PHP requeridas y opcionales*

Extensión	Uso	¿Obligatoria?
`pdo_pgsql`	Conexión a PostgreSQL	**Sí**
`pgsql`	Funciones PostgreSQL de bajo nivel	**Sí**
`mbstring`	Cadenas UTF-8	**Sí**
`openssl`	HTTPS, cifrado, tokens	**Sí**
`tokenizer`	Laravel/Composer	**Sí**
`xml` / `dom`	PHPUnit y dependencias	**Sí**
`ctype`, `json`, `filter`	Núcleo de Laravel	**Sí**
`bcmath`	Cálculos monetarios auxiliares	Recomendada
`fileinfo`	Validación de archivos	**Sí**
`curl`	Cliente HTTP (Guzzle)	**Sí**
`gd` o `imagick`	Procesamiento de imágenes local	Opcional
`intl`	Formateo internacional	Recomendada
`zip`	Composer y despliegues	Recomendada

Instalación en Debian/Ubuntu:

```
sudo apt update
sudo apt install -y php8.2-cli php8.2-fpm php8.2-pgsql php8.2-mbstring \
  php8.2-xml php8.2-curl php8.2-bcmath php8.2-zip php8.2-intl \
  php8.2-gd unzip git curl
```

Verificación:

```
php -r "var_dump(extension_loaded('pdo_pgsql'), extension_loaded('mbstring'));"
```

> **ADVERTENCIA**
>
> **Advertencia.** El error `could not find driver` al migrar casi siempre significa que falta `pdo_pgsql`, no que PostgreSQL esté apagado.

3.3.3 Límites de PHP recomendados

```
memory_limit = 256M
upload_max_filesize = 10M
post_max_size = 12M
max_execution_time = 60
date.timezone = America/Bogota
```

El backend acepta imágenes de eventos de hasta 5 MB; el límite de 10 MB deja margen para multipart.

3.4 PostgreSQL

3.4.1 Versión y codificación

Se recomienda PostgreSQL 13 o superior. La base debe crearse con UTF-8 y, preferiblemente, `LC_COLLATE` español o `C` para ordenamientos predecibles.

```
CREATE ROLE kaffa_app WITH LOGIN PASSWORD '<contraseña-segura>';
CREATE DATABASE kaffa
  WITH OWNER kaffa_app
       ENCODING 'UTF8'
       LC_COLLATE 'es_CO.UTF-8'
       LC_CTYPE 'es_CO.UTF-8'
       TEMPLATE template0;
CREATE DATABASE kaffa_test
  WITH OWNER kaffa_app
       ENCODING 'UTF8'
       TEMPLATE template0;
```

Si `es_CO.UTF-8` no está instalado, use `C.UTF-8` o `en_US.UTF-8`; no mezcle collations entre bases.

3.4.2 Acceso y permisos

- El usuario de la aplicación debe ser dueño de la base (para migraciones) o tener permisos `CREATE`, `ALTER`, `INSERT`, `UPDATE`, `DELETE`, `SELECT` y uso de secuencias.
- No use el superusuario `postgres` en producción.
- Restrinja `pg_hba.conf` a la red de la aplicación; no exponga el puerto 5432 a Internet.
- Si la aplicación y la base están en hosts distintos, habilite TLS (`sslmode=require`).

3.4.3 Zona horaria

Laravel guarda timestamps en UTC por convención y la aplicación opera en `America/Bogota`. No cambie la zona horaria de PostgreSQL sin revisar los cálculos de turnos y reportes. La zona se define en `config/app.php` y `TurnoService`.

3.5 Composer y dependencias PHP

```
composer --version
cd backend-kaffa
composer install --no-interaction --prefer-dist
```

Para producción:

```
composer install --no-dev --no-interaction --prefer-dist --optimize-autoloader
```

Tabla 10. *Dependencias PHP declaradas en composer.json*

Paquete	Uso
`laravel/framework`	Núcleo
`laravel/sanctum`	Tokens Bearer
`guzzlehttp/guzzle`	Cliente HTTP
`laravel/tinker`	Consola interactiva (dev)
`dedoc/scramble`	OpenAPI (dev)
`phpunit/phpunit`	Pruebas (dev)

> NOTA
>
> **Nota.** `composer.lock` debe versionarse. No ejecute `composer update` en producción sin una ventana de pruebas.

3.6 Node.js, pnpm y frontend

El backend no necesita Node para funcionar. Solo se requiere si va a compilar la SPA:

```
node -v
pnpm -v
cd REACT_clean/REACT_kaffa
pnpm install
pnpm run build
```

Tabla 11. *Herramientas de Node.js para el frontend*

Herramienta	Uso
Node.js 20/22 LTS	Build de Vite
pnpm 9+	Gestor único (`pnpm-lock.yaml`)
Vite	Servidor de desarrollo y build
ESLint	Calidad de código (no hay tests de frontend)

No use `npm install` en paralelo: el proyecto fijó `pnpm-lock.yaml` como fuente única. Si aparece un `package-lock.json`, elimínelo para evitar resoluciones divergentes.

3.7 Puertos y red

Tabla 12. *Puertos y exposición de red*

Puerto	Servicio	Exposición
80/443	Servidor web	Público
8000	`php artisan serve`	Solo desarrollo/local
5173	Vite dev server	Solo desarrollo/local
5432	PostgreSQL	Solo red interna
1025	Mailpit SMTP	Solo desarrollo/local
8025	Mailpit UI	Solo desarrollo/local
6379	Redis (opcional)	Solo red interna

Reglas de firewall recomendadas:

```
sudo ufw allow 22/tcp
sudo ufw allow 80,443/tcp
sudo ufw deny 5432/tcp
sudo ufw enable
```

3.8 Dominio, DNS y TLS

- Un registro `A` / `AAAA` para el dominio de la API o de la SPA.
- Certificado TLS válido; los tokens viajan en cada cabecera.
- Redirección permanente de HTTP a HTTPS.
- `APP_URL` y `FRONTEND_URL` deben coincidir con el esquema y dominio reales.
- Si la SPA y la API comparten dominio, `/api/v1` se proxya al backend; si están en dominios distintos, configure `CORS_ALLOWED_ORIGINS` con el origen exacto.

3.9 Cuentas de servicios externos

Tabla 13. *Cuentas de servicios externos necesarias*

Servicio	¿Obligatorio?	Qué se necesita
SMTP	**Sí** para registro y reseteo	Host, puerto, usuario, contraseña, remitente verificado
Mailpit	Solo desarrollo	Contenedor o binario local; `MAIL_HOST=127.0.0.1` si no usa Docker
Brevo	Recomendado en producción	SMTP key, remitente verificado, host `smtp-relay.sendinblue.com`
Cloudinary	**Sí** para imágenes web	`cloud_name`, `upload_preset` unsigned; API key/secret solo si se firma
Gemini	No	Claves reservadas; el Asistente actual es local
Sentry/APM	No	Opcional para observabilidad

ADVERTENCIA

Advertencia. El preset *unsigned* de Cloudinary es público por diseño. Cualquiera que lo conozca puede subir archivos a su cuenta. Restrinja formato/tamaño en el preset y monitoree el consumo.

3.10 Checklist previo a la instalación

- ☐ PHP 8.1+ con `pdo_pgsql`, `mbstring`, `openssl`, `xml`, `curl`, `fileinfo`.
- ☐ Composer 2.x disponible.
- ☐ PostgreSQL 13+ en ejecución y accesible.
- ☐ Base `kaffa` y usuario dedicado creados.
- ☐ Reloj sincronizado y zona `America/Bogota`.
- ☐ Repositorio clonado con permisos de escritura en `storage/` y `bootstrap/cache/`.
- ☐ `.env` con credenciales válidas (nunca versionado).
- ☐ SMTP o Mailpit configurado.
- ☐ Cuenta Cloudinary y preset disponibles (si se usará la web).
- ☐ Dominio/TLS definidos (si es producción).

3.11 Matriz de compatibilidad

Tabla 14. *Matriz de compatibilidad de versiones*

Combinación	Estado	Nota
PHP 8.1 + PostgreSQL 13 + Laravel 10	Soportada	Mínimo teórico
PHP 8.2 + PostgreSQL 15/16 + Laravel 10	**Recomendada**	Equilibrio soporte/estabilidad
PHP 8.3 + PostgreSQL 16	Soportada	Verificar extensiones
PHP 8.4/8.5	Funcional con avisos	Puede requerir ajustes de dependencias
MySQL/MariaDB	**No soportada**	El esquema y `phpunit.xml` usan `pgsql`
SQLite	**No soportada**	Hay SQL específico de PostgreSQL (`GREATEST`, locks)
Windows + IIS	No verificada	Use WSL2 o un servidor Linux

RECOMENDACIÓN

Recomendación. No intente “probar” con SQLite. El listener de inventario usa `GREATEST` y los servicios usan `lockForUpdate`; el comportamiento no sería equivalente y las pruebas perderían valor.

4. Instalación paso a paso

Este capítulo describe una instalación reproducible en un entorno limpio. Al finalizar, la API debe responder en `http://localhost:8000/api/v1`, la base debe tener las 34 tablas y los datos semilla, y las pruebas deben ejecutarse sobre `kaffa_test`.

4.1 Obtener el código

```
cd /var/www
sudo git clone <url-del-repositorio> kaffa
sudo chown -R kaffa:kaffa kaffa
cd kaffa/backend-kaffa
```

La estructura esperada es:

```
kaffa/
├── backend-kaffa/
├── docs/
└── REACT_clean/REACT_kaffa/
```

> **NOTA**
>
> **Nota.** El repositorio puede entregarse como archivo comprimido. En ese caso, descomprima y verifique que `backend-kaffa/composer.json` exista antes de continuar.

4.2 Instalar dependencias PHP

```
composer install --no-interaction --prefer-dist
```

Esto crea `vendor/` y ejecuta `php artisan package:discover`. Si falla por memoria:

```
COMPOSER_MEMORY_LIMIT=-1 composer install
```

4.3 Crear el archivo de entorno

```
cp .env.example .env
php artisan key:generate
```

`php artisan key:generate` escribe `APP_KEY`. Sin esa clave, Laravel no puede cifrar cookies, sesiones ni datos sensibles. **Nunca** comparta el `.env` ni lo versione.

Verifique que el archivo no esté en Git:

```
git check-ignore -v .env
git ls-files | grep -x '.env' || echo "OK: .env no versionado"
```

4.4 Configurar la base de datos

Edite `.env`:

```
APP_NAME=KAFFA
APP_ENV=local
APP_DEBUG=true
APP_URL=http://localhost:8000
FRONTEND_URL=http://localhost:5173

DB_CONNECTION=pgsql
DB_HOST=127.0.0.1
DB_PORT=5432
DB_DATABASE=kaffa
DB_USERNAME=kaffa_app
DB_PASSWORD=<tu-contraseña>
```

Cree la base y el usuario (si aún no existen):

```
sudo -u postgres psql <<'SQL'
CREATE ROLE kaffa_app WITH LOGIN PASSWORD 'cambia-esta-clave';
CREATE DATABASE kaffa OWNER kaffa_app ENCODING 'UTF8' TEMPLATE template0;
SQL
```

Pruebe la conexión antes de migrar:

```
psql "host=127.0.0.1 port=5432 dbname=kaffa user=kaffa_app password=<tu-contraseña>" -c "select
version();"
```

> **ADVERTENCIA**
>
> **Advertencia.** Si `DB_PASSWORD` queda vacía y PostgreSQL usa `scram-sha-256`, la conexión fallará. No copie credenciales de ejemplos ni de ramas antiguas.

4.5 Migrar y sembrar

```
php artisan migrate --seed
```

El comando ejecuta las 44 migraciones y luego, en orden:

1. `RolSeeder`: roles `admin`, `barista`, `cliente`.
2. `MedioPagoSeeder`: 6 medios (3 virtuales).
3. `UserSeeder`: usuarios de prueba verificados.
4. `TurnoSeeder`: turnos de hoy para todos los baristas.
5. `CatalogSeeder`: 3 categorías, 20 insumos, 10 productos y recetas.

Verificación rápida:

```
php artisan tinker --execute="echo \App\Models\User::count().' usuarios'.PHP_EOL;"
php artisan tinker --execute="echo \App\Models\Producto::count().' productos'.PHP_EOL;"
```

Para reconstruir desde cero en desarrollo:

```
php artisan migrate:fresh --seed
```

> **ADVERTENCIA**
>
> **Advertencia.** `migrate:fresh` elimina **todas** las tablas de la base configurada. Nunca lo ejecute en producción ni en una base con datos reales.

4.6 Enlazar el disco público

Las imágenes de eventos subidas desde el móvil se guardan en `storage/app/public/eventos`. Para que sean accesibles por URL:

```
php artisan storage:link
```

Esto crea `public/storage` como enlace simbólico. Verifique:

```
ls -l public/storage
```

Si el servidor web no sigue enlaces simbólicos, configure `disable_symlinks off` (Nginx) o `Options FollowSymLinks` (Apache).

4.7 Permisos de escritura

Laravel necesita escribir en:

- `storage/` (logs, caché, sesiones, archivos públicos)
- `bootstrap/cache/`

```
sudo chown -R kaffa:www-data storage bootstrap/cache
sudo find storage bootstrap/cache -type d -exec chmod 775 {} \;
sudo find storage bootstrap/cache -type f -exec chmod 664 {} \;
```

No use `chmod -R 777`; es un riesgo de seguridad y no es necesario.

4.8 Levantar el servidor de desarrollo

```
php artisan serve --host=127.0.0.1 --port=8000
```

La API queda en `http://127.0.0.1:8000/api/v1`. Pruebe:

```
curl -s http://127.0.0.1:8000/api/v1/productos | head -c 400
curl -s http://127.0.0.1:8000/api/v1/eventos | head -c 400
curl -s http://127.0.0.1:8000/api/v1/equipo | head -c 400
```

Las tres rutas son públicas y deben devolver JSON. Si recibe HTML o un error 500, revise `storage/logs/laravel.log`.

4.9 Probar el login

```
curl -s -X POST http://127.0.0.1:8000/api/v1/login \
  -H 'Content-Type: application/json' \
  -d '{"correo":"admin@kaffa.com","password":"admin123"}'
```

Respuesta esperada (resumida):

```
{
  "access_token": "1|...",
  "token_type": "Bearer",
  "usuario": { "id": 1, "nombre": "Administrador", "roles": [...] },
  "turno_activo": false,
  "turno_info": null
}
```

Guarde el token en una variable para las siguientes pruebas:

```
TOKEN=$(curl -s -X POST http://127.0.0.1:8000/api/v1/login \
  -H 'Content-Type: application/json' \
  -d '{"correo":"admin@kaffa.com","password":"admin123"}' | jq -r .access_token)
curl -s http://127.0.0.1:8000/api/v1/me -H "Authorization: Bearer $TOKEN" | jq
```

4.10 Usuarios de prueba

Tabla 15. *Usuarios de prueba creados por los seeders*

Rol	Correo	Contraseña	Acceso
Admin	`admin@kaffa.com`	`admin123`	Panel completo
Barista	`barista@kaffa.com`	`barista123`	POS, inventario, chat, caja
Cliente	`cliente@kaffa.com`	`cliente123`	Sitio público, pedidos, chat

> ADVERTENCIA
>
> **Advertencia.** Estas credenciales son de desarrollo y están sembradas en el código. En producción: elimine o cambie los usuarios semilla, use contraseñas fuertes y desactive `UserSeeder` / `CatalogSeeder` si no corresponden.

4.11 Frontend opcional

```
cd ../../REACT_clean/REACT_kaffa
pnpm install
pnpm run dev
```

La SPA queda en `http://localhost:5173` y proxya `/api` a `http://localhost:8000` (`vite.config.js`). Para producción:

```
pnpm run build
# publicar dist/ en el servidor web
```

4.12 Base de pruebas y PHPUnit

`phpunit.xml` usa PostgreSQL con base `kaffa_test`, `BCRYPT_ROUNDS=4` y caché/cola/sesión en `array`. Cree la base una vez:

```
sudo -u postgres createdb -O kaffa_app kaffa_test
```

Ejecute:

```
php artisan test
# o
vendor/bin/phpunit
```

Las pruebas usan `RefreshDatabase` y migran por ejecución. No apuntan a `kaffa`: es seguro ejecutarlas, pero consumen tiempo y conexiones.

4.13 Smoke test completo

Tabla 16. *Smoke test posterior a la instalación*

#	Prueba	Comando / acción	Resultado esperado
1	Catálogo público	`GET /productos` sin token	200 con `{data, links, meta}`
2	Login admin	`POST /login`	200 con `access_token`
3	Perfil	`GET /me` con Bearer	200 con roles
4	Recurso protegido sin token	`GET /insumos`	401 JSON
5	Catálogo admin	`POST /categorias` con token	201
6	Turno activo	`GET /turno-activo`	200 con `turno_activo`
7	Caja	`POST /cajas` con `monto_apertura_fisico`	201 con `estado=abierta`
8	Pedido	`POST /pedidos` con detalles y pago	201, stock descontado
9	Correo	Registro público	201 y código en Mailpit/Brevo
10	OpenAPI	`GET /docs/api` en local	UI de Scramble

4.14 Problemas comunes de instalación

Tabla 17. *Problemas comunes de instalación y solución*

Síntoma	Causa probable	Solución
`could not find driver`	Falta `pdo_pgsql`	Instalar extensión y reiniciar PHP-FPM
`SQLSTATE[08006]`	Host/puerto/credenciales incorrectos	Revisar `DB_*`; probar con `psql`
`No application encryption key`	Falta `APP_KEY`	`php artisan key:generate`

Síntoma	Causa probable	Solución
`Permission denied` en storage	Propietario incorrecto	`chown` a usuario de PHP-FPM
`404` en `/api/v1`	DocumentRoot apunta a otra carpeta	Apuntar a `backend-kaffa/public`
`419` o CSRF	Cliente intenta flujo por cookies	Usar Bearer; la API no usa CSRF
`429` al listar	Throttle compartido	Esperar; no reducir el límite sin analizar
Correo no llega	Mailpit no está en `mailpit` o SMTP mal configurado	Usar `127.0.0.1` o configurar Brevo
`storage:link` falla	Enlace existente	Borrar `public/storage` y repetir
`migrate` falla por extensión	Falta `bcmath` / `xml`	Instalar extensiones requeridas

RECOMENDACIÓN

Recomendación. Automatice este capítulo en un script idempotente (`scripts/install.sh`) y valide con un `healthcheck` que compruebe base, correo y almacenamiento. El repositorio actual no incluye Docker ni CI; un script reduce el error humano.

5. Configuración del backend

La configuración se concentra en `.env` y en `config/*.php`. Este capítulo documenta **cada variable que el proyecto usa**, su valor de desarrollo, el valor recomendado en producción y el efecto de cambiarla. No es una lista exhaustiva de la plantilla de Laravel: solo se incluyen variables con impacto real en KAFFA.

5.1 Regla de oro

Tabla 18. *Gestión de secretos por entorno*

Entorno	`.env`	Secretos
Desarrollo	`.env` local, `APP_DEBUG=true`	Pueden ser placeholders
Producción	`.env` en el servidor, permisos `640`, propietario del servicio	Nunca en Git, nunca en el frontend

```
chmod 640 .env
chown kaffa:www-data .env
```

Laravel lee `.env` al arrancar. Después de cambiarlo:

```
php artisan config:clear
php artisan config:cache   # solo producción
```

ADVERTENCIA

Advertencia. `php artisan config:cache` congela los valores. Si cambia `.env` y no limpia la caché, seguirá viendo la configuración anterior.

5.2 Variables de aplicación

Tabla 19. *Variables de aplicación*

Variable	Dev	Producción	Efecto
`APP_NAME`	`Laravel`	`KAFFA`	Nombre en correos y logs
`APP_ENV`	`local`	`production`	Comportamiento de Laravel
`APP_KEY`	generada	generada y respaldada	Cifrado; no cambiar sin migrar datos cifrados
`APP_DEBUG`	`true`	**`false`**	Con `true` expone trazas y rutas internas
`APP_URL`	`http://localhost`	`https://api.dominio.com`	URL base para enlaces y assets
`FRONTEND_URL`	`http://localhost:5173`	`https://app.dominio.com`	Contexto de correos y redirecciones
`LOG_CHANNEL`	`stack`	`stack` o `daily`	Canal de logs
`LOG_LEVEL`	`debug`	`warning` / `error`	Volumen de logs
`TIMEZONE`	`America/Bogota` (config)	Igual	Zona de la aplicación

`config/app.php` fija `timezone => 'America/Bogota'`. No confíe solo en la zona del sistema operativo; Laravel usa esta configuración para `now()`, turnos y facturas.

5.3 Base de datos

```
DB_CONNECTION=pgsql
DB_HOST=127.0.0.1
DB_PORT=5432
DB_DATABASE=kaffa
DB_USERNAME=kaffa_app
DB_PASSWORD=<secreto>
```

Tabla 20. *Variables de conexión a la base de datos*

Variable	Nota
`DB_CONNECTION`	Debe ser `pgsql`; el proyecto no soporta MySQL/SQLite
`DB_HOST`	`127.0.0.1` o host interno; nunca expuesto a Internet
`DB_PORT`	`5432` por defecto
`DB_DATABASE`	`kaffa`; tests usan `kaffa_test`
`DB_USERNAME`	Usuario dedicado, no `postgres`
`DB_PASSWORD`	Obligatoria en producción

Opcionales para TLS:

```
DB_SSLMODE=require
```

Si usa un pooler (PgBouncer), revise el modo de transacción: los `lockForUpdate` y las transacciones largas pueden comportarse distinto.

5.4 Sanctum y sesión

```
SANCTUM_EXPIRATION=1440
SESSION_DRIVER=file
SESSION_LIFETIME=120
```

Tabla 21. *Variables de Sanctum y sesión*

Variable	Dev	Producción	Efecto
`SANCTUM_EXPIRATION`	`1440`	`1440` o menos	Minutos de vida del token; `null` = sin expiración
`SESSION_DRIVER`	`file`	`redis` o `file`	Solo afecta flujos web; la API usa tokens
`SESSION_LIFETIME`	`120`	`120`	Minutos de sesión web

`config/sanctum.php` lee `SANCTUM_EXPIRATION` y lo aplica al crear tokens. Si el valor es `null`, el token no expira: es cómodo en desarrollo y peligroso en producción. `POST /auth/refresh` rota el token sin pedir credenciales.

> **ADVERTENCIA**
>
> **Advertencia.** La documentación antigua decía “por defecto no expira”; el `.env.example` actual fija 1440 minutos. Verifique siempre el valor efectivo con `php artisan tinker --execute="echo config('sanctum.expiration');"`.

5.5 CORS

```
CORS_ALLOWED_ORIGINS=http://localhost:5173
```

`config/cors.php`:

- Rutas: `api/*`, `sanctum/csrf-cookie`.
- Métodos: `GET, POST, PUT, PATCH, DELETE, OPTIONS`.
- Orígenes: lista separada por comas desde `CORS_ALLOWED_ORIGINS`.
- `supports_credentials = false`.

En producción:

```
CORS_ALLOWED_ORIGINS=https://app.dominio.com,https://admin.dominio.com
```

Nunca use `*` si la API maneja datos de negocio. Si la SPA y la API comparten dominio, CORS no interviene; el proxy de `/api` evita el preflight.

5.6 OTP y verificación

```
OTP_LENGTH=6
OTP_EXPIRA_MINUTOS=15
OTP_MAX_INTENTOS=5
OTP_REENVIO_SEGUNDOS=60
```

Tabla 22. *Variables del servicio OTP*

Variable	Efecto	Rango seguro
`OTP_LENGTH`	Dígitos del código	6 (mínimo 4)
`OTP_EXPIRA_MINUTOS`	Vigencia	10–15
`OTP_MAX_INTENTOS`	Intentos fallidos antes de bloquear	3–5
`OTP_REENVIO_SEGUNDOS`	Cooldown entre envíos	60

`config/otp.php` aplica mínimos: longitud ≥4, expiración ≥1, intentos ≥1, reenvío ≥0. Los códigos se guardan hasheados en `codigos_otp` y se eliminan al validarse.

5.7 Correo

5.7.1 Desarrollo con Mailpit

```
MAIL_MAILER=smtp
MAIL_HOST=127.0.0.1
MAIL_PORT=1025
MAIL_USERNAME=null
MAIL_PASSWORD=null
MAIL_ENCRYPTION=null
MAIL_FROM_ADDRESS="hello@example.com"
MAIL_FROM_NAME="KAFFA"
MAIL_TIMEOUT=10
MAIL_VERIFY_PEER=true
```

Si Mailpit corre en Docker y el backend en el host, use `MAIL_HOST=127.0.0.1` y publique el puerto 1025. Si ambos están en la misma red Docker, use `mailpit`.

Alternativa sin SMTP:

```
MAIL_MAILER=log
```

Los correos se escriben en `storage/logs/laravel.log`; útil para probar OTP sin servidor.

5.7.2 Producción con Brevo

```
MAIL_MAILER=smtp
MAIL_HOST=smtp-relay.sendinblue.com
MAIL_PORT=587
MAIL_ENCRYPTION=tls
MAIL_USERNAME=<login>@smtp-brevo.com
MAIL_PASSWORD=<smtp-key>
MAIL_FROM_ADDRESS=<remitente-verificado>
MAIL_FROM_NAME="KAFFA"
MAIL_TIMEOUT=10
MAIL_VERIFY_PEER=true
```

Brevo exige un remitente verificado y una **SMTP key** (no la API key). Use `smtp-relay.sendinblue.com` porque su certificado TLS coincide con el host; con `smtp-relay.brevo.com` algunos nodos regionales presentan un CN distinto y fallan con `Peer certificate CN ... did not match`.

> **ADVERTENCIA**
>
> **Advertencia.** `MAIL_VERIFY_PEER=false` desactiva la validación TLS. Úselo solo como diagnóstico temporal, nunca como configuración permanente.

5.8 Cloudinary

```
CLOUDINARY_CLOUD_NAME=roffd6et
CLOUDINARY_UPLOAD_PRESET=KAFFACCYS
CLOUDINARY_FOLDER=samples/ecommerce
CLOUDINARY_API_KEY=
CLOUDINARY_API_SECRET=
```

Tabla 23. *Variables de Cloudinary*

Variable	Uso	Exposición
`CLOUDINARY_CLOUD_NAME`	Cuenta de entrega	Pública
`CLOUDINARY_UPLOAD_PRESET`	Preset unsigned para el navegador	Pública por diseño
`CLOUDINARY_FOLDER`	Carpeta destino	Pública
`CLOUDINARY_API_KEY`	API firmada/Admin	Secreta
`CLOUDINARY_API_SECRET`	Firma/Admin	**Nunca** en el frontend

El backend solo valida que las URLs de `res.cloudinary.com` pertenezcan a la cuenta configurada (`App\Rules\CloudinaryUrl`). No recibe el archivo en el flujo web. La excepción es `POST /eventos/imagen`, que usa el disco local.

5.9 Caché, colas y sistema de archivos

```
CACHE_DRIVER=file
QUEUE_CONNECTION=sync
FILESYSTEM_DISK=local
```

Tabla 24. *Variables de caché, colas y almacenamiento*

Variable	Dev	Producción	Efecto
`CACHE_DRIVER`	`file`	`redis`	`Medio_Pago::virtualIds()` cachea 5 min
`QUEUE_CONNECTION`	`sync`	`redis` / `database`	Con `sync` los correos y listeners se ejecutan en la petición
`FILESYSTEM_DISK`	`local`	`local`	`storage/app`; el disco `public` se usa para eventos

Con `QUEUE_CONNECTION=sync`, un SMTP lento bloquea el registro. `MAIL_TIMEOUT=10` acota la espera. Si cambia a cola asíncrona, debe ejecutar un worker:

```
php artisan queue:work --tries=3 --timeout=60
```

No hay jobs programados en el código; si añade tareas, configure el scheduler y el cron.

5.10 Logging y errores

```
LOG_CHANNEL=stack
LOG_LEVEL=debug
```

En producción:

```
LOG_CHANNEL=daily
LOG_LEVEL=warning
LOG_DAILY_DAYS=14
```

Los fallos de envío de correo se registran explícitamente en `AuthController` con el correo y el error. No se registra la contraseña ni el código OTP en claro.

`Handler` devuelve JSON uniforme para 404 y `BusinessRuleException`. En `APP_DEBUG=true`, las trazas pueden incluir rutas y parámetros; no las exponga.

5.11 OpenAPI / Scramble

Scramble está en `require-dev`. La UI vive en `GET /docs/api` y el JSON en `/docs/api.json`.

```
SCRAMBLE_ENABLED=true
```

En producción:

```
SCRAMBLE_ENABLED=false
```

Para regenerar `docs/openapi.json`:

```
php artisan serve
curl -s http://localhost:8000/docs/api.json -o ../docs/openapi.json
```

RECOMENDACIÓN

Recomendación. No publique la UI de Scramble en producción. Si necesita documentación pública, genere el JSON en CI y publíquelo como archivo estático.

5.12 Variables reservadas

Tabla 25. *Variables reservadas o sin uso*

Variable	Estado	Nota
`GEMINI_API_KEY`	Reservada	El Asistente actual es local; no se usa
`GEMINI_MODEL`	Reservada	Solo si se conecta un LLM
`GEMINI_FALLBACK_MODEL`	Reservada	Ídem
`GEMINI_BASE_URL`	Reservada	Ídem
`AWS_*`	Plantilla	No se usa para imágenes
`PUSHER_*`	Plantilla	No hay websockets
`REDIS_*`	Opcional	Necesarias si usa Redis

5.13 Plantilla mínima de producción

```
APP_NAME=KAFFA
APP_ENV=production
APP_KEY=base64:...
APP_DEBUG=false
APP_URL=https://api.kaffa.example
FRONTEND_URL=https://app.kaffa.example

DB_CONNECTION=pgsql
DB_HOST=127.0.0.1
DB_PORT=5432
```

```
DB_DATABASE=kaffa
DB_USERNAME=kaffa_app
DB_PASSWORD=<secreto>

SANCTUM_EXPIRATION=1440
CORS_ALLOWED_ORIGINS=https://app.kaffa.example

CACHE_DRIVER=redis
QUEUE_CONNECTION=redis
SESSION_DRIVER=redis

MAIL_MAILER=smtp
MAIL_HOST=smtp-relay.sendinblue.com
MAIL_PORT=587
MAIL_ENCRYPTION=tls
MAIL_USERNAME=<login>@smtp-brevo.com
MAIL_PASSWORD=<smtp-key>
MAIL_FROM_ADDRESS=<remitente-verificado>
MAIL_FROM_NAME=KAFFA

LOG_CHANNEL=daily
LOG_LEVEL=warning
SCRAMBLE_ENABLED=false
```

5.14 Checklist de configuración

- ☐ `APP_DEBUG=false` y `APP_ENV=production`.
- ☐ `APP_KEY` respaldada en un gestor de secretos.
- ☐ `DB_PASSWORD` fuerte y usuario dedicado.
- ☐ `SANCTUM_EXPIRATION` definido.
- ☐ `CORS_ALLOWED_ORIGINS` con dominios exactos.
- ☐ SMTP probado con un registro real.
- ☐ Cloudinary con preset restringido.
- ☐ `LOG_CHANNEL` y rotación configurados.
- ☐ Scramble desactivado.
- ☐ `.env` con permisos `640` y fuera de Git.

6. Operación de la base de datos

Este capítulo cubre la base de datos como **servicio operativo**: creación, migraciones, seeders, respaldos, restauración, mantenimiento y consultas útiles. El modelo lógico y el diccionario completo están en el capítulo 8.

6.1 Motor y conexión

Tabla 26. *Parámetros de conexión a PostgreSQL*

Parámetro	Valor
Motor	PostgreSQL
Base principal	`kaffa`
Base de pruebas	`kaffa_test`
Puerto por defecto	`5432`
Codificación	UTF-8
Zona de aplicación	`America/Bogota`
Driver PHP	`pdo_pgsql`

El proyecto **no** soporta MySQL ni SQLite. El esquema usa tipos y funciones de PostgreSQL, y los tests configuran `pgsql` en `phpunit.xml`.

6.2 Crear la base desde cero

```
-- Como superusuario de PostgreSQL
CREATE ROLE kaffa_app WITH LOGIN PASSWORD '<secreto>';
CREATE DATABASE kaffa
  WITH OWNER kaffa_app
       ENCODING 'UTF8'
       LC_COLLATE 'C.UTF-8'
       LC_CTYPE 'C.UTF-8'
       TEMPLATE template0;
CREATE DATABASE kaffa_test
  WITH OWNER kaffa_app
       ENCODING 'UTF8'
       TEMPLATE template0;
```

Verificación:

```
psql -h 127.0.0.1 -U kaffa_app -d kaffa -c "\conninfo"
psql -h 127.0.0.1 -U kaffa_app -d kaffa -c "select current_database(), current_user;"
```

6.3 Migraciones

6.3.1 Ejecutar

```
php artisan migrate
```

6.3.2 Estado

```
php artisan migrate:status
```

6.3.3 Reconstruir en desarrollo

```
php artisan migrate:fresh --seed
```

6.3.4 Revertir la última tanda

```
php artisan migrate:rollback
```

6.3.5 Revertir todo

```
php artisan migrate:reset
```

> ADVERTENCIA
>
> **Advertencia.** `migrate:fresh`, `migrate:reset` y `db:wipe` destruyen datos. Úselos solo en desarrollo o en una base desechable. En producción, cada migración debe revisarse y probarse contra una copia.

6.3.6 Buenas prácticas para nuevas migraciones

1. Una migración = un cambio coherente.
2. Use `nullable()` explícito y valores por defecto seguros.
3. Añada índices para columnas de filtro/orden.
4. Para columnas nuevas en tablas con datos, haga backfill antes de `NOT NULL`.
5. Evite lógica de negocio en migraciones; use seeders o comandos.
6. Pruebe `up` y `down` en una base limpia.
7. No renombre tablas sin actualizar modelos, relaciones, policies y pruebas.

6.4 Seeders

Orden de `DatabaseSeeder`:

```
RolSeeder → MedioPagoSeeder → UserSeeder → TurnoSeeder → CatalogSeeder
```

Tabla 27. *Seeders de la base de datos*

Seeder	Datos	Idempotencia
`RolSeeder`	admin, barista, cliente	`firstOrCreate`
`MedioPagoSeeder`	6 medios (3 virtuales)	`firstOrCreate` por nombre
`UserSeeder`	3 usuarios verificados con foto	`firstOrCreate` por correo
`TurnoSeeder`	Turnos de hoy y asignación de baristas	Verifica existencia por fecha/tipo
`CatalogSeeder`	3 categorías, 20 insumos, 10 productos, recetas	`firstOrCreate`

```
php artisan db:seed
php artisan db:seed --class=CatalogSeeder
```

> **ADVERTENCIA**
>
> **Advertencia.** `TurnoSeeder` crea turnos para el día actual. Si la zona horaria del servidor no es `America/Bogota`, los turnos pueden quedar en el día equivocado.

6.5 Respaldos

6.5.1 Respaldo lógico completo

```
PGPASSWORD='<secreto>' pg_dump \
  -h 127.0.0.1 -U kaffa_app -d kaffa \
  -Fc -f /backups/kaffa_$(date +%F_%H%M).dump
```

`-Fc` genera un formato comprimido y flexible. Para un SQL plano:

```
pg_dump -h 127.0.0.1 -U kaffa_app -d kaffa -Fp -f kaffa.sql
```

6.5.2 Restauración

```
pg_restore -h 127.0.0.1 -U kaffa_app -d kaffa_restore --clean --if-exists kaffa_2026-10-20_0300.dump
```

Nunca restaure directamente sobre producción sin una ventana de mantenimiento y un respaldo previo.

6.5.3 Qué respaldar además de la base

Tabla 28. *Elementos a incluir en el respaldo*

Elemento	Motivo
`storage/app/public/eventos`	Imágenes subidas desde el móvil
`.env` (cifrado)	Configuración y secretos
`storage/logs`	Diagnóstico histórico
`docs/openapi.json`	Contrato de API versionado
Configuración del servidor web/PHP-FPM	Reproducibilidad

Cloudinary conserva las imágenes web; no necesita respaldo local, pero sí inventario de URLs.

6.5.4 Política recomendada

- Respaldo completo diario, retención 14–30 días.
- Respaldo incremental/WAL si el volumen lo justifica.
- Prueba de restauración mensual en un entorno aislado.
- Copia fuera del servidor (almacenamiento de objetos o servidor secundario).
- Cifrado en reposo y control de acceso.

6.6 Mantenimiento

6.6.1 Estadísticas y espacio

```
-- Actualizar estadísticas del planificador
ANALYZE;

-- Reclamar espacio de filas muertas (no bloquea lecturas)
VACUUM (VERBOSE, ANALYZE);

-- Revisar tamaño por tabla
SELECT relname AS tabla,
       pg_size_pretty(pg_total_relation_size(relid)) AS tamano
FROM pg_catalog.pg_statio_user_tables
ORDER BY pg_total_relation_size(relid) DESC;
```

6.6.2 Conexiones

```
SELECT count(*) AS conexiones,
       state
FROM pg_stat_activity
WHERE datname = 'kaffa'
GROUP BY state;
```

Si se agotan las conexiones, revise el pool de PHP-FPM y el número de workers antes de aumentar `max_connections`.

6.6.3 Índices sin uso

```
SELECT schemaname, relname AS tabla, indexrelname AS indice, idx_scan
FROM pg_stat_user_indexes
WHERE idx_scan = 0
ORDER BY pg_relation_size(indexrelid) DESC;
```

No elimine un índice solo porque `idx_scan=0`; puede ser estacional o proteger una restricción única.

6.6.4 Bloqueos

```
SELECT pid, usename, state, wait_event_type, wait_event, query
FROM pg_stat_activity
WHERE datname = 'kaffa' AND state <> 'idle'
ORDER BY query_start;
```

Los servicios usan `lockForUpdate` sobre stock, cajas y contadores. Un bloqueo largo suele indicar una transacción que no cierra o un deadlock reintentado.

6.7 Consultas útiles de operación

6.7.1 Ventas del día

```
SELECT count(*) AS pedidos,
       coalesce(sum(total), 0) AS total
FROM pedidos
WHERE deleted_at IS NULL
```

```
  AND estado IN ('pagado', 'entregado')
  AND created_at >= CURRENT_DATE;
```

6.7.2 Productos más vendidos

```
SELECT p.nombre,
       sum(d.cantidad) AS unidades,
       sum(d.subtotal) AS ingresos
FROM detalle__pedidos d
JOIN pedidos pe ON pe.id = d.pedido_id AND pe.deleted_at IS NULL
JOIN productos p ON p.id = d.producto_id
WHERE pe.estado IN ('pagado', 'entregado')
  AND pe.created_at >= CURRENT_DATE - INTERVAL '7 days'
GROUP BY p.nombre
ORDER BY unidades DESC
LIMIT 10;
```

6.7.3 Stock bajo

```
SELECT nombre, stock_actual, stock_minimo, unidad_medida
FROM insumos
WHERE deleted_at IS NULL
  AND stock_actual <= stock_minimo
ORDER BY (stock_actual - stock_minimo) ASC;
```

6.7.4 Cajas abiertas

```
SELECT c.id, u.nombre AS abierta_por, c.fecha_apertura, c.monto_apertura_fisico
FROM cajas c
JOIN users u ON u.id = c.abierta_por
WHERE c.estado = 'abierta' AND c.deleted_at IS NULL;
```

6.7.5 Arqueo de una caja

```
SELECT c.id,
       c.monto_apertura_sistema,
       coalesce(sum(CASE WHEN m.tipo = 'ingreso' THEN m.monto ELSE 0 END), 0) AS ingresos,
       coalesce(sum(CASE WHEN m.tipo = 'egreso' THEN m.monto ELSE 0 END), 0) AS egresos,
       c.monto_apertura_sistema
         + coalesce(sum(CASE WHEN m.tipo = 'ingreso' THEN m.monto ELSE 0 END), 0)
         - coalesce(sum(CASE WHEN m.tipo = 'egreso' THEN m.monto ELSE 0 END), 0) AS sistema
FROM cajas c
LEFT JOIN movimiento__cajas m ON m.caja_id = c.id
WHERE c.id = :caja_id
GROUP BY c.id;
```

6.7.6 Tokens activos

```
SELECT id, tokenable_id, name, last_used_at, expires_at
FROM personal_access_tokens
WHERE (expires_at IS NULL OR expires_at > now())
ORDER BY last_used_at DESC NULLS LAST;
```

6.7.7 Pendientes de registro

```
SELECT correo, nombre, created_at
FROM registros_pendientes
WHERE created_at < now() - INTERVAL '24 hours';
```

6.8 Integridad y mantenimiento de datos

- No borre filas de `movimiento__cajas` para “cuadrar” una caja: rompe la trazabilidad. Use movimientos de ajuste autorizados.
- No modifique `factura_contadores` manualmente; puede generar números duplicados.
- No elimine `personal_access_tokens` de un usuario activo sin avisar: cerrará su sesión.
- Los soft delete no liberan espacio; `VACUUM` no elimina filas vivas.
- Las tablas de detalle sin `deleted_at` se borran físicamente; revise las FKs antes de un `DELETE` manual.

6.9 Checklist de operación

- ☐ Respaldo diario verificado.
- ☐ Restauración probada en un entorno aislado.
- ☐ `ANALYZE` programado.
- ☐ Alertas de espacio en disco y conexiones.
- ☐ Índices revisados tras cambios de consulta.
- ☐ Zona horaria y NTP correctos.
- ☐ `storage/app/public/eventos` incluido en el respaldo.
- ☐ Credenciales de base rotadas según política.

7. Servicios externos

El backend depende de servicios externos para dos funciones críticas: **enviar códigos OTP** (registro y recuperación) y **alojar imágenes**. Este capítulo explica el contrato de cada servicio, cómo se configura, qué falla y cómo diagnosticarlo.

7.1 Mapa de dependencias

Tabla 29. *Mapa de dependencias externas*

Servicio	Función	Protocolo	¿Crítico?
SMTP / Brevo	Correos con códigos OTP	SMTP	Sí para altas y reseteos
Mailpit	Captura de correos en desarrollo	SMTP/HTTP	Solo desarrollo
Cloudinary	Imágenes web (productos, eventos, fotos, comprobantes)	HTTPS	Sí para flujo web con imágenes
Disco `public`	Imágenes de eventos subidas desde móvil	HTTP multipart	Solo para esa función
Gemini	LLM externo (reservado)	HTTPS	No; no se usa
CDNs del frontend	Chart.js, jsPDF, Font Awesome, Poppins	HTTPS	No; afecta la UI, no la API

7.2 Correo transaccional

7.2.1 Flujo de un código OTP

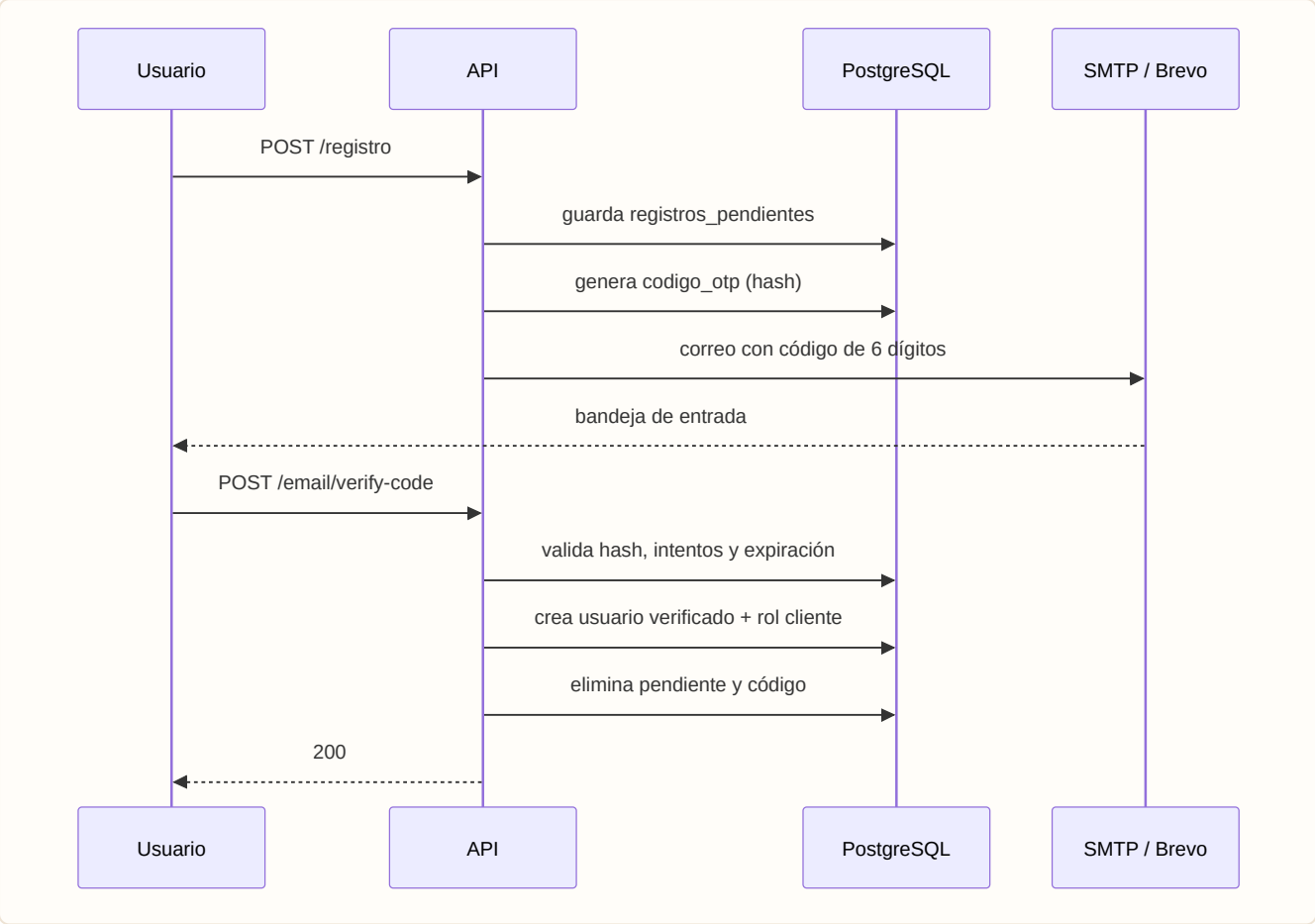


Figura 1. Verificación de correo por OTP. El código nunca se guarda en claro.

7.2.2 Reglas del código

Tabla 30. *Reglas y valores por defecto del código OTP*

Regla	Valor por defecto	Fuente
Longitud	6 dígitos	`OTP_LENGTH`
Vigencia	15 minutos	`OTP_EXPIRA_MINUTOS`
Intentos	5	`OTP_MAX_INTENTOS`
Reenvío	60 segundos	`OTP_REENVIO_SEGUNDOS`
Almacenamiento	Hash	`OtpService`
Uso	Un solo uso	Se elimina al validar
Pendiente	24 horas	`RegistrationService::purgarAntiguos`

7.2.3 Mailpit en desarrollo

```
MAIL_MAILER=smtp
MAIL_HOST=127.0.0.1
```

```
MAIL_PORT=1025
```

Interfaz: `http://localhost:8025`. Si Mailpit está en Docker y el backend en el host, publique `1025:1025` y `8025:8025`. Si el backend está en otra red, use el nombre del contenedor (`mailpit`).

Alternativa sin servidor:

```
MAIL_MAILER=log
```

El código aparece en `storage/logs/laravel.log`. No use esta opción en producción.

7.2.4 Brevo en producción

Checklist:

1. Crear cuenta y verificar el remitente (`Senders & IP → Senders`).
2. Generar una **SMTP key** (`SMTP & API → SMTP`), no una API key.
3. Configurar `MAIL_USERNAME` con el login SMTP (`<id>@smtp-brevo.com`).
4. Usar `smtp-relay.sendinblue.com:587` con `tls`.
5. Enviar un correo de prueba antes de abrir el registro al público.

```
php artisan tinker --execute="Mail::raw('Prueba KAFFA', fn(\$m) => \$m->to('tu-correo@example.com')->subject('Prueba SMTP'));"
```

7.2.5 Fallos típicos

Tabla 31. *Fallos típicos del correo transaccional*

Síntoma	Causa	Solución
`Connection could not be established`	Host/puerto incorrectos o firewall	Probar `telnet host 587`
`Peer certificate CN ... did not match`	Nodo regional de Brevo con CN distinto	Usar `smtp-relay.sendinblue.com`; no desactivar TLS
`535 Authentication failed`	SMTP key incorrecta o usuario mal formado	Regenerar credenciales
El correo llega a spam	Remitente no verificado/DKIM	Verificar dominio y remitente en Brevo
El registro responde 201 pero no llega nada	`MAIL_MAILER=log` o error silenciado	Revisar `storage/logs/laravel.log`
Timeout en registro	SMTP lento y cola `sync`	`MAIL_TIMEOUT=10`; usar cola Redis

ADVERTENCIA

Advertencia. El registro captura excepciones de envío y responde 201 para no filtrar información. Eso significa que un fallo de correo **no** se ve en la respuesta HTTP; solo en logs. Monitoree `storage/logs` o configure alertas.

7.3 Cloudinary

7.3.1 Modelo de subida

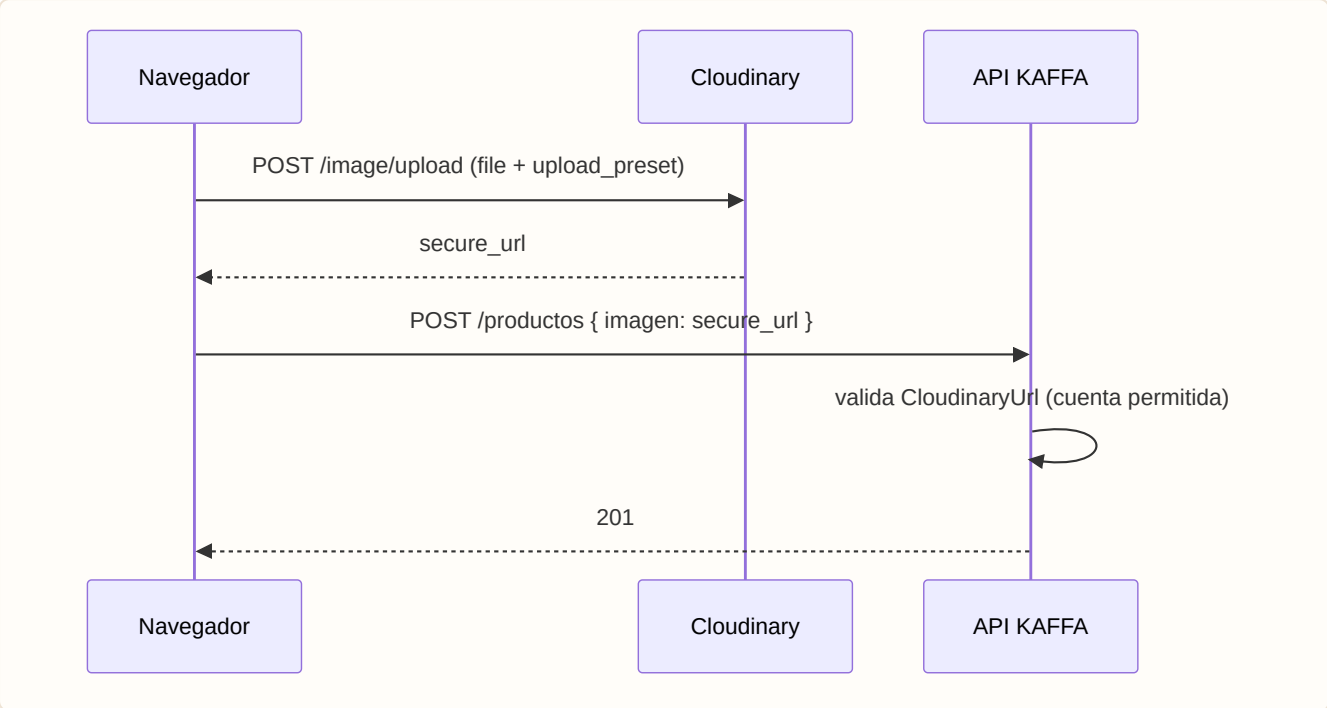


Figura 2. Subida web: el archivo no pasa por el backend.

7.3.2 Configuración

```
CLOUDINARY_CLOUD_NAME=roffd6et
CLOUDINARY_UPLOAD_PRESET=KAFFACCYS
CLOUDINARY_FOLDER=samples/ecommerce
```

El frontend usa `VITE_CLOUDINARY_CLOUD_NAME` y `VITE_CLOUDINARY_UPLOAD_PRESET`. Son valores públicos por diseño; el **API secret** nunca debe llegar al navegador.

7.3.3 Validación en el backend

`App\Rules\CloudinaryUrl`:

- Acepta URLs `https://res.cloudinary.com/<cloud>/...`.
- Exige que `<cloud>` coincida con `CLOUDINARY_CLOUD_NAME` (o el configurado).
- Rechaza otras cuentas y esquemas no HTTPS.
- Se usa en productos, eventos, foto de perfil y comprobantes.

Si la regla falla, el Form Request responde 422. No hay excepción para URLs de Unsplash u otros dominios: las imágenes heredadas deben migrarse o permitirse explícitamente en la regla.

7.3.4 Riesgos del preset unsigned

- Cualquiera con el preset puede subir archivos.
- Se puede consumir la cuota o llenar la cuenta con contenido no deseado.

- No hay control de usuario en Cloudinary.

Mitigaciones:

1. Restringir formatos y tamaño en el preset.
2. Usar una carpeta dedicada.
3. Monitorear uso y alertas de cuota.
4. Migrar a subida firmada: el backend genera `signature` y `timestamp` con el API secret; el navegador sube con esos valores.
5. Validar en el backend que la URL pertenece a la cuenta (ya implementado).

7.4 Disco público para eventos móviles

`POST /eventos/imagen` es la única ruta que recibe un archivo:

```
curl -X POST https://api.kaffa.example/api/v1/eventos/imagen \
  -H "Authorization: Bearer $TOKEN" \
  -F "imagen=@evento.jpg"
```

Respuesta:

```
{ "url": "https://api.kaffa.example/storage/eventos/abc123.jpg" }
```

Requisitos:

- `php artisan storage:link`.
- Permisos de escritura en `storage/app/public/eventos`.
- Validación de tipo y tamaño (≤ 5 MB).
- El cliente debe persistir la URL devuelta en `imagenes[]` del evento.

> **ADVERTENCIA**
>
> **Advertencia.** Si el servidor se reconstruye sin respaldar `storage/app/public/eventos`, las imágenes móviles se pierden aunque la base conserve la URL.

7.5 Gemini (reservado)

El Asistente KAFFA actual es **local**: interpreta intenciones por palabras clave y consulta la base. No llama a Gemini. Las variables `GEMINI_*` existen en `config/services.php` y `.env.example` para una integración futura, pero no se usan.

Si se conecta un LLM:

- Enviar solo métricas agregadas, nunca datos personales.
- Aplicar timeout y fallback.
- No registrar prompts con datos sensibles.
- Documentar el proveedor y el tratamiento de datos.
- Evaluar costos y latencia.

7.6 CDNs del frontend

La SPA carga Chart.js, jsPDF, Font Awesome y Google Fonts desde CDN. Esto no afecta al backend, pero sí a la disponibilidad de la interfaz. Si la cafetería opera con internet inestable, considere empaquetar esas dependencias en el build.

7.7 Matriz de fallos y degradación

Tabla 32. *Matriz de fallos y degradación por servicio*

Servicio caído	Efecto	Degradación posible
SMTP	No se envían OTP; registro/reseteo incompletos	Responder 201 y reintentar; monitorear logs
Cloudinary	No se suben imágenes web	Permitir URL manual temporal; no bloquear ventas en efectivo
Disco `public`	No se suben imágenes móviles	Usar Cloudinary o posponer la imagen
PostgreSQL	API no responde	Sin operación; restaurar servicio
Redis (si se usa)	Caché/colas fallan	Volver a `file` / `sync` temporalmente
Gemini	Sin efecto	El Asistente sigue local

7.8 Checklist de servicios externos

- ☐ Correo de prueba recibido en Mailpit/Brevo.
- ☐ Remitente verificado en Brevo.
- ☐ `MAIL_TIMEOUT` definido.
- ☐ Cloudinary con preset restringido.
- ☐ `CLOUDINARY_API_SECRET` fuera del frontend.
- ☐ `storage:link` creado y respaldado.
- ☐ Alertas de cuota de Cloudinary.
- ☐ Variables `GEMINI_*` vacías si no se usan.
- ☐ Documentación de contacto de cada proveedor.

8. Modelo de datos

El backend usa **PostgreSQL** como fuente de verdad transaccional. Este capítulo describe el motor, las convenciones de diseño, las 34 tablas, sus relaciones, el diccionario de datos completo y los seeders. Está pensado para que un ingeniero pueda escribir consultas, migraciones o integraciones sin adivinar nombres, tipos ni reglas de integridad.

> **VERIFICADO**
>
> **Verificado.** El esquema se reconstruye con `php artisan migrate:fresh --seed` sobre PostgreSQL. La conexión declarada en `.env.example` es `pgsql`, puerto `5432`, base `kaffa`; los tests usan `kaffa_test`.

8.0 Cómo leer el diccionario

Cada tabla se documenta con sus columnas, tipo Laravel/PostgreSQL, nulabilidad, claves y observaciones de negocio. Las claves foráneas se indican como `FK → tabla.columna`; los borrados lógicos se marcan con `deleted_at`. Los importes usan `decimal(10,2)` y se serializan con dos decimales.

8.1 Motor y convenciones

- **Motor**: PostgreSQL (`.env`: `DB_CONNECTION=pgsql`, `DB_HOST=127.0.0.1`, `DB_PORT=5432`, `DB_DATABASE=kaffa`). Los tests usan la base `kaffa_test` (`phpunit.xml`).
- **Zona horaria de la aplicación**: `America/Bogota` (`config/app.php`).
- **Nomenclatura**: tablas en español; las tabletas de detalle de pagos y pivotes usan doble guion bajo (`pago__pedidos`, `usuario__rols`, `movimiento__cajas`), siguiendo el estilo del generador de código original.
- **Borrado lógico (soft delete)**: columnas `deleted_at` en la mayoría de tablas de negocio (verificado por migraciones y rasgo `SoftDeletes` en los modelos). Excepciones: `pago__pedidos`, `detalle__pedidos`, `detalle__compras`, `compra__pagos`, `gasto__pagos`, `movimiento__cajas`, `recetas`, `usuario__rols`, `barista_turno`, `turnos`, `rols`, `medio__pagos`, `factura__ventas`, `factura_contadores`, `mensajes`, `asistente_mensajes`.
- **Dinero**: `decimal(10,2)` en todas las columnas monetarias; casts `decimal:2` en los modelos Eloquent.
- **Migraciones**: 44 archivos en `backend-kaffa/database/migrations/` (creación de tablas + alteración/índices).

> **NOTA**
>
> `.env.example` ya declara la conexión correcta (`pgsql`/5432/`kaffa`), corregida en la pasada de seguridad de 2026-09.

8.2 Inventario de tablas (34)

8.2.1 Infraestructura de Laravel (6)

Tabla 33. *Inventario: tablas de infraestructura de Laravel*

Tabla	Propósito
`users`	Usuarios del sistema (con columnas personalizadas `nombre`, `correo`, `activo`, `foto`, `email_verified_at`).
`password_reset_tokens`	Tokens de recuperación de contraseña (columna `email` por convención de Laravel).
`failed_jobs`	Cola de trabajos fallidos (sin uso activo: `QUEUE_CONNECTION=sync`).
`personal_access_tokens`	Tokens Sanctum (`tokenable` polimórfico, `token` único, `abilities`, `expires_at`).
`codigos_otp`	Códigos OTP de un solo uso (hasheados) para verificación de correo y reseteo de contraseña.
`registros_pendientes`	Altas de clientes **aún sin verificar** (nombre, correo y contraseña hasheada). El usuario no se crea en `users` hasta confirmar el código.

8.2.2 Núcleo de negocio (21)

Tabla 34. *Inventario: tablas del núcleo de negocio*

Tabla	Propósito
`rols`	Roles del sistema: `admin`, `barista`, `cliente` (sembrados).
`usuario__rols`	Pivote N:M usuario ↔ rol (un usuario puede tener varios roles).
`categorias`	Categorías de productos del menú.
`productos`	Productos del menú con `precio_venta` e `imagen` (URL).
`recetas`	Pivote N:M producto ↔ insumo con `cantidad` (cuánto insumo consume cada producto).
`insumos`	Inventario: insumos con `stock_actual`, `stock_minimo`, `unidad_medida`.
`proveedors`	Proveedores de insumos (NIT único).
`medio__pagos`	Medios de pago; `es_virtual` distingue los que exigen comprobante.
`pedidos`	Pedidos (venta) con estado y propina.
`detalle__pedidos`	Líneas de pedido (producto, cantidad, precio, `nota`).
`pago__pedidos`	Pagos de pedidos (con `comprobante_url` si el medio es virtual).
`factura__ventas`	Factura 1:1 por pedido, número secuencial único.
`factura_contadores`	Contador transaccional de numeración de facturas por año.
`compras`	Compras a proveedores con `numero_factura_proveedor`.
`detalle__compras`	Líneas de compra (insumo, cantidad, `precio_costo`).
`compra__pagos`	Pagos de compras (con `comprobante_url`).
`gastos`	Gastos operativos (el monto vive en sus pagos).
`gasto__pagos`	Pagos de gastos (con `comprobante_url`).
`mermas`	Mermas de insumos (cantidad, motivo).
`cajas`	Sesiones de caja con arqueo (apertura/cierre, sistema vs físico, efectivo vs digital).

Tabla	Propósito
`movimiento__cajas`	Movimientos de caja (`ingreso` / `egreso`) enlazados a pagos.

8.2.3 Módulos de características (5)

Tabla 35. *Inventario: tablas de los módulos de características*

Tabla	Propósito
`turnos`	Turnos de trabajo: rango `fecha` – `fecha_fin` + `tipo` (`mañana` / `tarde` / `abierto`), únicos por `fecha+tipo` .
`barista_turno`	Pivote N:M barista ↔ turno (quiénes trabajan cada turno).
`reportes`	Reportes de novedades creados por baristas (prioridad, leído).
`mensajes`	Chat cliente ↔ barista/admin (leído).
`eventos`	Eventos publicados en la landing/Home (título, fecha, lugar, publicación).
`evento_imagenes`	Galería de imágenes por evento (orden, máximo 5 por evento).
`asistente_mensajes`	Historial del chat con el Asistente KAFFA (por usuario, todos los roles).

8.3 Diagrama entidad–relación

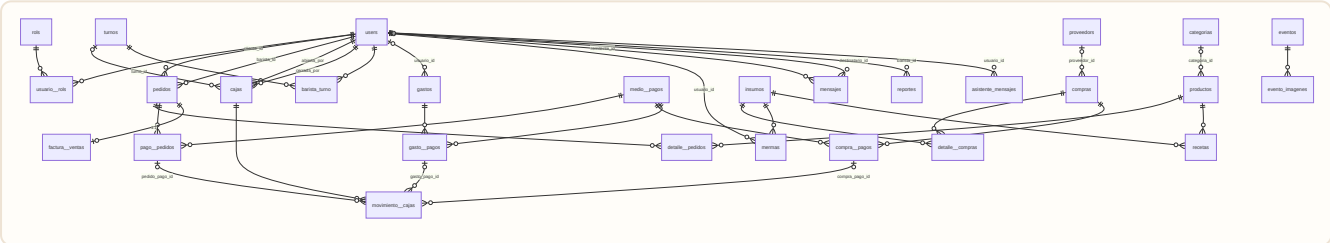


Figura 1. *Diagrama entidad–relación del esquema KAFFA*

8.4 Diccionario de datos

Fuente de cada tabla: los archivos de migración indicados. `PK` = clave primaria; `FK` = clave foránea; `NULL` = admite nulos; los tipos están según `$table->…` de Laravel.

8.4.1 `users` — `2014_10_12_000000_create_users_table.php` + alters

Tabla 36. *Diccionario de la tabla `users`*

Campo	Tipo	NULL	Clave	Detalle
id	bigint	No	PK	autoincrement
nombre	varchar(100)	No		
correo	varchar(120)	No	UNIQUE	login y correos (el modelo sobrescribe `getEmailForPasswordReset` para usar `correo`)
password	varchar(255)	No		hash bcrypt (migración de índices la convierte de TEXT a VARCHAR(255))
activo	boolean	No		default `true`

Campo	Tipo	NULL	Clave	Detalle
foto	varchar(500)	Sí		URL (patrón Cloudinary); añadida en `2026_09_16_000000_add_foto_to_users_table.php`
email_verified_at	timestamp	Sí		verificación de correo; añadida con backfill `now()` para usuarios existentes (`2026_09_09_141436…php`)
remember_token	varchar(100)	Sí		
deleted_at	timestamp	Sí		soft delete
created_at / updated_at	timestamp	No		

8.4.2 `rols` — `2026_04_07_000002_create_rols_table.php`

Tabla 37. *Diccionario de la tabla* `rols`

Campo	Tipo	NULL	Clave	Detalle
id	bigint	No	PK	
nombre	varchar(50)	No	UNIQUE	valores sembrados: `admin` , `barista` , `cliente`
created_at / updated_at	timestamp	No		

8.4.3 `usuario__rols` — `2026_04_07_000007_create_usuario__rols_table.php`

Tabla 38. *Diccionario de la tabla* `usuario__rols`

Campo	Tipo	NULL	Clave	Detalle
usuario_id	bigint	No	FK → users.id, PK compuesta	onDelete cascade
rol_id	bigint	No	FK → rols.id, PK compuesta	onDelete cascade
created_at / updated_at	timestamp	No		

> **NOTA**
> Un usuario puede tener varios roles simultáneos (p. ej. barista + cliente).

8.4.4 `categorias` — `2026_04_07_000003_create_categorias_table.php`

Tabla 39. *Diccionario de la tabla* `categorias`

Campo	Tipo	NULL	Clave	Detalle
id	bigint	No	PK	
nombre	varchar(100)	No		sembradas: Cafés, Postres, Snacks
deleted_at	timestamp	Sí		
created_at / updated_at	timestamp	No		

8.4.5 `productos` — `2026_04_07_000008_create_productos_table.php` + `2026_08_26_000001_add_imagen_to_productos.php`

Tabla 40. *Diccionario de la tabla* `productos`

Campo	Tipo	NULL	Clave	Detalle
id	bigint	No	PK	
nombre	varchar(150)	No		
descripcion	text	Sí		
imagen	varchar(500)	Sí		URL de imagen (añadida 2026-08-26)
precio_venta	decimal(10,2)	No		COP
categoria_id	bigint	Sí	FK → categorias.id	
activo	boolean	No		default `true`
deleted_at	timestamp	Sí		
created_at / updated_at	timestamp	No		

8.4.6 `recetas` — `2026_04_07_000009_create_recetas_table.php`

Tabla 41. *Diccionario de la tabla* `recetas`

Campo	Tipo	NULL	Clave	Detalle
producto_id	bigint	No	FK → productos.id, PK compuesta	cascade
insumo_id	bigint	No	FK → insumos.id, PK compuesta	cascade
cantidad	decimal(10,2)	No		consumo del insumo por producto
created_at / updated_at	timestamp	No		

8.4.7 `insumos` — `2026_04_07_000005_create_insumos_table.php`

Tabla 42. *Diccionario de la tabla* `insumos`

Campo	Tipo	NULL	Clave	Detalle
id	bigint	No	PK	
nombre	varchar(100)	No		validado único en `StoreInsumoRequest`
unidad_medida	varchar(20)	Sí		p. ej. kg, lt, und
stock_actual	decimal(10,2)	No		default 0
stock_minimo	decimal(10,2)	No		default 0
activo	boolean	No		default `true`
deleted_at	timestamp	Sí		
created_at / updated_at	timestamp	No		

> **NOTA**
>
> El stock se **incrementa** con compras, se **reduce** con ventas (listener `DeducirInventario`) y mermas, y se **restaura** al eliminar mermas o compras (`CompraService`, `MermaService`).

8.4.8 `proveedors` — `2026_04_07_000006_create_proveedors_table.php`

Tabla 43. *Diccionario de la tabla* `proveedors`

Campo	Tipo	NULL	Clave	Detalle
id	bigint	No	PK	
nit	varchar(50)	No	UNIQUE	
nombre	varchar(150)	No		
telefono	varchar(50)	Sí		
correo	varchar(120)	Sí		
direccion	text	Sí		
activo	boolean	No		default `true`
deleted_at	timestamp	Sí		
created_at / updated_at	timestamp	No		

8.4.9 `medio_pagos` — `2026_04_07_000004…` + `2026_06_09_020704_add_es_virtual…`

Tabla 44. *Diccionario de la tabla* `medio_pagos`

Campo	Tipo	NULL	Clave	Detalle
id	bigint	No	PK	
nombre	varchar(50)	No	UNIQUE	sembrados: Efectivo, Tarjeta de Crédito, Tarjeta de Débito, Transferencia Bancaria, Pago Móvil, QR (Pago Digital)
es_virtual	boolean	No		default `false`; `true` ⇒ el pago exige `comprobante_url`
created_at / updated_at	timestamp	No		

> **NOTA**
>
> Los IDs de medios virtuales se cachean 5 minutos (`Medio_Pago::virtualIds()` , clave `medios_pago:virtual_ids`) y la caché se invalida al guardar/eliminar un medio.

8.4.10 `pedidos` — `2026_04_07_000011_create_pedidos_table.php`

Tabla 45. *Diccionario de la tabla* `pedidos`

Campo	Tipo	NULL	Clave	Detalle
id	bigint	No	PK	
cliente_id	bigint	Sí	FK → users.id	
barista_id	bigint	Sí	FK → users.id	
total	decimal(10,2)	Sí		consistencia validada en `PedidoService`
propina	decimal(10,2)	No		default 0
estado	varchar(20)	No		default `pendiente` ; valores validados: `pendiente` , `pagado` , `cancelado` , `entregado`

Campo	Tipo	NULL	Clave	Detalle
deleted_at	timestamp	Sí		
created_at / updated_at	timestamp	No		

8.4.11 `detalle__pedidos` — `2026_04_07_000016…` + `2026_09_04_173307_add_nota…`

Tabla 46. *Diccionario de la tabla* `detalle__pedidos`

Campo	Tipo	NULL	Clave	Detalle
id	bigint	No	PK	
pedido_id	bigint	No	FK → pedidos.id	cascade
producto_id	bigint	No	FK → productos.id	
cantidad	decimal(10,2)	No		validada entera ≥ 1 en `StorePedidoRequest`
precio_unitario	decimal(10,2)	Sí		
subtotal	decimal(10,2)	Sí		
nota	varchar(500)	Sí		nota por línea (añadida 2026-09-04)
created_at / updated_at	timestamp	No		

> **NOTA**
>
> Sin soft delete: `PedidoService::eliminar` borra físicamente los detalles al eliminar un pedido.

8.4.12 `pago__pedidos` — `2026_04_07_000015…` + `2026_06_09_020705_add_comprobante_url…`

Tabla 47. *Diccionario de la tabla* `pago__pedidos`

Campo	Tipo	NULL	Clave	Detalle
id	bigint	No	PK	
pedido_id	bigint	No	FK → pedidos.id	cascade
medio_pago_id	bigint	No	FK → medio__pagos.id	
monto	decimal(10,2)	No		mínimo 0.01
comprobante_url	varchar(500)	Sí		obligatoria si el medio es virtual (validación `withValidator`)
created_at / updated_at	timestamp	No		

8.4.13 `factura__ventas` — `2026_04_07_000014_create_factura__ventas_table.php`

Tabla 48. *Diccionario de la tabla* `factura__ventas`

Campo	Tipo	NULL	Clave	Detalle
id	bigint	No	PK	
pedido_id	bigint	No	FK → pedidos.id, UNIQUE	1 pedido = 1 factura

Campo	Tipo	NULL	Clave	Detalle
numero_factura	varchar(50)	No	UNIQUE	generado por el servidor: `F{año}{contador 6 dígitos}`
subtotal	decimal(10,2)	No		
impuestos	decimal(10,2)	No		default 0
total	decimal(10,2)	No		
fecha_emision	timestamp	No		default now
created_at / updated_at	timestamp	No		

> **NOTA**
>
> El cliente **no envía** `numero_factura` (comentario en `StoreFacturaVentaRequest`); lo genera `FacturaService::generarNumeroFactura()` dentro de una transacción con `lockForUpdate` sobre `factura_contadores`.

8.4.14 `factura_contadores` — `2026_08_03_000000_create_factura_contadores_table.php`

Tabla 49. *Diccionario de la tabla* `factura_contadores`

Campo	Tipo	NULL	Clave	Detalle
id	bigint	No	PK	
nombre	varchar(20)	No	UNIQUE(nombre, anio)	prefijo del contador (p. ej. `F`)
anio	smallint unsigned	No	UNIQUE(nombre, anio)	
valor	int unsigned	No		default 0; se incrementa con bloqueo pesimista
created_at / updated_at	timestamp	No		

8.4.15 `compras` — `2026_04_07_000013_create_compras_table.php`

Tabla 50. *Diccionario de la tabla* `compras`

Campo	Tipo	NULL	Clave	Detalle
id	bigint	No	PK	
proveedor_id	bigint	Sí	FK → proveedors.id	
numero_factura_proveedor	varchar(100)	No		indexada (no única a nivel BD)
fecha_compra	timestamp	No		default now
total_compra	decimal(10,2)	No		
deleted_at	timestamp	Sí		
created_at / updated_at	timestamp	No		

8.4.16 `detalle__compras` — `2026_04_07_000017…`

Tabla 51. *Diccionario de la tabla* `detalle__compras`

Campo	Tipo	NULL	Clave	Detalle
id	bigint	No	PK	
compra_id	bigint	No	FK → compras.id	cascade
insumo_id	bigint	No	FK → insumos.id	
cantidad	decimal(10,2)	No		
precio_costo	decimal(10,2)	No		
created_at / updated_at	timestamp	No		

8.4.17 `compra__pagos` — `2026_04_07_000018…` + comprobante_url

Tabla 52. *Diccionario de la tabla* `compra__pagos`

Campo	Tipo	NULL	Clave	Detalle
id	bigint	No	PK	
compra_id	bigint	No	FK → compras.id	cascade
medio_pago_id	bigint	No	FK → medio__pagos.id	
monto	decimal(10,2)	No		
fecha_pago	timestamp	No		default now
comprobante_url	varchar(500)	Sí		obligatoria si el medio es virtual
created_at / updated_at	timestamp	No		

8.4.18 `gastos` — `2026_04_07_000012…`

Tabla 53. *Diccionario de la tabla* `gastos`

Campo	Tipo	NULL	Clave	Detalle
id	bigint	No	PK	
descripcion	text	No		
categoria	varchar(100)	Sí		indexada
usuario_id	bigint	Sí	FK → users.id	forzado al usuario autenticado en `GastoService`
deleted_at	timestamp	Sí		
created_at / updated_at	timestamp	No		

> **NOTA**
>
> El monto de un gasto no está en esta tabla: se registra en sus filas de `gasto__pagos`.

8.4.19 `gasto__pagos` — `2026_04_07_000019…` + comprobante_url

Tabla 54. *Diccionario de la tabla* `gasto__pagos`

Campo	Tipo	NULL	Clave	Detalle
id	bigint	No	PK	
gasto_id	bigint	No	FK → gastos.id	cascade
medio_pago_id	bigint	No	FK → medio__pagos.id	
monto	decimal(10,2)	No		
fecha_pago	timestamp	No		default now
comprobante_url	varchar(500)	Sí		obligatoria si el medio es virtual
created_at / updated_at	timestamp	No		

8.4.20 `mermas` — `2026_04_07_000020…`

Tabla 55. *Diccionario de la tabla* `mermas`

Campo	Tipo	NULL	Clave	Detalle
id	bigint	No	PK	
insumo_id	bigint	No	FK → insumos.id	
cantidad	decimal(10,2)	No		validada contra stock en `MermaService`
motivo	text	Sí		
usuario_id	bigint	Sí	FK → users.id	forzado en `MermaService`
deleted_at	timestamp	Sí		
created_at / updated_at	timestamp	No		

8.4.21 `cajas` — `2026_04_07_000010…` + `2026_08_26_000004_add_detalle_caja…`

Tabla 56. *Diccionario de la tabla* `cajas`

Campo	Tipo	NULL	Clave	Detalle
id	bigint	No	PK	
abierta_por	bigint	No	FK → users.id	forzado al autenticado al abrir
cerrada_por	bigint	Sí	FK → users.id	forzado al autenticado al cerrar
turno_id	bigint	Sí	FK → turnos.id	asignado automáticamente al abrir (turno activo del barista; null para admin)
estado	varchar(20)	No		default `abierta`; indexada
monto_apertura_sistema	decimal(10,2)	No		default 0
monto_apertura_fisico	decimal(10,2)	No		requerido al abrir
monto_apertura_efectivo	decimal(10,2)	No		default 0; desglose de apertura
monto_apertura_digital	decimal(10,2)	No		default 0; desglose de apertura

Campo	Tipo	NULL	Clave	Detalle
monto_cierre_sistema	decimal(10,2)	Sí		**calculado por el servidor** al cerrar: apertura + ingresos − egresos
monto_cierre_fisico	decimal(10,2)	Sí		lo declara quien cierra
monto_cierre_efectivo	decimal(10,2)	Sí		desglose de cierre
monto_cierre_digital	decimal(10,2)	Sí		desglose de cierre
fecha_apertura	timestamp	No		default now
fecha_cierre	timestamp	Sí		
deleted_at	timestamp	Sí		
created_at / updated_at	timestamp	No		

> **NOTA**
>
> Regla de negocio: no se puede eliminar una caja con movimientos (`Caja::eliminarRegistro` lanza `BusinessRuleException`).

8.4.22 `movimiento__cajas` — `2026_04_07_000021…`

Tabla 57. *Diccionario de la tabla* `movimiento__cajas`

Campo	Tipo	NULL	Clave	Detalle
id	bigint	No	PK	
caja_id	bigint	No	FK → cajas.id	
tipo	varchar(20)	No		`ingreso` o `egreso`; indexada
monto	decimal(10,2)	No		
pedido_pago_id	bigint	Sí	FK → pago__pedidos.id	
compra_pago_id	bigint	Sí	FK → compra__pagos.id	
gasto_pago_id	bigint	Sí	FK → gasto__pagos.id	
descripcion	text	Sí		
created_at / updated_at	timestamp	No		

> **NOTA**
>
> Los movimientos no se crean directamente por la API: los emiten los servicios al registrar pagos (`MovimientoCajaPolicy` lo documenta).

8.4.23 `turnos` — `2026_06_14_152013…` + `2026_10_06_000000_add_fecha_fin_to_turnos_table.php`

Tabla 58. *Diccionario de la tabla* `turnos`

Campo	Tipo	NULL	Clave	Detalle
id	bigint	No	PK	
fecha	date	No	UNIQUE(fecha, tipo)	inicio del turno
fecha_fin	date	Sí		fin del rango (`2026_10_06_000000_add_fecha_fin_to_turnos_table`); si es nulo equivale a `fecha` (un solo día). Las filas existentes se rellenan con `fecha`.
tipo	varchar — enum `mañana` / `tarde` / `abierto`	No	UNIQUE(fecha, tipo)	horarios: mañana 07:00–13:00, tarde 13:00–18:00, **abierto 05:00–22:00** (`TurnoService`)
created_at / updated_at	timestamp	No		

> **NOTA**
> El turno **cubre** una fecha si cae dentro de `[fecha, fecha_fin]`. El tipo `abierto` tiene prioridad sobre `mañana` / `tarde` (`TurnoService::getActiveTurno`).

8.4.24 `barista_turno` — `2026_06_14_152014…`

Tabla 59. *Diccionario de la tabla* `barista_turno`

Campo	Tipo	NULL	Clave	Detalle
barista_id	bigint	No	FK → users.id, PK compuesta	cascade
turno_id	bigint	No	FK → turnos.id, PK compuesta	cascade
created_at / updated_at	timestamp	No		

8.4.25 `reportes` — `2026_08_26_000002…`

Tabla 60. *Diccionario de la tabla* `reportes`

Campo	Tipo	NULL	Clave	Detalle
id	bigint	No	PK	
barista_id	bigint	Sí	FK → users.id	forzado al barista autenticado al crear
mensaje	text	No		novedad reportada
prioridad	varchar(20)	No		default `media`; valores: `baja`, `media`, `alta`
leido	boolean	No		default `false`
deleted_at	timestamp	Sí		
created_at / updated_at	timestamp	No		

8.4.26 `mensajes` — `2026_08_26_000003…`

Tabla 61. *Diccionario de la tabla* `mensajes`

Campo	Tipo	NULL	Clave	Detalle
id	bigint	No	PK	
remitente_id	bigint	No	FK → users.id	cascade
destinatario_id	bigint	No	FK → users.id	cascade
mensaje	text	No		máx. 1000 (validación)
leido	boolean	No		default `false`
created_at / updated_at	timestamp	No		

> **NOTA**
>
> Reglas: no auto-mensajes, no chat cliente↔cliente, destinatario inactivo rechazado (422) — `MensajeController::store`.

8.4.27 `asistente_mensajes` — `2026_10_05_000001…`

Tabla 62. *Diccionario de la tabla* `asistente_mensajes`

Campo	Tipo	NULL	Clave	Detalle
id	bigint	No	PK	
usuario_id	bigint	No	FK → users.id	cascade; dueño del hilo (cualquier rol)
rol	varchar(20)	No		`user` (pregunta) o `assistant` (respuesta del asistente)
contenido	text	No		texto del turno
created_at / updated_at	timestamp	No		
Índice				`(usuario_id, id)` para recuperar el hilo cronológicamente

> **NOTA**
>
> El historial se persiste **por usuario** (todos los roles) y sólo se accede a él mediante `/api/v1/asistente/*`; cada quien ve únicamente su hilo. Ver `03-api.md` §17 y `04-modulos.md` §13.

8.4.28 `personal_access_tokens` — `2019_12_14_000001…`

Tabla 63. *Diccionario de la tabla* `personal_access_tokens`

Campo	Tipo	NULL	Clave	Detalle
id	bigint	No	PK	
tokenable_type / tokenable_id	string / bigint	No	índice	morfismo (User)
name	varchar	No		`auth_token`
token	varchar(64)	No	UNI-QUE	hash del token

Campo	Tipo	NULL	Clave	Detalle
abilities	text	Sí		p. ej. `["*"]` (admin); se almacenan pero no se validan (ver `05-seguridad.md`)
last_used_at	timestamp	Sí		
expires_at	timestamp	Sí		se fija si `SANCTUM_EXPIRATION` está definida
created_at / updated_at	timestamp	No		

8.4.29 `password_reset_tokens` — `2014_10_12_100000…`

Tabla 64. *Diccionario de la tabla `password_reset_tokens`*

Campo	Tipo	NULL	Clave	Detalle
email	string	No	PK	es el `correo` del usuario
token	string	No		
created_at	timestamp	Sí		

8.4.30 `failed_jobs` — `2019_08_19_000000…`

Estructura estándar de Laravel (id, uuid único, connection, queue, payload, exception, failed_at). Sin uso activo con `QUEUE_CONNECTION=sync`.

8.4.31 `codigos_otp` — `2026_10_07_000001_create_codigos_otp_table.php`

Tabla 65. *Diccionario de la tabla `codigos_otp`*

Columna	Tipo	Nulo	Notas
id	bigint	No	PK
correo	varchar(120)	No	correo destino del código (indexado)
tipo	varchar(20)	No	`verificacion` o `reset`
codigo_hash	varchar	No	hash del código OTP (nunca en claro)
expira_at	timestamp	Sí	caducidad (15 min)
intentos	tinyint	No	intentos fallidos (máx. 5)
enviado_at	timestamp	Sí	último envío (cooldown de reenvío 60 s)
created_at / updated_at	timestamp	Sí	

Restricción única `(correo, tipo)`. Usada por `OtpService` para la verificación de correo y el restablecimiento de contraseña; el código es de un solo uso (se elimina al validarse).

8.4.32 `registros_pendientes` — `2026_10_13_000001_create_registros_pendientes_table.php`

Tabla 66. *Diccionario de la tabla* `registros_pendientes`

Columna	Tipo	Nulo	Notas
id	bigint	No	PK
correo	varchar(120)	No	correo del alta (único)
nombre	varchar(100)	No	nombre del solicitante
password	varchar	No	contraseña **hasheada** (`hashed` cast); nunca en claro
created_at / updated_at	timestamp	Sí	

Implementa el **alta diferida**: el registro público guarda aquí los datos y envía el OTP, pero **no** crea la fila en `users`. Al validar el código (`RegistrationService::confirmar`) se crea el usuario ya verificado (con rol `cliente`) y se elimina el pendiente. Los pendientes con más de 24 h se purgan (`RegistrationService::purgarAntiguos`).

8.4.33 `eventos` — `2026_10_08_000002_create_eventos_table.php`

Tabla 67. *Diccionario de la tabla* `eventos`

Columna	Tipo	Nulo	Notas
id	bigint	No	PK
titulo	varchar(255)	No	
descripcion	text	No	
lugar	varchar(255)	Sí	
fecha	date	No	índice
hora	time	Sí	
publicado	boolean	No	default `false` ; índice. Sólo los publicados se muestran en la landing/Home
deleted_at	timestamp	Sí	soft delete
created_at / updated_at	timestamp	Sí	

> **NOTA**
>
> El listado público (`GET /eventos`) devuelve sólo `publicado = true` ; el panel admin (`GET /eventos/admin`) ve también los borradores. La galería vive en `evento_imagenes` (máximo 5). Ver `03-api.md` §3.1 y `04-modulos.md` §16.

8.4.34 `evento_imagenes` — `2026_10_08_000003_create_evento_imagenes_table.php`

Tabla 68. *Diccionario de la tabla* `evento_imagenes`

Columna	Tipo	Nulo	Notas
id	bigint	No	PK
evento_id	bigint	No	FK → eventos.id (cascade)
url	varchar(500)	No	URL validada con `App\Rules\CloudinaryUrl` (o URL local devuelta por `POST /eventos/imagen`)
orden	int unsigned	No	default 0; índice `(evento_id, orden)`
created_at / updated_at	timestamp	Sí	

> **NOTA**
>
> La misma migración migra la antigua columna `eventos.imagen` a la galería (como portada, orden 0) y elimina esa columna.

8.5 Índices de rendimiento — `2026_07_31_000000_add_performance_indexes_to_tables.php`

Tabla 69. *Índices de rendimiento añadidos por tabla*

Tabla	Índices adicionados
pedidos	`estado`, `created_at`
pago__pedidos	`created_at`
compras	`created_at`, `numero_factura_proveedor`
gastos	`created_at`, `categoria`
mermas	`created_at`
movimiento__cajas	`created_at`, `tipo`
factura__ventas	`created_at`
cajas	`estado`
users	conversión de `password` TEXT → VARCHAR(255) (SQL específico para MySQL y pgsql)

Además, índices únicos declarados en las migraciones: `factura__ventas.pedido_id` y `numero_factura`, `proveedors.nit`, `productos/medio__pagos` según tabla, `factura_contadores(nombre, anio)`, `turnos(fecha, tipo)`.

8.6 Seeders — `backend-kaffa/database/seeders/`

Orden de ejecución (`DatabaseSeeder`): `RolSeeder` → `MedioPagoSeeder` → `UserSeeder` → `TurnoSeeder` → `CatalogSeeder`.

Tabla 70. *Seeders del proyecto (orden de ejecución)*

Seeder	Datos
`RolSeeder`	Roles `admin`, `barista`, `cliente` (`firstOrCreate`).
`MedioPagoSeeder`	6 medios: Efectivo, Tarjeta de Crédito, Tarjeta de Débito (no virtuales); Transferencia Bancaria, Pago Móvil, QR (Pago Digital) (virtuales).
`UserSeeder`	`admin@kaffa.com` / `admin123`, `barista@kaffa.com` / `barista123`, `cliente@kaffa.com` / `cliente123`; todos verificados y con foto demo (URL Cloudinary).
`TurnoSeeder`	Para el día actual (America/Bogota) crea turnos mañana y tarde y asigna a **todos** los baristas.
`CatalogSeeder`	3 categorías (Cafés, Postres, Snacks), 20 insumos con stocks, 10 productos con precios en COP e `imagen` (URL externa) y recetas producto↔insumo (p. ej. Capuchino = 0.02 kg café + 0.25 lt leche).

NOTA

El `UserFactory` (`database/factories/UserFactory.php`) está alineado con el esquema real (`nombre` / `correo`, `activo`, estados `unverified()` / `inactive()`). No lo usan los tests actuales (que crean usuarios manualmente), pero queda operativo para `User::factory()`.

9. Flujos de datos y consistencia

Este capítulo describe **cómo se mueven los datos** entre módulos y qué garantías ofrece el backend. Es la lectura obligatoria antes de modificar `PedidoService`, `CompraService`, `Caja` o `FacturaService`.

9.1 Alta diferida y verificación

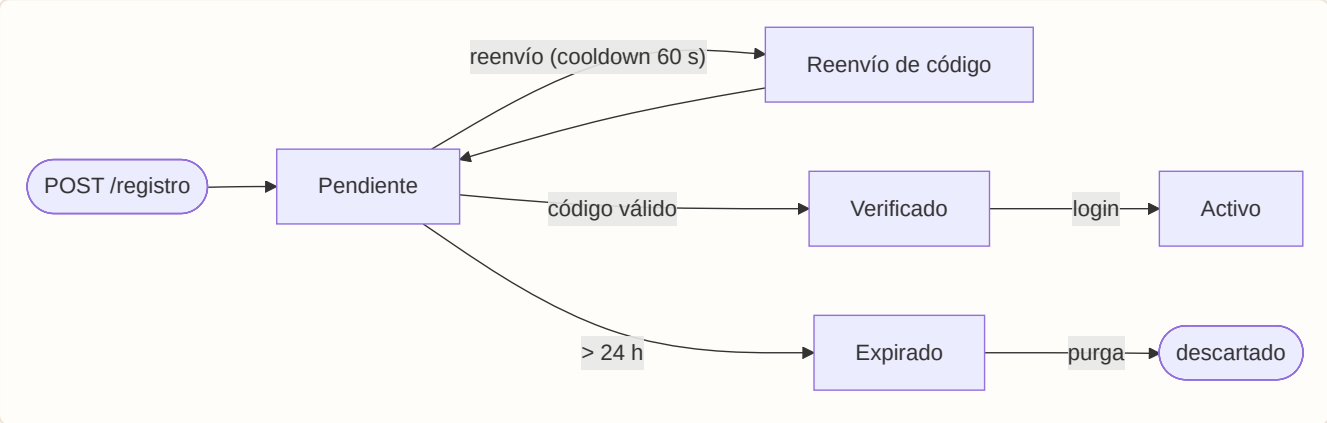


Figura 1. Ciclo de un alta de cliente.

Tabla 71. *Pasos del alta diferida de clientes*

Paso	Tabla	Efecto
Registro	`registros_pendientes`	Guarda nombre, correo y contraseña hasheada
Código	`codigos_otp`	Hash, expiración, intentos y `enviado_at`
Verificación	`users`, `usuario__rols`	Crea usuario verificado con rol `cliente`; elimina pendiente y código
Reenvío	`codigos_otp`	Reemplaza el código si pasó el cooldown
Purga	`registros_pendientes`	Elimina pendientes > 24 h

La contraseña se hashea con el cast `hashed` de `RegistroPendiente` y de `User`. Nunca se guarda en claro.

9.2 Pedido: del carrito al inventario

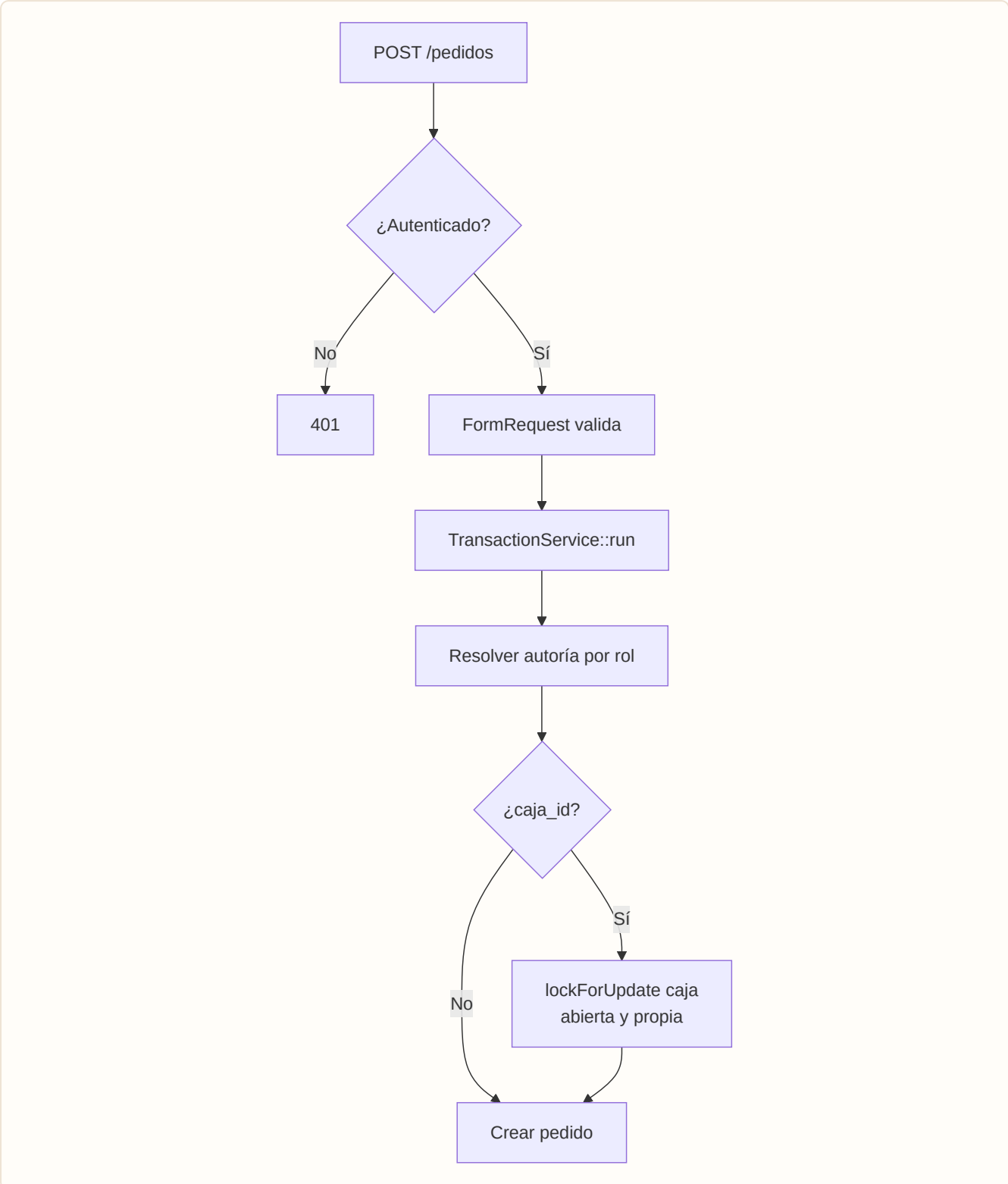


Figura 2A. Validación y autorización de un pedido.

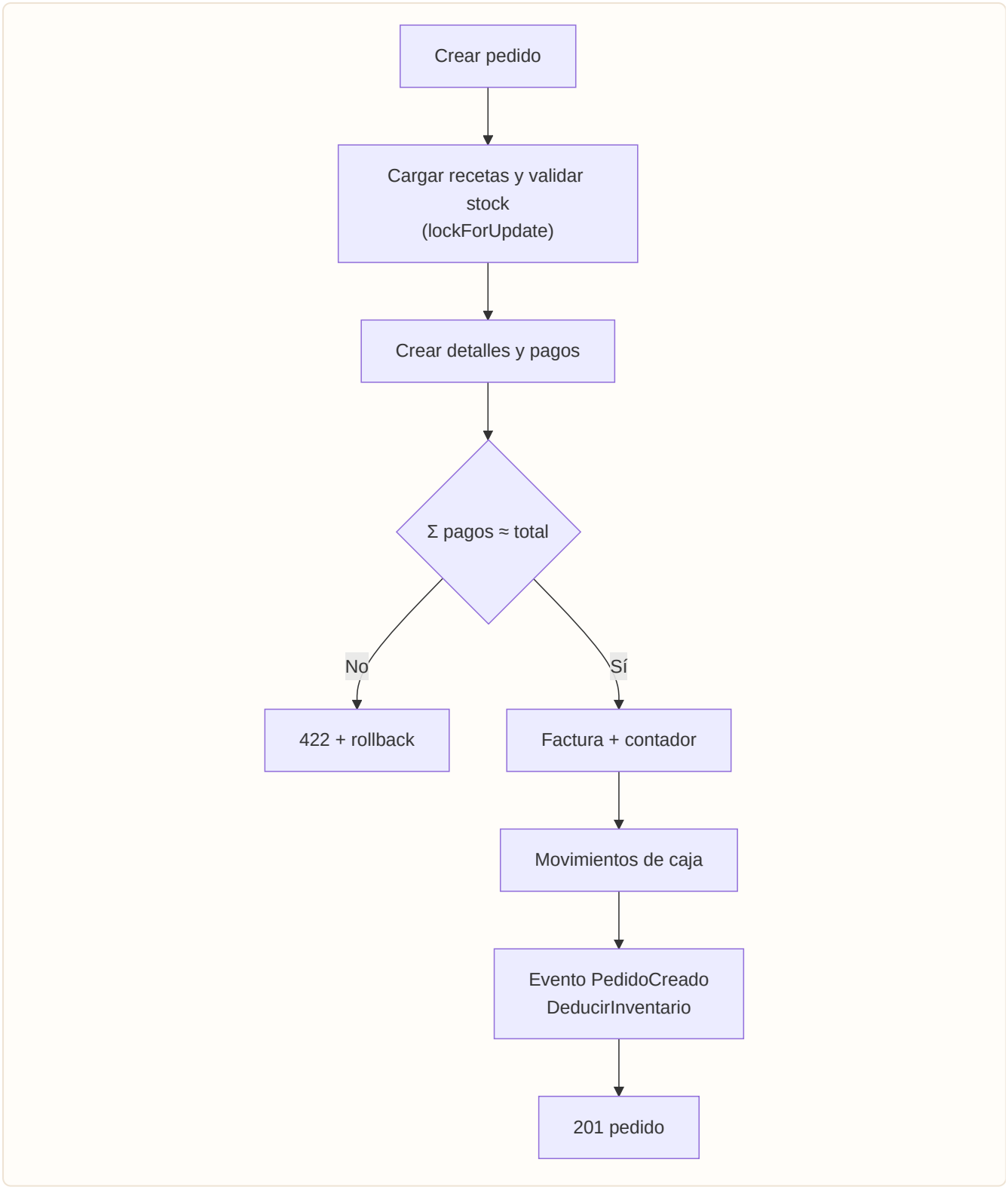


Figura 2B. Persistencia y efectos de un pedido.

9.2.1 Autoría

Tabla 72. *Autoría de un pedido según rol*

Rol	`cliente_id`	`barista_id`	`estado`
Cliente	Forzado a su ID	`null`	Forzado a `pendiente`
Barista	Opcional (cliente existente)	Forzado a su ID	El enviado, por defecto `pendiente`
Admin	Libre	Libre	Libre

El cliente **no puede** crear un pedido a nombre de otro ni marcarlo como pagado. El barista firma el pedido de mostrador. El admin puede corregir autoría en backoffice.

9.2.2 Caja

Si se envía `caja_id`:

- La caja debe existir (`CAJA_NO_EXISTE`).
- Debe estar `abierta` (`CAJA_CERRADA`).
- Un barista solo puede usar una caja que él abrió (`CAJA_AJENA`).
- Se bloquea con `lockForUpdate` para evitar dos cierres simultáneos.

Cada pago genera un `movimiento__cajas` de tipo `ingreso`.

9.2.3 Stock

1. Se cargan los productos con `insumos` (recetas).
2. Se agrega el consumo por insumo: `cantidad_detalle × receta.cantidad`.
3. Se bloquean los insumos afectados con `lockForUpdate`.
4. Si `stock_actual < consumo`, se lanza `Stock insuficiente...` y se revierte todo.
5. Al crear el pedido, el evento `PedidoCreado` ejecuta `DeducirInventario`, que hace un `UPDATE` atómico con `GREATEST(0, stock_actual - ?)`.

La validación previa evita vender sin stock; el `GREATEST` evita valores negativos si algo cambia entre la validación y el descuento. La operación completa está en una transacción.

9.2.4 Pagos

- `pagos[].monto ≥ 0.01`.
- La suma debe coincidir con el total (±0.01).
- Estado `pagado` exige al menos un pago.
- Medio virtual (`es_virtual=true`) exige `comprobante_url` validada con `CloudinaryUrl`.
- El comprobante se guarda como URL; el archivo vive en Cloudinary.

9.2.5 Factura

Si se envía `factura`, el servidor:

1. Genera `numero_factura` con `FacturaService::generarNumeroFactura()`.
2. Hace UPSERT del contador `(nombre, año)`.
3. Bloquea la fila con `lockForUpdate`.

4. Incrementa `valor`.
5. Formatea `F{AÑO}{NNNNNN}`.

El cliente no puede imponer el número. La transacción evita huecos y duplicados.

9.2.6 Eliminación

`PedidoService::eliminar`:

- Borra detalles (hard delete).
- Borra movimientos de caja vinculados a sus pagos.
- Borra pagos y factura.
- Hace soft delete del pedido.
- **No restaura stock**.

> **ADVERTENCIA**
>
> **Advertencia.** Eliminar un pedido no revierte el inventario. Si fue un error, el operador debe registrar una merma o un ajuste autorizado, no borrar y recrear a ciegas.

9.3 Compra: aumento de stock y egreso

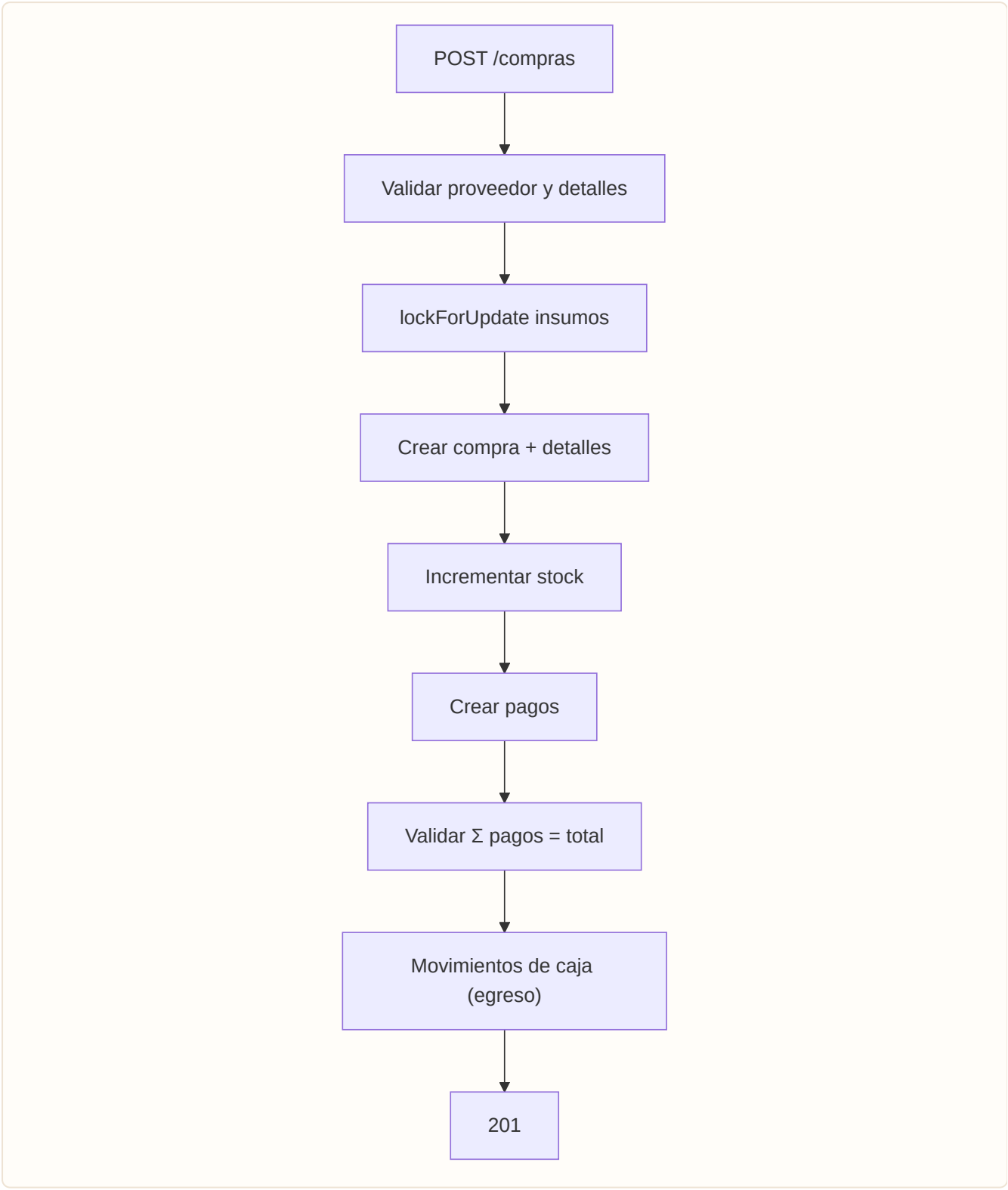


Figura 3. Registro de una compra.

Reglas:

- `detalles` es obligatorio y debe tener al menos una línea.
- Cada detalle incrementa `stock_actual` del insumo.

- La suma de pagos debe coincidir con `total_compra` (±0.01).
- Cada pago genera un egreso de caja si se usa caja.
- Al eliminar la compra, se revierte el stock; si quedaría negativo, la operación falla.

9.4 Gasto y merma

Gasto: registra un egreso operativo. `usuario_id` se fuerza al autenticado; los pagos y movimientos se crean en la misma transacción. Al eliminar, se limpian pagos y movimientos.

Merma: reduce stock con motivo y responsable. Bloquea el insumo, valida que la cantidad no supere el stock y descuenta. Al eliminar, restaura el stock.

Tabla 73. *Efecto de cada operación sobre stock, caja y autoría*

Operación	Stock	Caja	Autoría
Compra	Aumenta	Egreso	Proveedor
Gasto	No cambia	Egreso	Usuario autenticado
Merma	Disminuye	No cambia	Usuario autenticado
Venta	Disminuye	Ingreso	Cliente/barista

9.5 Caja y arqueo

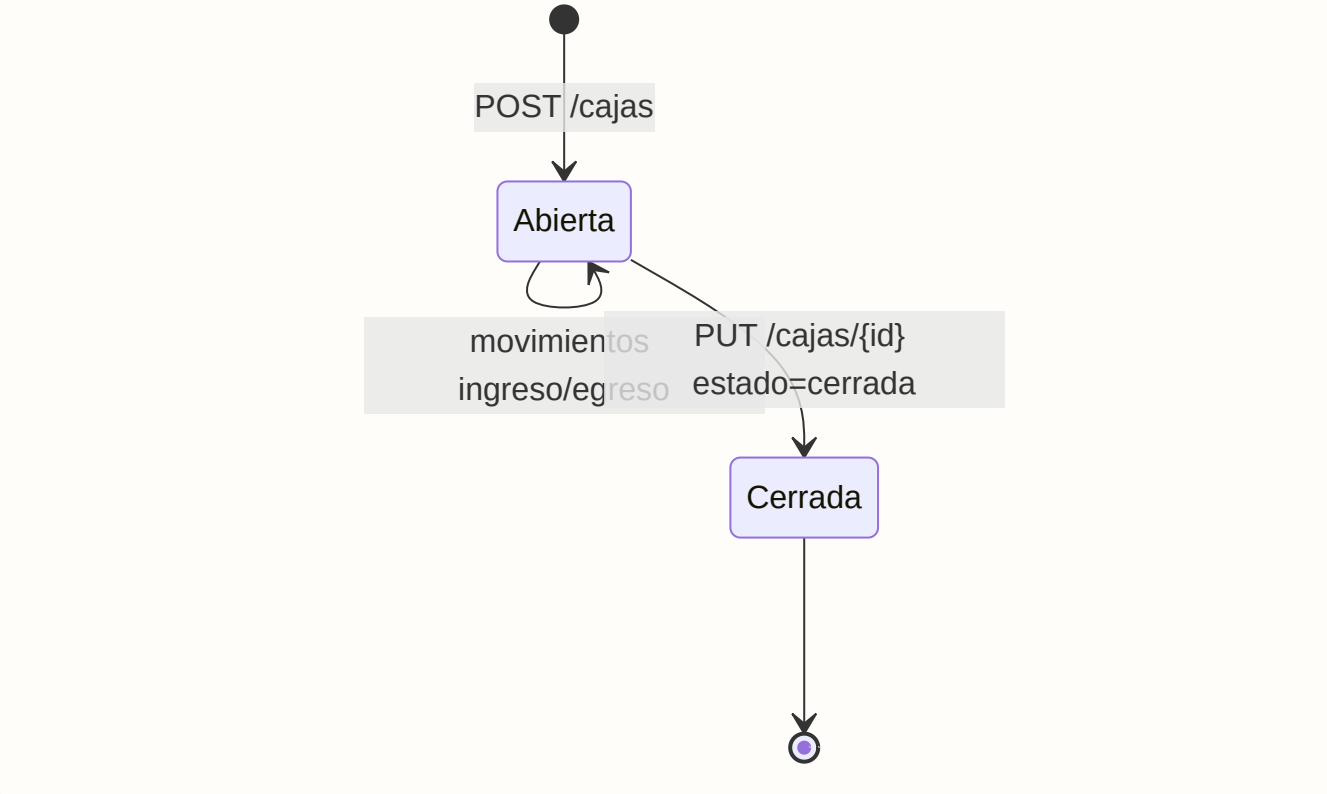


Figura 4. Ciclo de una caja.

9.5.1 Apertura

El cliente envía:

```
{
  "monto_apertura_fisico": 100000,
  "monto_apertura_efectivo": 80000,
  "monto_apertura_digital": 20000
}
```

El servidor fija:

- `abierta_por = Auth::id()`.
- `estado = abierta`.
- `fecha_apertura = now()`.
- `turno_id = turno activo` (barista) o `null` (admin).
- `monto_apertura_sistema = monto_apertura_fisico` si no se envía.

9.5.2 Cierre

El cliente envía `estado='cerrada'` y los montos declarados. El servidor:

```
monto_cierre_sistema = monto_apertura_sistema
                     + Σ ingresos
                     - Σ egresos
```

Fija `cerrada_por` y `fecha_cierre`. El cliente **no puede** enviar `monto_cierre_sistema`; se ignora y se recalcula.

Diferencia de arqueo:

```
diferencia = monto_cierre_fisico - monto_cierre_sistema
```

- `0`: perfecto.
- Negativa: faltante.
- Positiva: sobrante.

9.5.3 Eliminación

No se puede eliminar una caja con movimientos (`422`). Primero deben anularse o trasladarse los movimientos con autorización.

9.6 Turnos

Un turno cubre un rango `[fecha, fecha_fin]`. Si `fecha_fin` es nula o igual a `fecha`, dura un día.

Tabla 74. *Horarios de los tipos de turno*

Tipo	Horario
`mañana`	07:00–13:00
`tarde`	13:00–18:00
`abierto`	05:00–22:00

`TurnoService::getActiveTurno`:

1. Calcula hora y fecha en `America/Bogota`.
2. Si está en 05:00–22:00 y el barista tiene un turno `abierto` que cubre hoy, lo devuelve (prioridad).
3. Si no, determina `mañana` / `tarde` por hora.
4. Busca un turno del tipo correspondiente que cubra la fecha.

Fuera de horario, el middleware responde `FUERA_HORARIO`; sin asignación, `TURNO_INACTIVO`; si el turno ya terminó, `TURNO_EXPIRADO`.

9.7 Asistente KAFFA

El Asistente es local:

1. Persiste el mensaje del usuario.
2. Normaliza la pregunta y detecta intención por palabras clave.
3. Consulta la base según el rol:
   - Admin: ventas, productos, inventario, pedidos, gastos, predicción y promociones.
   - Barista: ventas, productos, inventario y pedidos.
   - Cliente: menú, medios de pago, cómo pedir, eventos y sede.
4. Persiste la respuesta.
5. Devuelve `{respuesta, mensaje_id, created_at}`.

No hay llamada externa; si se conecta un LLM, debe mantenerse el aislamiento por rol y usuario.

9.8 Invariantes del sistema

Tabla 75. *Invariantes del sistema y dónde se garantizan*

Invariante	Dónde se garantiza
Un pedido facturado tiene número único	`factura__ventas.numero_factura` + `FacturaService`
No se vende sin stock	Validación previa + transacción
No se opera una caja ajena	`PedidoService`, `CompraService`, `GastoService`
No se cierra una caja con arqueo falseado	`Caja::actualizarRegistro`
No se elimina una caja con movimientos	`Caja::eliminarRegistro`
Un cliente solo ve sus pedidos	`Pedido::scopeByUser`
Un barista solo muta en turno	`EnsureActiveTurno`
Un token no muta fuera de su ability	`EnsureTokenCanAbility`

RECOMENDACIÓN

Recomendación. Cualquier nuevo módulo que toque stock, caja o facturas debe reutilizar `TransactionService` y `lockForUpdate`. No implemente "un pedido rápido" por fuera de `PedidoService`.

10. Organización del código

El backend sigue la estructura estándar de Laravel 10 con una decisión deliberada: **controladores delgados y servicios transaccionales**. Este capítulo explica dónde vive cada responsabilidad para que un ingeniero nuevo sepa qué archivo tocar.

10.1 Mapa de responsabilidades

Tabla 76. *Mapa de responsabilidades por necesidad*

Necesidad	Archivo / carpeta
Añadir un endpoint	`routes/api.php`
Validar un payload	`app/Http/Requests`
Regla de negocio compleja	`app/Services`
CRUD simple con filtros	`app/Models/Concerns/HasBasicQueries.php`
Relaciones y casts	`app/Models`
Autorización por registro	`app/Policies` + `AuthServiceProvider`
Autenticación por rol/turno/ability	`app/Http/Middleware`
Formato de salida	`app/Http/Resources/BaseResource.php`
Efecto secundario	`app/Events` + `app/Listeners`
Error de negocio	`app/Exceptions/BusinessRuleException.php`
Configuración	`config/*.php` + `.env`

10.2 Rutas

`routes/api.php` es la **fuente de verdad de endpoints**. Se carga con prefijo `api/v1` desde `RouteServiceProvider`:

```
Route::middleware('api')
    ->prefix('api/v1')
    ->group(base_path('routes/api.php'));
```

Convenciones:

- Rutas públicas primero (login, registro, catálogo, eventos).
- Grupo `auth:sanctum` para todo lo demás.
- Subgrupos por rol (`role:admin`, `role:admin,barista`) y por escritura (`ability:`).
- Nombres de ruta estables (`pedidos.store`, `cajas.update`) para `route()` y pruebas.
- `whereNumber('id')` en rutas públicas que colisionan con rutas administrativas (`/eventos/admin` vs `/eventos/{id}`).

> **NOTA**
>
> **Nota.** `route:list` muestra 142 entradas porque cada `apiResource` se declara dos veces (lectura y escritura) y los `GET` incluyen `HEAD`. La matriz del capítulo 26 normaliza esas entradas.

10.3 Controladores

Hay 29 controladores de dominio + `Controller` base en `app/Http/Controllers`. Su patrón:

```php
public function store(StorePedidoRequest $request)
{
    return new BaseResource(PedidoService::crear($request->validated()));
}

public function update(UpdatePedidoRequest $request, int $id)
{
    $payload = $request->validated();
    // reglas específicas de autoría
    return new BaseResource(Pedido::actualizarRegistro($id, $payload));
}
```

Buenas prácticas:

- No escribir reglas de stock/caja/factura en el controlador.
- Usar `validated()` y no `$request->all()`.
- Devolver `BaseResource` para ocultar campos sensibles.
- No exponer modelos con `password` o `remember_token`.
- Mantener el método corto; si supera ~30 líneas, mover lógica a un servicio.

10.4 Form Requests

`app/Http/Requests` contiene `Store*Request` y `Update*Request`. Características:

- Reglas explícitas por campo.
- `withValidator` para reglas condicionales (comprobante en medios virtuales).
- `authorize()` devuelve `true` porque la autorización se hace por middleware/policy.
- Mensajes en español por defecto de Laravel; el cliente recibe `errors` por campo.

Ejemplo de regla transversal:

```php
$validator->after(function ($validator) {
    $pagos = $this->input('pagos', []);
    $virtualIds = Medio_Pago::virtualIds();
    foreach ($pagos as $i => $pago) {
        if (in_array($pago['medio_pago_id'] ?? null, $virtualIds)
            && empty($pago['comprobante_url'])) {
            $validator->errors()->add(
                "pagos.$i.comprobante_url",
                'El comprobante es obligatorio para medios de pago virtuales.'
            );
        }
    }
});
```

10.5 Servicios

Tabla 77. *Servicios de negocio y su responsabilidad*

Servicio	Responsabilidad
`PedidoService`	Crear/eliminar pedidos, stock, pagos, factura, caja
`CompraService`	Compras, aumento de stock, egresos
`GastoService`	Gastos, pagos y egresos
`MermaService`	Mermas y ajuste de stock
`FacturaService`	Numeración transaccional
`TurnoService`	Horarios, turno activo, minutos restantes
`TransactionService`	Transacción con reintentos por deadlock
`RegistrationService`	Alta diferida, confirmación y purga
`OtpService`	Generación/validación de códigos
`AsistenteService`	Respuestas locales por rol

Los servicios son clases con métodos estáticos. No reciben `Request`; reciben arrays validados. Esto facilita pruebas y reutilización desde comandos o jobs.

10.6 Modelos y trait HasBasicQueries

Los 29 modelos usan Eloquent. El trait `App\Models\Concerns\HasBasicQueries` centraliza:

- `consultar(array $filtros)`: filtros exactos, búsqueda `LIKE`, rangos de fecha, orden y paginación (1–100, por defecto 15).
- `buscarPorId(int $id)`: `findOrFail` + `Gate::authorize('view-record')`.
- `crearRegistro`, `actualizarRegistro`, `eliminarRegistro`: transacción + gate.
- `scopeByUser`: hook para restringir por rol.
- `eagerRelations`: relaciones a precargar para evitar N+1.

Modelos con lógica especial:

- `Caja`: fuerza `abierta_por`, `turno_id`, calcula arqueo y bloquea eliminación con movimientos.
- `Pedido`: `scopeByUser` limita al cliente a sus pedidos.
- `Medio_Pago`: `virtualIds()` cachea 5 minutos.
- `Turno`: casts de fecha y relación con baristas.
- `User`: roles, `isAdmin()`, `isBarista()`, `isCliente()`, `hasRole()`.

10.7 Policies y gates

`AuthServiceProvider` registra 17 policies de dominio + BasePolicy y tres gates:

Tabla 78. *Gates de autorización y su uso*

Gate	Uso
`view-record`	Ver un registro
`create-record`	Crear un recurso
`manage-record`	Actualizar/eliminar

`BasePolicy` implementa reglas comunes; cada policy ajusta ownership:

- `PedidoPolicy`: cliente solo sus pedidos; barista/admin todos.
- `CajaPolicy`: barista solo su caja.
- `FacturaVentaPolicy`: admin; cliente solo la factura de sus pedidos.
- `MovimientoCajaPolicy`: lectura admin/barista; escritura solo admin.
- `EventoPolicy`: lectura pública de publicados; gestión admin.

10.8 Middleware

Tabla 79. *Middleware registrado y su efecto*

Alias	Clase	Efecto
`role:`	`CheckRole`	Exige uno de los roles
`turno.activo`	`EnsureActiveTurno`	Barista en turno; admin/cliente exentos
`ability:`	`EnsureTokenCanAbility`	Token con al menos una ability
`auth`	`Authenticate`	401 JSON, sin redirección
`throttle`	Laravel	Límite por ruta

Orden recomendado en escrituras: `role` → `turno.activo` → `throttle` → `ability`. El orden actual funciona, pero `role` antes de `ability` da mensajes más claros.

10.9 Recursos y serialización

`BaseResource` elimina `password` y `remember_token` de las respuestas. Laravel envuelve colecciones paginadas en:

```
{
  "data": [ ... ],
  "links": { "first": "...", "last": "...", "prev": null, "next": null },
  "meta": { "current_page": 1, "last_page": 3, "per_page": 15, "total": 42 }
}
```

`AuthController::me` y `updatePerfil` usan `makeHidden` para el mismo fin.

10.10 Eventos y listeners

`PedidoCreado` se dispara al final de `PedidoService::crear`. `DeducirInventario`:

1. Carga `detalles.producto.insumos`.

2. Agrega consumo por insumo.
3. Ejecuta un `UPDATE` por insumo con `GREATEST(0, stock_actual - ?)`.

El listener es síncrono (`QUEUE_CONNECTION=sync`). Si falla, la transacción del pedido se revierte. Si se cambia a cola asíncrona, el descuento podría ocurrir después del commit y dejar una ventana de inconsistencia; habría que rediseñar la idempotencia.

10.11 Excepciones

`BusinessRuleException` lleva mensaje, código HTTP y un código de negocio opcional:

```
throw new BusinessRuleException(
    'La caja indicada no está abierta.',
    422,
    'CAJA_CERRADA'
);
```

`Handler` responde:

```
{ "message": "La caja indicada no está abierta.", "code": "CAJA_CERRADA" }
```

Los 404 de API se normalizan a `{"message": "Recurso no encontrado."}`. Las validaciones usan el formato estándar de Laravel.

10.12 Convenciones de código

- PHP 8.1+: tipos de retorno, propiedades tipadas cuando sea posible.
- PSR-12 y Laravel Pint (`vendor/bin/pint`).
- Nombres de dominio en español (`Pedido`, `Caja`, `Merma`), términos técnicos en inglés (`Service`, `Request`).
- No usar `env()` fuera de `config/`; usar `config('clave')`.
- No usar `DB::raw` con entrada de usuario; usar bindings.
- Fechas con `now()` y zona `America/Bogota`.
- Dinero con `decimal:2`; no usar `float` para cálculos contables críticos.
- Evitar consultas en bucles; usar `with()` y agregaciones.

> **RECOMENDACIÓN**
>
> **Recomendación.** Antes de añadir un módulo, copie el patrón de un recurso existente (ruta → request → controlador → servicio → policy → prueba) y actualice la matriz de endpoints y OpenAPI.

11. Autenticación y seguridad

Este capítulo describe los mecanismos **implementados y verificados**. No es una certificación ni un pentest: es una guía para entender cómo se protege la API y qué debe revisarse antes de exponerla.

11.1 Modelo de autenticación

KAFFA usa **tokens personales de Sanctum**:

- El cliente envía `Authorization: Bearer {token}`.
- El token se crea en `POST /login` y se guarda hasheado en `personal_access_tokens`.
- No hay sesiones por cookies ni CSRF en la API.
- `EnsureFrontendRequestsAreStateful` está comentado; la API es *pure bearer token*.
- `SANCTUM_EXPIRATION=1440` fija 24 horas por defecto.

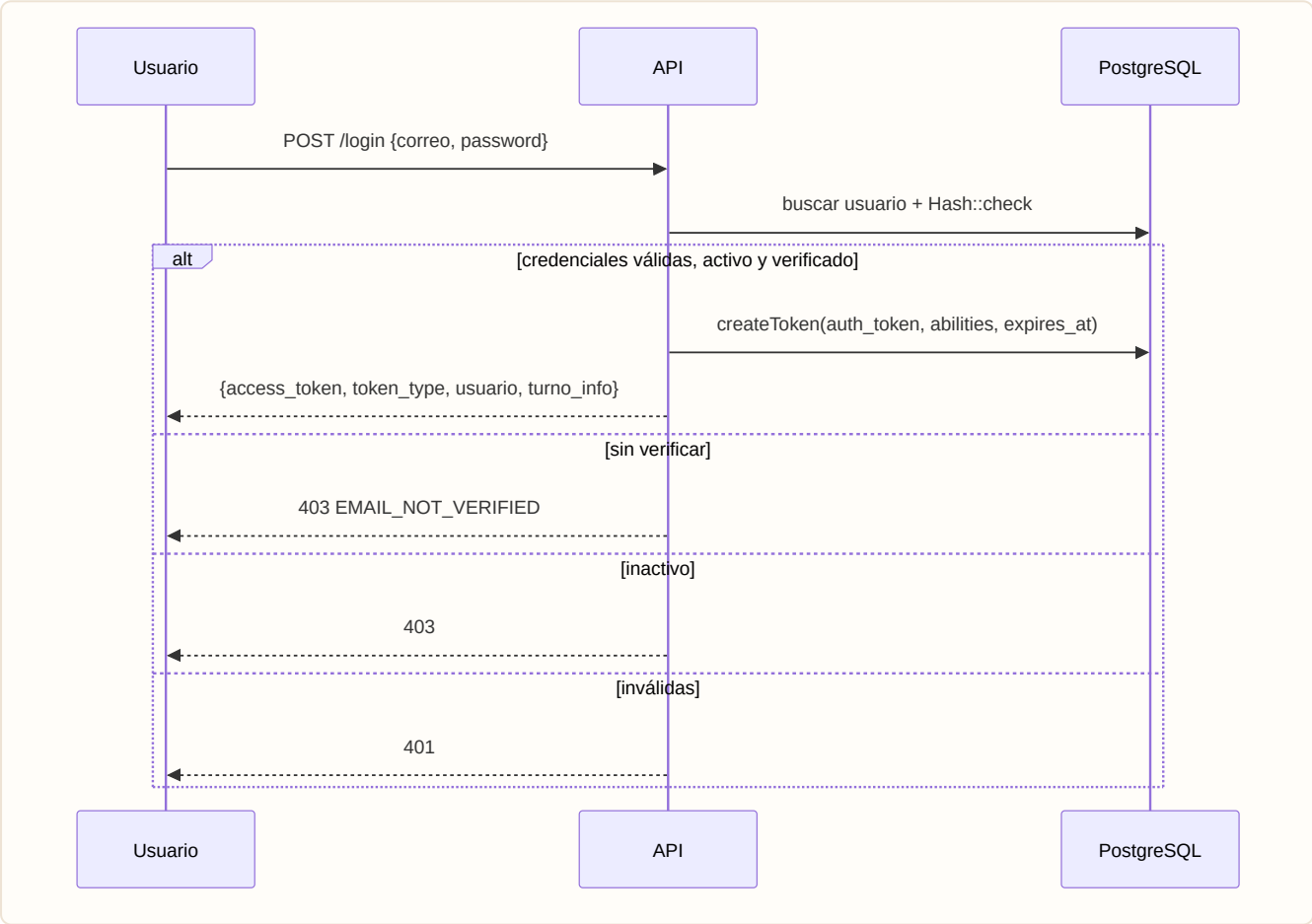


Figura 1. Login y emisión de token.

11.2 Registro y verificación por OTP

Tabla 80. *Flujo de registro y verificación por OTP*

Paso	Endpoint	Efecto
1	`POST /registro`	Guarda pendiente, envía código; **no** crea usuario
2	`POST /email/verify-code`	Valida código y crea usuario verificado con rol `cliente`
3	`POST /email/verification-notification`	Reenvía código (cooldown 60 s)
4	`POST /login`	Exige `email_verified_at` no nulo

Propiedades:

- Código de 6 dígitos, hasheado, un solo uso.
- Expira en 15 minutos; máximo 5 intentos; reenvío cada 60 s.
- Pendientes > 24 h se purgan.
- Respuesta uniforme en reenvío para evitar enumeración.
- Usuarios creados por admin nacen verificados.

> ADVERTENCIA
>
> **Advertencia.** El registro responde 201 aunque el envío de correo falle (se registra en logs). Monitoree los fallos de SMTP: un usuario puede quedar pendiente sin recibir el código.

11.3 Sesión y revocación

Tabla 81. *Endpoints de sesión y revocación de tokens*

Endpoint	Efecto
`POST /logout`	Revoca el token actual
`POST /auth/refresh`	Revoca el actual y emite uno nuevo
`POST /auth/logout-all`	Revoca todos los tokens del usuario
`POST /reset-password`	Cambia contraseña y revoca todos los tokens

La rotación con `refresh` permite sesiones largas sin re-login. La SPA actual **no** invoca `refresh`; si el token expira, el usuario debe volver a iniciar sesión. Es un `[GAP]` de integración, no del backend.

11.4 Roles

Tres roles sembrados:

Tabla 82. *Roles del sistema*

Rol	Descripción
`admin`	Acceso total; token con ability `*`
`barista`	Operación POS con turno activo
`cliente`	Autoservicio

Un usuario puede tener varios roles. `User::hasRole()` consulta la pivote `usuario__rols`. El middleware `role:` acepta una lista separada por comas y exige **alguno**.

Respuestas:

- Sin token: `401 {"message":"No autenticado."}`.
- Sin rol: `403 {"message":"No tienes permiso para realizar esta acción."}`.

11.5 Abilities

Las abilities se acuñan en `AuthController::abilitiesFor`:

Tabla 83. *Abilities asignadas a cada rol*

Rol	Abilities
Admin	`*`
Barista	`pedidos:manage`, `compras:manage`, `gastos:manage`, `mermas:manage`, `cajas:manage`, `movimiento-cajas:read`, `proveedores:read`, `catalogo:read`, `turnos:read`
Cliente	`catalogo:read`, `pedidos:create`, `pedidos:read`

El middleware `ability:` acepta varias abilities separadas por coma y exige **alguna**. Si el token no la posee:

```
{
  "message": "Tu token no tiene el permiso requerido para esta acción.",
  "code": "TOKEN_SIN_ABILITY",
  "required_ability": ["pedidos:manage"]
}
```

> NOTA
>
> **Nota de migración.** Los tokens emitidos antes de un cambio de abilities conservan las antiguas. El usuario debe reautenticarse o rotar con `POST /auth/refresh`.

11.6 Matriz RBAC resumida

Tabla 84. *Matriz RBAC resumida*

Capacidad	Admin	Barista	Cliente
Catálogo: lectura	✔	✔	✔ (categorías/productos sin sesión)
Catálogo: escritura	✔	—	—
Eventos: lectura	✔	✔	✔ (publicados)
Eventos: gestión	✔	—	—
Usuarios/roles/turnos/facturas	✔	—	—
Pedidos: ver	Todos	Todos	Solo propios
Pedidos: crear	✔	✔ (mostrador)	✔ (propios, pendiente)
Pedidos: actualizar/eliminar	✔	✔ (turno)	—
Pagos: registrar	✔	✔ (turno)	—

Capacidad	Admin	Barista	Cliente
Compras/gastos/mermas/cajas	✔	✔ (turno)	—
Proveedores: escritura	✔	—	—
Movimientos de caja: escritura	✔	—	—
Chat	✔	✔	✔ (con personal)
Reportes: crear	✔	✔	—
Reportes: gestionar	✔	—	—
Asistente KAFFA	✔	✔	✔ (sin métricas)

11.7 Policies y ownership

Además del rol, cada registro se evalúa con policies:

- **Pedido**: cliente solo `cliente_id = Auth::id()`; barista/admin todos.
- **Caja**: barista solo `abierta_por = Auth::id()`; admin todas.
- **Factura**: admin; cliente solo la de sus pedidos.
- **Movimiento de caja**: lectura admin/barista; escritura solo admin.
- **Evento**: lectura pública de publicados; gestión admin.
- **Usuario/Rol/Turno**: admin.

Los gates `view-record`, `create-record` y `manage-record` se invocan desde `HasBasicQueries` y los controladores. Si una policy devuelve `false`, Laravel responde 403.

11.8 Middleware `turno.activo`

Solo afecta a baristas. Admin y cliente pasan sin validación.

Tabla 85. *Respuestas del middleware de turno activo*

Situación	Código	HTTP
Fuera de 05:00–22:00	`FUERA_HORARIO`	403
Sin turno que cubra la hora	`TURNO_INACTIVO`	403
Turno terminado	`TURNO_EXPIRADO`	403
Turno válido	—	Continúa

El middleware inyecta `turno_activo` en el request. Los servicios pueden usarlo para asociar la caja al turno.

11.9 Throttling

Tabla 86. *Límites de throttling por grupo*

Grupo	Límite
API global	60/min por usuario/IP
Login	5/min

Grupo	Límite
Registro	10/min
Forgot/reset/reenvío	5/min
Verify-code	10/min
Catálogo/eventos públicos	60/min
Escrituras de catálogo/eventos	30/min
Escrituras operativas/pedidos	10/min
Asistente	30/min

El límite global se define en `RouteServiceProvider` con `Limit::perMinute(60)->by(user?->id ?: ip)`. En rutas autenticadas, el cupo es por usuario; en públicas, por IP.

> **ADVERTENCIA**
>
> **Advertencia.** El throttle de escritura es compartido por usuario entre rutas del mismo grupo. Por eso las lecturas operativas se separaron: listar `GET /compras` no debe consumir el cupo de `POST /compras`.

11.10 CORS

`config/cors.php`:

- Orígenes desde `CORS_ALLOWED_ORIGINS` (coma-separados).
- Métodos explícitos.
- `supports_credentials=false`.
- `max_age=86400`.

En producción, nunca `*`. Si hay múltiples frontends, enumere cada origen exacto (esquema + host + puerto).

11.11 Protección de datos

- `BaseResource` oculta `password` y `remember_token`.
- `AuthController::me` y `updatePerfil` aplican `makeHidden`.
- `GET /equipo` expone solo `id`, `nombre`, `foto`; nunca correos.
- `Handler::$dontFlash` excluye contraseñas de trazas.
- `TrimStrings` y `ConvertEmptyStringsToNull` están activos.
- `ValidatePostSize` limita el cuerpo.
- Los códigos OTP se guardan hasheados.
- Los logs de error de correo incluyen correo y mensaje, no contraseña.

11.12 Hallazgos y estado

Tabla 87. *Hallazgos de seguridad y estado*

#	Hallazgo	Estado	Acción
1	`.env` en el directorio	Corregido/no era riesgo	`.gitignore` lo excluye; solo `.env.example` versionado
2	`.env.example` con MySQL	Resuelto	`pgsql` /5432/ `kaffa`
3	Tokens sin expiración	Resuelto	`SANCTUM_EXPIRATION=1440`
4	CORS `*`	Resuelto	Origen de Vite por defecto; dominio real en prod
5	`APP_DEBUG=true`	Mitigado	Se mantiene en dev; checklist de producción
6	`QUEUE_CONNECTION=sync`	Documentado	Correcto en dev; Redis en prod
7	Credenciales reales en historial de Git	Mitigado / pendiente	Rotar contraseña de PostgreSQL y purgar historial si el repo fue público

ADVERTENCIA

Advertencia. Este manual no reproduce la contraseña filtrada. Si el repositorio estuvo expuesto, rote la credencial y revise accesos aunque el archivo ya no la contenga.

11.13 Checklist de seguridad

- ☐ HTTPS obligatorio; HSTS si es posible.
- ☐ `APP_DEBUG=false` y `APP_ENV=production`.
- ☐ `CORS_ALLOWED_ORIGINS` restringido.
- ☐ `SANCTUM_EXPIRATION` definido.
- ☐ Contraseñas de usuarios semilla cambiadas.
- ☐ Cloudinary preset restringido; API secret fuera del frontend.
- ☐ Scramble desactivado.
- ☐ Logs sin secretos y con rotación.
- ☐ Backups cifrados y con acceso limitado.
- ☐ Dependencias actualizadas (`composer audit`, `pnpm audit`).
- ☐ Revisar policies al añadir rutas nuevas.
- ☐ No confiar solo en `role:`; aplicar `ability:` y policy.
- ☐ Monitorear 401/403/429 anómalos.

NOTA

Evaluación de calidad. El cumplimiento de estos controles se mide frente a ISO/IEC 27001:2022 en el capítulo 32 (§32.4).

12. Servicios de negocio

Los servicios son el corazón del backend. Este capítulo explica su algoritmo, las tablas que tocan, los bloqueos que usan y los errores que producen. Está pensado para modificar código sin romper invariantes.

12.1 `TransactionService`

```php
public static function run(Closure $callback, int $attempts = 3)
{
    return DB::transaction($callback, $attempts);
}
```

`DB::transaction` hace commit si el callback termina y rollback si lanza una excepción. `$attempts=3` reintenta ante **deadlocks** (PostgreSQL `40P01`, MySQL `1213`). No reintenta errores de validación ni de negocio.

> ADVERTENCIA
>
> **Advertencia.** El callback debe ser idempotente ante reintentos. No envíe correos ni llame APIs externas dentro de una transacción con reintentos.

12.2 `PedidoService::crear`

12.2.1 Algoritmo

```
1. Abrir transacción.
2. Obtener usuario autenticado; si no, 401.
3. Validar estado permitido.
4. Resolver autoría:
   - admin: cliente_id/barista_id del payload
   - barista: barista_id = Auth::id(); cliente_id opcional
   - cliente: cliente_id = Auth::id(); estado = pendiente
5. Si caja_id:
   - lockForUpdate
   - existe, abierta, propia del barista
6. Crear pedido.
7. Cargar productos y recetas; agregar consumo por insumo.
8. lockForUpdate insumos; validar stock.
9. Crear detalles; sumar subtotales.
10. Crear pagos; sumar montos.
11. Validar Σ pagos ≈ total (±0.01).
12. Si estado=pagado y no hay pagos: 422.
13. Si factura: generar número y crear factura.
14. Si caja: crear movimiento ingreso por cada pago.
15. Actualizar total si corresponde.
16. Disparar PedidoCreado → DeducirInventario.
17. Devolver pedido con relaciones.
```

12.2.2 Errores de negocio

Tabla 88. *Errores de negocio de PedidoService*

Código	HTTP	Condición
`PERMISO_DENEGADO`	403	Rol no autorizado
`CAJA_NO_EXISTE`	422	`caja_id` inexistente
`CAJA_CERRADA`	422	Caja no abierta
`CAJA_AJENA`	422	Barista usa caja de otro
—	422	Stock insuficiente
—	422	Σ pagos ≠ total
—	422	Pedido pagado sin pagos

12.2.3 Pseudocódigo de stock

```
consumo = {}
for detalle in detalles:
    producto = Producto::with('insumos')->find(detalle.producto_id)
    for insumo in producto.insumos:
        consumo[insumo.id] += detalle.cantidad * insumo.pivot.cantidad

insumos = Insumo::lockForUpdate()->whereIn('id', consumo.keys())->get()
for id, cantidad in consumo:
    if insumos[id].stock_actual < cantidad:
        throw Stock insuficiente
```

12.2.4 Eliminación

`PedidoService::eliminar` borra detalles, movimientos de caja, pagos y factura, y hace soft delete del pedido. **No repone stock.** Si el pedido fue un error, registre una merma o un ajuste.

12.3 `CompraService::crear`

```
1. Transacción.
2. Validar caja (si se envía) con lockForUpdate.
3. Crear compra.
4. Por cada detalle:
   - lockForUpdate insumo
   - crear detalle
   - incrementar stock_actual
5. Crear pagos.
6. Validar Σ pagos = total_compra (±0.01).
7. Crear movimientos egreso por pago.
8. Devolver compra.
```

Al eliminar:

```
1. Transacción.
2. lockForUpdate insumos de los detalles.
3. Verificar que stock_actual - cantidad >= 0 para cada uno.
4. Restar stock.
5. Borrar detalles, pagos, movimientos y compra (soft delete).
```

Si la reversión dejaría stock negativo, lanza `BusinessRuleException` y no elimina.

12.4 `GastoService`

- Fuerza `usuario_id = Auth::id()`.
- Crea el gasto y sus pagos.
- Valida comprobante en medios virtuales.
- Genera movimientos `egreso` si hay caja.
- Al eliminar, borra pagos y movimientos y hace soft delete.

El monto del gasto **no** vive en `gastos`; se calcula sumando `gasto__pagos`. Las consultas de reportes deben agregar esa tabla.

12.5 `MermaService`

```
1. Transacción.
2. lockForUpdate insumo.
3. Validar cantidad <= stock_actual.
4. Crear merma con usuario_id = Auth::id().
5. Restar stock.
6. Al eliminar: sumar stock y borrar merma.
```

La merma no genera movimiento de caja; es una pérdida de inventario, no un egreso monetario.

12.6 `Caja`

12.6.1 Apertura (`Caja::crearRegistro`)

```php
$data['abierta_por'] = Auth::id();
$data['estado'] = $data['estado'] ?? 'abierta';
$data['fecha_apertura'] = $data['fecha_apertura'] ?? now();
$data['turno_id'] = TurnoService::getActiveTurno(Auth::user())?->id;
$data['monto_apertura_sistema'] = $data['monto_apertura_fisico'] ?? 0;
```

El `turno_id` es `null` para admin. El desglose efectivo/digital es opcional; si falta, todo el físico se asume efectivo.

12.6.2 Cierre (`Caja::actualizarRegistro`)

```php
$vaACerrar = ($data['estado'] ?? null) === 'cerrada' || isset($data['monto_cierre_fisico']);
if ($vaACerrar) {
    $data['cerrada_por'] = Auth::id();
    $data['fecha_cierre'] = $data['fecha_cierre'] ?? now();
    unset($data['monto_cierre_sistema']);
    $caja = self::with('movimientos')->findOrFail($id);
    $apertura = (float) $caja->monto_apertura_sistema;
    $ingresos = $caja->movimientos->where('tipo','ingreso')->sum('monto');
    $egresos  = $caja->movimientos->where('tipo','egreso')->sum('monto');
    $data['monto_cierre_sistema'] = $apertura + $ingresos - $egresos;
}
```

El cliente puede declarar `monto_cierre_fisico`, efectivo y digital, pero **no** el sistema.

12.6.3 Eliminación

Si `movimientos()->exists()`, lanza 422. No hay borrado en cascada de movimientos.

12.7 `FacturaService::generarNumeroFactura`

```php
DB::table('factura_contadores')->upsert(
    ['nombre' => 'F', 'anio' => $anio, 'valor' => 0],
    ['nombre', 'anio'],
    ['nombre', 'anio']
);
$contador = DB::table('factura_contadores')
    ->where('nombre', 'F')->where('anio', $anio)
    ->lockForUpdate()->first();
$siguiente = (int) $contador->valor + 1;
DB::table('factura_contadores')->where('id', $contador->id)
    ->update(['valor' => $siguiente, 'updated_at' => now()]);
return 'F' . $anio . str_pad($siguiente, 6, '0', STR_PAD_LEFT);
```

Propiedades:

- UPSERT + `SELECT ... FOR UPDATE` evita carreras.
- El contador es por año.
- No hay huecos si la transacción falla (rollback).
- El formato es `F{AÑO}{NNNNNN}`.

> ADVERTENCIA
>
> **Advertencia.** No llame `generarNumeroFactura` fuera de una transacción; el bloqueo se libera al terminar la sentencia y podría duplicar números.

12.8 `TurnoService`

12.8.1 Horarios

Tabla 89. *Horarios de TurnoService*

Tipo	Inicio	Fin
`mañana`	07:00	13:00
`tarde`	13:00	18:00
`abierto`	05:00	22:00

12.8.2 Turno activo

```
ahora = now(America/Bogota)
if 05:00 <= hora < 22:00:
    buscar turno 'abierto' del barista que cubra hoy
    if existe: return
tipo = 'mañana' si 07:00–13:00; 'tarde' si 13:00–18:00; null fuera
buscar turno del tipo que cubra hoy
```

Un turno cubre hoy si `fecha <= hoy <= fecha_fin` o `fecha_fin` es nula y `fecha = hoy`.

12.8.3 Minutos restantes

`minutosRestantes` usa la hora fin del tipo y la hora actual. Puede ser negativo si el turno ya terminó; el middleware responde `TURNO_EXPIRADO`.

12.9 `RegistrationService` y `OtpService`

12.9.1 Registro

```
iniciar(nombre, correo, password):
  if usuario existe: validar unique (422)
  upsert registros_pendientes (password hashed)
  generar OTP tipo verificacion
  enviar correo
```

Si el correo ya existe como usuario, `Store` falla por `unique:users,correo`. Si existe como pendiente, se actualiza y se reenvía.

12.9.2 Confirmación

```
confirmar(correo, codigo):
  buscar pendiente
  validar OTP
  DB::transaction:
    crear User verificado (email_verified_at=now)
    asignar rol cliente
    eliminar pendiente
```

12.9.3 OTP

- `generar`: `updateOrCreate` por `(correo,tipo)`, hash, expiración, intentos=0, enviado_at=now.
- `validar`: no encontrado, expirado, demasiados, incorrecto (incrementa), ok (elimina).
- `puedeReenviar`: compara `enviado_at` con el cooldown.

Los parámetros se leen de `config/otp.php`, que a su vez lee `.env`.

12.10 `AsistenteService`

El servicio es local y deterministico:

```
responder(pregunta, usuarioId, rol):
  persistir mensaje user
  respuesta = generarRespuesta(pregunta, rol)
  persistir mensaje assistant
  return {respuesta, mensaje_id, created_at}
```

Detección de intención por palabras clave normalizadas (sin tildes, minúsculas). Staff:

Tabla 90. *Intenciones detectadas por el AsistenteService*

Intención	Palabras	Respuesta
Predicción	predice, pronóstico, tendencia	Promedio móvil 7 días (solo admin)
Promoción	marketing, descuento, oferta	Recomendaciones (solo admin)
Gastos	gasto, egreso, margen	Resumen de gastos (solo admin)
Inventario	insumo, stock, reponer	Insumos bajo mínimo
Productos	más vendido, top, ranking	Top de productos
Pedidos	pedido, pendiente, cola	Conteo por estado
Ventas	venta, ingreso, factura, caja	Resumen hoy/7 días/mes

Cliente: menú, medios de pago, cómo pedir, eventos, sede. No recibe métricas internas.

RECOMENDACIÓN

Recomendación. Si se conecta un LLM, mantenga la detección de intención y el filtro por rol; no envíe el contexto completo de la base al proveedor.

12.11 Tabla resumen de servicios

Tabla 91. *Resumen transaccional de los servicios*

Servicio	Transacción	Locks	Efectos
PedidoService	Sí	Caja, insumos	Pedido, pagos, factura, caja, stock
CompraService	Sí	Caja, insumos	Compra, stock, pagos, caja
GastoService	Sí	Caja	Gasto, pagos, caja
MermaService	Sí	Insumo	Merma, stock
FacturaService	Requiere	Contador	Número de factura
TurnoService	Sí (crear/editar)	—	Turno y baristas
RegistrationService	Sí (confirmar)	—	Usuario, rol, pendiente
OtpService	No	—	Código OTP
AsistenteService	No	—	Mensajes

13. Errores y códigos de negocio

La API devuelve JSON uniforme. Este capítulo reúne los códigos HTTP, los códigos de negocio y la forma de diagnosticarlos. Un cliente puede leer `message` para mostrar un texto y `code` para decidir una acción sin analizar el texto.

13.1 Formato general

```
{
  "message": "La caja indicada no está abierta.",
  "code": "CAJA_CERRADA"
}
```

Errores de validación (422):

```
{
  "message": "The given data was invalid.",
  "errors": {
    "pagos.0.comprobante_url": [
      "El comprobante es obligatorio para medios de pago virtuales."
    ]
  }
}
```

13.2 Códigos HTTP

Tabla 92. *Uso de los códigos HTTP*

HTTP	Uso	Ejemplo
200	Lectura/actualización correcta	`GET /me`
201	Creación correcta	`POST /pedidos`
401	Sin token o token inválido	`GET /insumos` sin Bearer
403	Rol/ability/turno insuficiente	`TOKEN_SIN_ABILITY`
404	Recurso o ruta inexistente	`GET /pedidos/999999`
422	Validación o regla de negocio	Stock insuficiente
429	Throttle excedido	11 escrituras/min
500	Error no controlado	Fallo de base o bug

13.3 Códigos de negocio

Tabla 93. *Catálogo de códigos de negocio*

Código	HTTP	Significado	Acción del cliente
`EMAIL_NOT_VERIFIED`	403	Correo sin verificar	Pedir código y verificar
`FUERA_HORARIO`	403	Barista fuera de 05:00–22:00	Esperar o pedir turno

Código	HTTP	Significado	Acción del cliente
`TURNO_INACTIVO`	403	Sin turno asignado	Contactar admin
`TURNO_EXPIRADO`	403	Turno terminado	Cerrar caja
`PERMISO_DENEGADO`	403	Rol no autorizado a crear pedido	Revisar rol
`CAJA_NO_EXISTE`	422	`caja_id` inexistente	Corregir caja
`CAJA_CERRADA`	422	Caja no abierta	Abrir caja
`CAJA_AJENA`	422	Caja de otro barista	Usar la propia
`TOKEN_SIN_ABILITY`	403	Token sin alcance	Reautenticar/rotar

`TOKEN_SIN_ABILITY` incluye `required_ability` con la lista de abilities aceptadas.

13.4 Dónde se lanza cada error

Tabla 94. *Origen de cada error*

Origen	Tipo	Ejemplo
`CheckRole`	401/403	Rol incorrecto
`EnsureActiveTurno`	403	`FUERA_HORARIO`
`EnsureTokenCanAbility`	403	`TOKEN_SIN_ABILITY`
Form Requests	422	Campo requerido
`PedidoService`	422	Stock, pagos, caja
`CompraService`	422	Stock negativo al revertir
`Caja`	422	Caja con movimientos
`FacturaService`	500	Contador no inicializado
`Handler`	404	Modelo/ruta no encontrada
Laravel	429	Throttle

13.5 Errores de autenticación

Tabla 95. *Errores de autenticación y su respuesta*

Caso	Respuesta
Credenciales inválidas	`401 {"message":"Credenciales inválidas."}`
Usuario inactivo	`403 {"message":"Este usuario está inactivo..."}`
Correo sin verificar	`403 {"code":"EMAIL_NOT_VERIFIED"}`
Token expirado	`401 {"message":"No autenticado."}`
Token revocado	`401`

La API **nunca** redirige a una página de login HTML; `Authenticate::redirectTo` devuelve `null`.

13.6 Errores de validación frecuentes

Tabla 96. *Errores de validación frecuentes*

Campo	Regla	Mensaje típico
`correo`	email, único	"The correo has already been taken."
`password`	8+, mayúscula, minúscula, número, símbolo	"The password must contain..."
`pagos.*.comprobante_url`	obligatorio si medio virtual	"El comprobante es obligatorio..."
`detalles.*.cantidad`	entero ≥1	"The cantidad must be at least 1."
`monto_apertura_fisico`	requerido, ≥0	"The monto apertura fisico field is required."
`mensaje` (chat)	máx. 1000	"The mensaje may not be greater than 1000 characters."
`mensaje` (asistente)	máx. 2000	"The mensaje may not be greater than 2000 characters."

13.7 Diagnóstico por síntoma

Tabla 97. *Diagnóstico de errores por síntoma*

Síntoma	Causa probable	Dónde mirar
401 en todo	Token ausente/expirado	`Authorization`, `SANCTUM_EXPIRATION`
403 con rol correcto	Falta ability o turno	`EnsureTokenCanAbility`, `EnsureActiveTurno`
422 en pago virtual	Falta comprobante	`withValidator`
422 en pedido	Stock o suma de pagos	`PedidoService`
404 en ruta existente	Prefijo `/api/v1` mal	`RouteServiceProvider`
429 tras listar	Throttle compartido	`routes/api.php`
500 con traza	Bug o base caída	`storage/logs/laravel.log`
JSON truncado	Error fatal de PHP	Log de PHP-FPM

13.8 Logging

Laravel registra excepciones en `storage/logs/laravel.log` (o diario). Para diagnóstico:

```
tail -f storage/logs/laravel.log
```

No registre tokens, contraseñas, códigos OTP ni cuerpos completos con datos personales. Si necesita depurar, use un identificador de correlación y redacte campos sensibles.

13.9 Recomendaciones para clientes

- Lea `code` antes que `message`.
- Para 422, recorra `errors` y marque los campos.
- Para 401, limpie el token y redirija a login.

- Para 403 `TOKEN_SIN_ABILITY`, renueve el token.
- Para 429, respete `Retry-After` si está presente y aplique backoff.
- No reintente automáticamente un `POST /pedidos` sin idempotencia: puede duplicar la venta.

14. Concurrencia, transacciones y rendimiento

KAFFA opera con varios baristas y clientes simultáneos. Este capítulo explica cómo evita sobreventa, cierres dobles, facturas duplicadas y consultas lentas.

14.1 Transacciones

Toda operación que toca más de una tabla usa `TransactionService::run`:

- Pedido: pedido + detalles + pagos + factura + movimientos + stock.
- Compra: compra + detalles + pagos + stock + movimientos.
- Gasto: gasto + pagos + movimientos.
- Merma: merma + stock.
- Caja: apertura/cierre + arqueo.

Si cualquier paso lanza una excepción, PostgreSQL hace rollback. Esto evita estados parciales.

14.2 Bloqueos pesimistas

Tabla 98. *Bloqueos pesimistas por recurso*

Recurso	Lock	Motivo
Caja	`lockForUpdate`	Evitar doble cierre/uso concurrente
Insumos	`lockForUpdate`	Evitar sobreventa
Contador de facturas	`lockForUpdate`	Evitar números duplicados
Insumo en compra/merma	`lockForUpdate`	Evitar stock negativo

`lockForUpdate` bloquea la fila hasta el commit/rollback. Mantenga las transacciones cortas: no envíe correos ni llame APIs externas dentro.

14.3 Deadlocks

Un deadlock ocurre cuando dos transacciones se bloquean mutuamente. PostgreSQL aborta una con `40P01`. `DB::transaction($callback, 3)` reintenta hasta 3 veces.

Para reducir deadlocks:

- Bloquee filas en orden consistente (por ID ascendente).
- Agrupe consumos por insumo antes de bloquear.
- Evite transacciones largas.
- No mezcle operaciones de distintos dominios en una misma transacción.

14.4 Inventario atómico

El listener usa:

```
UPDATE insumos
SET stock_actual = GREATEST(0, stock_actual - ?)
WHERE id = ?
```

Ventajas:

- Un solo statement por insumo.
- Sin `SELECT` + `UPDATE` propenso a carrera.
- `GREATEST` evita negativos.
- Bindings parametrizados, sin SQL injection.

La validación previa y el descuento ocurren en la misma transacción, pero el `UPDATE` es la última barrera.

14.5 Facturación sin carreras

Secuencia:

1. `UPSERT` del contador `(nombre, anio)`.
2. `SELECT ... FOR UPDATE`.
3. Incremento en memoria.
4. `UPDATE`.
5. Retorno del número.

El bloqueo se libera al commit. Si dos pedidos intentan facturar a la vez, el segundo espera y obtiene el siguiente número.

14.6 Paginación y filtros

`HasBasicQueries::consultar`:

- `per_page` por defecto 15, máximo 100.
- Filtros exactos por columnas `fillable` + PK.
- `LIKE` para `nombre`, `descripcion`, `correo`, `nit`, `direccion`, `motivo`.
- `fecha_desde` / `fecha_hasta` sobre `created_at`.
- `sort_by` y `sort_order` validados.
- `eagerRelations` para evitar N+1.

Ejemplo:

```
GET /api/v1/pedidos?estado=pagado&fecha_desde=2026-10-01&sort_by=created_at&sort_order=desc&per_page=50
```

14.7 Índices

La migración `2026_07_31_000000_add_performance_indexes_to_tables.php` añade índices a:

- `pedidos(estado)`, `pedidos(created_at)`
- `pago__pedidos(created_at)`
- `compras(created_at)`, `compras(numero_factura_proveedor)`

- `gastos(created_at)`, `gastos(categoria)`
- `mermas(created_at)`
- `movimiento__cajas(created_at)`, `movimiento__cajas(tipo)`
- `factura__ventas(created_at)`
- `cajas(estado)`

Además hay únicos en `factura__ventas(pedido_id, numero_factura)`, `proveedors.nit`, `turnos(fecha,tipo)`, etc.

> **RECOMENDACIÓN**
> **Recomendación.** Antes de añadir un índice, mida con `EXPLAIN (ANALYZE, BUFFERS)`. Un índice en una columna de baja cardinalidad (p. ej. `activo`) puede no usarse.

14.8 Caché

`Medio_Pago::virtualIds()` cachea los IDs de medios virtuales 5 minutos con la clave `medios_pago:virtual_ids`. Se invalida al guardar/eliminar un medio.

Si cambia `CACHE_DRIVER` a Redis, revise la serialización y la expiración. No cachee datos sensibles por usuario sin una clave que incluya el usuario.

14.9 Capacidad y límites

Tabla 99. *Límites de capacidad y cuellos de botella*

Recurso	Límite actual	Cuello de botella
API global	60/min por usuario/IP	Picos de sondeo
Escrituras operativas	10/min	Flujo de caja
Conexiones PostgreSQL	Según `max_connections`	Workers PHP-FPM
Correo	`MAIL_TIMEOUT=10`	SMTP lento
Imágenes	5 MB	Cloudinary/disco

Para una cafetería pequeña, un servidor de 2 vCPU/4 GB y PostgreSQL local es suficiente. Para varias sedes, separe la base y use Redis para caché/colas.

14.10 Pruebas de carga recomendadas

1. Login concurrente de 50 usuarios.
2. 20 pedidos simultáneos sobre el mismo insumo.
3. Dos cierres de caja simultáneos.
4. 100 facturas concurrentes.
5. Sondeo de pedidos/chat durante 10 minutos.

Mida latencia p95, errores 5xx, deadlocks y uso de CPU/IO. No ejecute pruebas de carga contra producción sin autorización.

14.11 Antipatrones prohibidos

- `SELECT` de stock y `UPDATE` posterior sin lock.
- Generar número de factura con `MAX(numero)+1`.
- Abrir transacción y enviar correo dentro.
- Reintentar un `POST` sin idempotencia.
- Consultar relaciones dentro de un bucle (N+1).
- Usar `float` para sumas contables críticas.
- Desactivar `lockForUpdate` “para que sea más rápido”.

15. Convenciones de la API

Esta parte es la referencia práctica para integrar clientes. La fuente de verdad es `routes/api.php` + `openapi.json`; la matriz completa está en el capítulo 26.

15.1 Base URL y versionado

Tabla 100. *Base URL por entorno*

Entorno	Base URL
Desarrollo	`http://127.0.0.1:8000/api/v1`
Producción	`https://api.dominio.com/api/v1`
SPA en el mismo dominio	`/api/v1` (proxy)

El prefijo `api/v1` se registra en `RouteServiceProvider`. No hay versión por cabecera. Para cambios incompatibles, cree `/api/v2`.

15.2 Autenticación

```
Authorization: Bearer 1|xxxxxxxxxxxxxxxxxxxxxxx
Accept: application/json
Content-Type: application/json
```

- El token se obtiene en `POST /login`.
- Sin token, las rutas protegidas responden `401` JSON.
- El token expira según `SANCTUM_EXPIRATION` (1440 min por defecto).
- `POST /auth/refresh` rota el token; `POST /logout` revoca el actual.

15.3 Formato de datos

- JSON UTF-8.
- Fechas en ISO 8601 (`2026-10-20T15:47:46-05:00`); conformidad evaluada en §32.8.
- Fechas de negocio (`fecha` de turno/evento) en `YYYY-MM-DD`.
- Hora en `HH:MM` o `HH:MM:SS`.
- Dinero como número decimal (COP sin decimales en la práctica, pero el esquema admite 2).
- Booleanos JSON (`true` / `false`), no `1` / `0`.

15.4 Paginación

Todas las listas usan la envoltura de Laravel:

```
{
  "data": [ ... ],
  "links": {
    "first": "http://.../api/v1/pedidos?page=1",
```

```
    "last": "http://.../api/v1/pedidos?page=3",
    "prev": null,
    "next": "http://.../api/v1/pedidos?page=2"
  },
  "meta": {
    "current_page": 1,
    "from": 1,
    "last_page": 3,
    "per_page": 15,
    "to": 15,
    "total": 42
  }
}
```

Tabla 101. *Parámetros de paginación*

Parámetro	Valor
`page`	Página (1..n)
`per_page`	1–100; por defecto 15
`sort_by`	Columna permitida
`sort_order`	`asc` / `desc` (por defecto `desc`)
`fecha_desde`	`YYYY-MM-DD` sobre `created_at`
`fecha_hasta`	`YYYY-MM-DD` sobre `created_at`

15.5 Filtros

Filtros exactos por columnas `fillable` y PK:

```
GET /api/v1/pedidos?estado=pagado&cliente_id=3
GET /api/v1/productos?categoria_id=2&activo=1
GET /api/v1/usuarios?rol=barista
```

Búsqueda parcial (`LIKE`) en `nombre`, `descripcion`, `correo`, `nit`, `direccion`, `motivo`:

```
GET /api/v1/productos?nombre=capu
```

Los filtros desconocidos se ignoran; no producen error. Si necesita un filtro nuevo, agréguelo al modelo y a la matriz.

15.6 Ordenamiento

```
GET /api/v1/pedidos?sort_by=created_at&sort_order=desc
```

Por defecto ordena por PK descendente. `sort_by` debe ser una columna permitida; si no, se ignora y se usa la PK.

15.7 Códigos de estado y errores

Tabla 102. *Códigos de estado y su significado*

Código	Significado
200	OK
201	Creado
204	Sin contenido (no usado de forma general)
401	No autenticado
403	Sin permiso/rol/ability/turno
404	No encontrado
422	Validación o regla de negocio
429	Throttle
500	Error interno

El cuerpo de error se describe en el capítulo 13.

15.8 Throttle

Tabla 103. *Límites de throttle de la API*

Grupo	Límite
Global	60/min por usuario/IP
Login	5/min
Registro	10/min
Recuperación	5/min
Verify-code	10/min
Escrituras de catálogo/eventos	30/min
Escrituras operativas	10/min
Asistente	30/min

Si recibe 429, espere y aplique backoff. No reintente inmediatamente.

15.9 Ejemplo `curl` completo

```
BASE=http://127.0.0.1:8000/api/v1

# 1. Login
TOKEN=$(curl -s -X POST "$BASE/login" \
  -H 'Content-Type: application/json' \
  -d '{"correo":"admin@kaffa.com","password":"admin123"}' | jq -r .access_token)

# 2. Perfil
curl -s "$BASE/me" -H "Authorization: Bearer $TOKEN" | jq

# 3. Listar productos
```

```
curl -s "$BASE/productos?per_page=5" -H "Authorization: Bearer $TOKEN" | jq

# 4. Crear categoría
curl -s -X POST "$BASE/categorias" \
  -H "Authorization: Bearer $TOKEN" \
  -H 'Content-Type: application/json' \
  -d '{"nombre":"Tés"}' | jq

# 5. Caja
curl -s -X POST "$BASE/cajas" \
  -H "Authorization: Bearer $TOKEN" \
  -H 'Content-Type: application/json' \
  -d '{"monto_apertura_fisico":100000,"monto_apertura_efectivo":80000,"monto_apertura_digital":20000}' |
jq
```

15.10 Idempotencia y reintentos

La API **no** implementa `Idempotency-Key`. Un reintento de `POST /pedidos` puede duplicar la venta. Recomendaciones para clientes:

- No reintentar automáticamente escrituras no idempotentes.
- Si hay timeout, consultar primero si el recurso se creó (por ejemplo, pedidos del cliente en los últimos minutos).
- Para pagos, usar una referencia externa y conciliación manual.
- Implementar idempotencia en el backend si se integra una pasarela.

15.11 Zona horaria y dinero

- La aplicación opera en `America/Bogota`; los timestamps se serializan con offset.
- Los reportes “de hoy” usan la fecha local, no UTC.
- Los importes se guardan como `decimal(10,2)`; evite aritmética de punto flotante en el cliente para totales fiscales.
- El redondeo de pagos se valida con tolerancia ±0.01.

15.12 Content-Type y archivos

- JSON para la mayoría de endpoints.
- `multipart/form-data` solo en `POST /eventos/imagen`.
- Las imágenes web se suben a Cloudinary y se envía la URL.
- Tamaño máximo de imagen de evento: 5 MB.

15.13 Encabezados recomendados

```
Accept: application/json
Content-Type: application/json
Authorization: Bearer {token}
X-Requested-With: XMLHttpRequest
```

`X-Requested-With` no es obligatorio, pero ayuda a Laravel a responder JSON en algunos errores.

15.14 Versionado de OpenAPI

`docs/openapi.json` se genera con Scramble. Si cambia una ruta, request o response:

1. Actualice el código.
2. Regenerar `openapi.json`.
3. Actualice la matriz del capítulo 26.
4. Añada una prueba.
5. Registre el cambio en `CHANGELOG.md`.

16. API de autenticación y sesión

Este capítulo documenta los endpoints de identidad. Todos devuelven JSON y ninguno usa cookies.

16.1 Endpoints

Tabla 104. *Endpoints de autenticación y sesión*

Método	Endpoint	Auth	Throttle	Descripción
POST	`/login`	No	5/min	Login y emisión de token
POST	`/registro`	No	10/min	Alta diferida de cliente
POST	`/forgot-password`	No	5/min	Envía OTP de reseteo
POST	`/reset-password`	No	5/min	Cambia contraseña con OTP
POST	`/email/verify-code`	No	10/min	Verifica alta con OTP
POST	`/email/verification-notification`	No	5/min	Reenvía OTP de alta
GET	`/me`	Sí	60/min	Perfil con roles
GET	`/user`	Sí	60/min	Alias de `/me`
PUT	`/perfil`	Sí	60/min	Actualiza perfil propio
POST	`/logout`	Sí	60/min	Revoca token actual
POST	`/auth/refresh`	Sí	60/min	Rota token
POST	`/auth/logout-all`	Sí	60/min	Revoca todos los tokens
GET	`/health`	Sí	60/min	Valida token

16.2 `POST /login`

Request:

```
{ "correo": "admin@kaffa.com", "password": "admin123" }
```

Response 200:

```
{
  "access_token": "1|xxxxxxxx",
  "token_type": "Bearer",
  "usuario": {
    "id": 1,
    "nombre": "Administrador",
    "correo": "admin@kaffa.com",
    "activo": true,
    "email_verified_at": "2026-10-01T10:00:00-05:00",
    "roles": [{ "id": 1, "nombre": "admin" }]
  },
  "turno_activo": false,
  "turno_info": null
}
```

Para un barista, `turno_activo` y `turno_info` se calculan con `TurnoService`:

```
{
  "turno_activo": true,
```

```
  "turno_info": {
    "id": 4,
    "fecha": "2026-10-20",
    "tipo": "mañana",
    "minutos_restantes": 95
  }
}
```

Errores:

Tabla 105. *Respuestas de POST /login*

HTTP	Cuerpo	Cuándo
401	`{"message":"Credenciales inválidas."}`	Correo o contraseña incorrectos
403	`{"message":"Este usuario está inactivo..."}`	`activo=false`
403	`{"message":"Debes verificar...","code":"EMAIL_NOT_VERIFIED"}`	Sin `email_verified_at`
422	`errors`	Falta correo/password o formato inválido
429	—	Más de 5 intentos/min

ADVERTENCIA

Advertencia. El login compara con `Hash::check`; no distingue “usuario no existe” de “contraseña incorrecta” en el mensaje público. Eso reduce la enumeración de cuentas.

16.3 `POST /registro`

Request:

```
{
  "nombre": "María Pérez",
  "correo": "maria@example.com",
  "password": "ClaveSegura#2026"
}
```

Reglas:

- `nombre`: requerido, máx. 100.
- `correo`: email, máx. 120, único en `users`.
- `password`: mínimo 8, mayúscula, minúscula, número y símbolo.

Response 201:

```
{
  "message": "Revisa tu correo electrónico e ingresa el código para activar tu cuenta.",
  "correo": "maria@example.com"
}
```

No devuelve token y **no crea el usuario**. Guarda un `registro_pendiente` y envía un OTP. Si se repite el registro con el mismo correo, actualiza el pendiente y reenvía.

16.4 `POST /email/verify-code`

Request:

```
{ "correo": "maria@example.com", "codigo": "123456" }
```

Efecto:

- Valida el OTP de tipo `verificacion`.
- Crea el `User` con `email_verified_at=now()`.
- Asigna rol `cliente`.
- Elimina el pendiente y el código.
- Devuelve 200.

Errores 422: código incorrecto, expirado, demasiados intentos o pendiente inexistente.

16.5 `POST /email/verification-notification`

Request:

```
{ "correo": "maria@example.com" }
```

Reenvía el código si pasó el cooldown de 60 s. La respuesta es uniforme para no revelar si el correo existe.

16.6 `POST /forgot-password`

Request:

```
{ "correo": "maria@example.com" }
```

Response 200 uniforme:

```
{ "message": "Si el correo existe, recibirás un código para restablecer tu contraseña." }
```

El mensaje exacto puede variar; lo importante es que **no** revela si la cuenta existe.

16.7 `POST /reset-password`

Request:

```
{
  "correo": "maria@example.com",
  "codigo": "123456",
  "password": "NuevaClave#2026",
  "password_confirmation": "NuevaClave#2026"
}
```

Efecto:

- Valida el OTP de tipo `reset`.
- Actualiza el hash de la contraseña.
- Revoca **todos** los tokens del usuario.
- Devuelve 200.

16.8 `GET /me` y `GET /user`

Response:

```
{
  "id": 3,
  "nombre": "Cliente Demo",
  "correo": "cliente@kaffa.com",
  "activo": true,
  "roles": [{ "id": 3, "nombre": "cliente" }]
}
```

Nunca incluye `password` ni `remember_token`. `GET /user` es un alias funcional.

16.9 `PUT /perfil`

Request (todos los campos son opcionales):

```
{
  "nombre": "María P.",
  "correo": "maria.nueva@example.com",
  "password": "OtraClave#2026",
  "foto": "https://res.cloudinary.com/roffd6et/image/upload/v.../foto.jpg"
}
```

Reglas:

- `correo` único, ignorando el propio usuario.
- `password` con la misma política fuerte.
- `foto` URL válida, máx. 500, `CloudinaryUrl`.
- `foto: null` elimina la foto.

16.10 `POST /logout`

Revoca el token actual. Response:

```
{ "message": "Sesión cerrada exitosamente." }
```

El token deja de servir inmediatamente.

16.11 `POST /auth/refresh`

Rota el token:

1. Revoca el actual.

2. Emite uno nuevo con nuevas abilities y expiración.

Response:

```
{
  "access_token": "2|yyyyyyyy",
  "token_type": "Bearer",
  "message": "Token renovado exitosamente."
}
```

El cliente debe reemplazar el token guardado. Si la respuesta se pierde, el token anterior ya no sirve y el usuario debe volver a loguearse.

16.12 `POST /auth/logout-all`

Revoca todos los tokens del usuario. Útil tras un cambio de contraseña o sospecha de robo.

```
{ "message": "Sesiones cerradas en todos los dispositivos." }
```

16.13 `GET /health`

Response:

```
{ "status": "ok" }
```

No comprueba base de datos ni servicios externos; solo valida el token. Para un health check real, añada un endpoint que ejecute `SELECT 1` y verifique almacenamiento.

16.14 Flujo recomendado para clientes

1. `POST /registro`.
2. Mostrar formulario de código.
3. `POST /email/verify-code`.
4. `POST /login`.
5. Guardar token de forma segura.
6. `GET /me` al arrancar.
7. Renovar con `POST /auth/refresh` antes de expirar.
8. `POST /logout` al salir.

> **RECOMENDACIÓN**
>
> **Recomendación.** No guarde el token en `localStorage` si el cliente maneja datos muy sensibles y hay riesgo de XSS. Para SPA, considere cookies `HttpOnly` con un backend stateful o un patrón BFF. La API actual está diseñada para Bearer puro.

17. Endpoints públicos

Los endpoints públicos permiten que un visitante vea la landing y el menú sin iniciar sesión. No exponen datos sensibles ni permiten escritura.

17.1 Listado

Tabla 106. *Endpoints públicos*

Método	Endpoint	Throttle	Descripción
GET	`/equipo`	60/min	Administradora y baristas por fecha/turno
GET	`/categorias`	60/min	Categorías activas
GET	`/categorias/{id}`	60/min	Detalle de categoría
GET	`/productos`	60/min	Productos del menú
GET	`/productos/{id}`	60/min	Detalle de producto
GET	`/eventos`	60/min	Eventos publicados
GET	`/eventos/{id}`	60/min	Detalle de evento publicado

17.2 `GET /equipo`

Parámetros:

Tabla 107. *Parámetros de GET /equipo*

Parámetro	Valor	Default
`fecha`	`YYYY-MM-DD`	Hoy en `America/Bogota`
`tipo`	`mañana`, `tarde`, `abierto`	`mañana`

Response:

```
{
  "administradora": { "id": 1, "nombre": "Ana", "foto": "https://..." },
  "baristas": [
    { "id": 2, "nombre": "Santiago", "foto": "https://..." }
  ],
  "fecha": "2026-10-20",
  "tipo": "mañana"
}
```

Reglas:

- La administradora es el primer admin activo.
- Los baristas son los asignados al turno que cubre `fecha` y su tipo.
- Solo se exponen `id`, `nombre`, `foto`; nunca correos.
- `tipo` inválido o `fecha` inválida ⇒ 422.
- Si no hay turno, `baristas` puede ser `[]`.

17.3 `GET /categorias`

Response paginada:

```
{
  "data": [
    { "id": 1, "nombre": "Cafés", "created_at": "...", "updated_at": "..." }
  ],
  "links": { ... },
  "meta": { ... }
}
```

Solo lectura. Las escrituras están en el grupo admin del capítulo 18.

17.4 `GET /productos`

Filtros útiles:

```
GET /api/v1/productos?activo=1&categoria_id=1&per_page=20
```

Response:

```
{
  "data": [
    {
      "id": 1,
      "nombre": "Capuchino",
      "descripcion": "Café espresso con leche vaporizada",
      "imagen": "https://res.cloudinary.com/...",
      "precio_venta": "8500.00",
      "categoria_id": 1,
      "activo": true
    }
  ],
  "links": { ... },
  "meta": { ... }
}
```

> **NOTA**
>
> **Nota.** El campo `precio_venta` se serializa como string decimal (`"8500.00"`) por el cast `decimal:2` . El cliente debe parsearlo.

17.5 `GET /eventos`

Solo eventos con `publicado=true`, ordenados por `fecha`, `hora` ascendente.

```
{
  "data": [
    {
      "id": 7,
      "titulo": "Feria del café",
      "descripcion": "Catas y talleres",
      "lugar": "Salón principal",
      "fecha": "2026-11-05",
      "hora": "10:00:00",
```

```
        "publicado": true,
        "imagenes": [
          { "id": 1, "url": "https://...", "orden": 0 }
        ]
      }
    ],
    "links": { ... },
    "meta": { ... }
  }
```

`GET /eventos/{id}` devuelve 404 si el evento no está publicado. La ruta usa `whereNumber('id')` para no chocar con `/eventos/admin`.

17.6 Seguridad de los endpoints públicos

- No incluyen correos, teléfonos, tokens ni métricas internas.
- Están limitados por throttle 60/min por IP.
- No permiten escritura.
- El catálogo público no expone insumos, recetas ni medios de pago.
- Los eventos borradores solo se ven con `GET /eventos/admin` (admin).

17.7 Ejemplos `curl`

```
curl -s http://127.0.0.1:8000/api/v1/equipo | jq
curl -s "http://127.0.0.1:8000/api/v1/equipo?fecha=2026-10-20&tipo=tarde" | jq
curl -s "http://127.0.0.1:8000/api/v1/productos?activo=1" | jq
curl -s http://127.0.0.1:8000/api/v1/eventos | jq
```

> **RECOMENDACIÓN**
>
> **Recomendación.** Para el Home móvil, cachee el catálogo unos minutos en el cliente y respete el throttle. No sondee `/productos` cada segundo.

18. API de catálogo

El catálogo agrupa categorías, productos, insumos, recetas y medios de pago. La lectura es pública para categorías/productos y autenticada para insumos/recetas/medios; la escritura es exclusiva del admin.

18.1 Matriz de acceso

Tabla 108. *Matriz de acceso al catálogo*

Recurso	Lectura	Escritura	Ability
`/categorias`	Pública	Admin	`catalogo:manage`
`/productos`	Pública	Admin	`catalogo:manage`
`/insumos`	Autenticados	Admin	`catalogo:manage`
`/recetas`	Autenticados	Admin	`catalogo:manage`
`/medios-pago`	Autenticados	Admin	`catalogo:manage`

Las escrituras usan `role:admin`, `throttle:30,1` y `ability:catalogo:manage`.

18.2 Categorías

Crear

```
POST /api/v1/categorias
Authorization: Bearer {admin}
Content-Type: application/json

{ "nombre": "Tés" }
```

Reglas:

- `nombre` requerido, máx. 100.
- No hay unicidad estricta en la migración; evite duplicados por convención.

Actualizar / eliminar

```
PUT /api/v1/categorias/{id}
DELETE /api/v1/categorias/{id}
```

Eliminar es soft delete. Un producto puede quedar con `categoria_id` apuntando a una categoría eliminada; revise la integridad antes.

18.3 Productos

Crear

```
{
  "nombre": "Matcha Latte",
  "descripcion": "Té matcha con leche",
  "imagen": "https://res.cloudinary.com/roffd6et/image/upload/...",
  "precio_venta": 9500,
  "categoria_id": 1,
  "activo": true
}
```

Reglas:

- `nombre` requerido, máx. 150.
- `precio_venta` numérico ≥ 0.
- `imagen` URL máx. 500 (Cloudinary).
- `categoria_id` existente o nulo.
- `activo` booleano.

Recetas

Un producto consume insumos a través de `recetas`:

```
POST /api/v1/recetas
{
  "producto_id": 1,
  "insumo_id": 3,
  "cantidad": 0.25
}
```

`cantidad` es el consumo por unidad vendida. Si un producto no tiene receta, no descuenta inventario.

18.4 Insumos

Crear

```
{
  "nombre": "Leche entera",
  "unidad_medida": "lt",
  "stock_actual": 20,
  "stock_minimo": 5,
  "activo": true
}
```

Reglas:

- `nombre` único (validado en `StoreInsumoRequest`).
- `stock_actual` y `stock_minimo` ≥ 0.
- El stock solo se fija al crear; luego lo mueven compras, mermas y ventas.
- No edite `stock_actual` por API para “cuadrar”; use compra/merma.

Lectura

```
GET /api/v1/insumos?stock_actual=0
GET /api/v1/insumos?nombre=leche
```

18.5 Medios de pago

Crear

```
{ "nombre": "Nequi", "es_virtual": true }
```

Reglas:

- `nombre` único, máx. 50.
- `es_virtual=true` ⇒ los pagos que lo usen exigen comprobante.
- Los IDs virtuales se cachean 5 min.

Medios sembrados:

Tabla 109. *Medios de pago y su carácter virtual*

Medio	Virtual
Efectivo	No
Tarjeta de Crédito	No
Tarjeta de Débito	No
Transferencia Bancaria	Sí
Pago Móvil	Sí
QR (Pago Digital)	Sí

18.6 Imágenes de producto

El backend no recibe el archivo. El flujo es:

1. El navegador sube a Cloudinary con el preset unsigned.
2. Cloudinary devuelve `secure_url`.
3. El cliente envía esa URL en `imagen`.
4. El backend valida `CloudinaryUrl`.

Si la URL pertenece a otra cuenta, responde 422.

18.7 Errores frecuentes

Tabla 110. *Errores frecuentes del catálogo*

HTTP	Causa	Solución
401	Sin token en insumos/recetas/medios	Enviar Bearer
403	Barista intenta escribir catálogo	Usar admin
422	`nombre` duplicado en insumo/medio	Cambiar nombre
422	`precio_venta` negativo	Corregir
422	URL de imagen de otra cuenta	Subir a la cuenta correcta

18.8 Ejemplo completo `curl`

```
TOKEN=$(curl -s -X POST http://127.0.0.1:8000/api/v1/login \
  -H 'Content-Type: application/json' \
  -d '{"correo":"admin@kaffa.com","password":"admin123"}' | jq -r .access_token)

curl -s -X POST http://127.0.0.1:8000/api/v1/productos \
  -H "Authorization: Bearer $TOKEN" \
  -H 'Content-Type: application/json' \
  -d '{
    "nombre":"Matcha Latte",
    "descripcion":"Té matcha con leche",
    "precio_venta":9500,
    "categoria_id":1,
    "activo":true
  }' | jq
```

RECOMENDACIÓN

Recomendación. Mantenga un catálogo de imágenes con nombres estables y transformaciones de Cloudinary (`f_auto,q_auto,w_...`). No guarde URLs con parámetros de sesión.

19. API de pedidos, pagos y facturas

Este es el módulo más crítico: mueve dinero, stock, caja y facturación. Cualquier cambio debe ir acompañado de pruebas y revisión de los invariantes del capítulo 9.

19.1 Endpoints

Tabla 111. *Endpoints de pedidos, pagos y facturas*

Método	Endpoint	Roles	Middleware
GET	`/pedidos`	Admin/barista: todos; cliente: propios	`auth`
POST	`/pedidos`	Admin/barista/c	`role`, `turno.activo`, `throttle:10,1`, `ability:pedidos:create,pedidos:manage`
GET	`/pedidos/{id}`	Admin/barista; cliente propio	`auth`
PUT	`/pedidos/{id}`	Admin/barista	`role`, `turno.activo`, `throttle:10,1`, `ability:pedidos:manage`
DELETE	`/pedidos/{id}`	Admin/barista	Ídem
GET	`/pago-pedidos`	Admin/barista	`role`
POST	`/pago-pedidos`	Admin/barista	`role`, `turno.activo`, `throttle`, `ability`
PUT/DELETE	`/pago-pedidos/{id}`	Admin/barista	Ídem
GET	`/factura-ventas`	Admin	`role:admin`
POST/PUT/DELETE	`/factura-ventas`	Admin	`ability:facturas:manage`

19.2 `POST /pedidos`

Payload completo

```
{
  "cliente_id": 3,
  "barista_id": 2,
  "total": 25500,
  "propina": 2000,
  "estado": "pendiente",
  "caja_id": 1,
  "descripcion_movimiento": "Venta de café",
  "detalles": [
    {
      "producto_id": 1,
      "cantidad": 2,
      "precio_unitario": 10000,
      "subtotal": 20000,
      "nota": "sin azúcar"
    }
  ],
  "pagos": [
    {
      "medio_pago_id": 4,
      "monto": 25500,
```

```
      "comprobante_url": "https://res.cloudinary.com/roffd6et/image/upload/.../recibo.jpg"
    }
  ],
  "factura": {
    "subtotal": 23000,
    "impuestos": 2500,
    "total": 25500
  }
}
```

Reglas

- `estado` ∈ `pendiente | pagado | cancelado | entregado`.
- `detalles[].cantidad` entero ≥ 1.
- `nota` máx. 500.
- `pagos[].monto` ≥ 0.01.
- Comprobante obligatorio si el medio es virtual.
- Σ pagos ≈ total (±0.01).
- `pagado` exige ≥ 1 pago.
- Caja existente, abierta y propia del barista.
- Stock validado por receta antes de crear.
- Factura generada por el servidor.
- Cada pago genera movimiento de caja `ingreso`.
- El evento `PedidoCreado` descuenta stock.

Response 201

```
{
  "data": {
    "id": 101,
    "cliente_id": 3,
    "barista_id": null,
    "total": "25500.00",
    "propina": "2000.00",
    "estado": "pendiente",
    "detalles": [
      {
        "id": 201,
        "producto_id": 1,
        "cantidad": "2.00",
        "precio_unitario": "10000.00",
        "subtotal": "20000.00",
        "nota": "sin azúcar",
        "producto": { "id": 1, "nombre": "Capuchino" }
      }
    ],
    "pagos": [
      {
        "id": 301,
        "medio_pago_id": 4,
        "monto": "25500.00",
        "comprobante_url": "https://..."
      }
    ],
    "factura_venta": {
      "id": 401,
      "numero_factura": "F2026000042",
```

```
      "total": "25500.00"
    }
  }
}
```

Los importes se serializan como strings decimales.

19.3 Autoría por rol

Tabla 112. *Autoría y comportamiento por rol*

Rol	Comportamiento
Cliente	`cliente_id = Auth::id()`, `estado = pendiente`, sin barista
Barista	`barista_id = Auth::id()`, `cliente_id` opcional
Admin	Puede asignar ambos libremente

Un cliente que envíe `cliente_id` de otro será ignorado y se usará su propio ID.

19.4 `PUT /pedidos/{id}`

Solo acepta:

```
{ "total": 26000, "propina": 2500, "estado": "pagado" }
```

No se puede cambiar `cliente_id` ni `barista_id` (autoría inmutable). Si un barista actualiza un pedido sin barista, el backend lo autoasigna.

19.5 `DELETE /pedidos/{id}`

Elimina detalles, movimientos de caja, pagos y factura; soft delete del pedido. **No restaura stock**. Devuelve:

```
{ "message": "Registro eliminado" }
```

19.6 Pagos de pedido

Crear

```
{
  "pedido_id": 101,
  "medio_pago_id": 1,
  "monto": 5000,
  "comprobante_url": null
}
```

Reglas:

- `pedido_id` existente.
- `medio_pago_id` existente.

- `monto` ≥ 0.01.
- Comprobante si el medio es virtual.
- Requiere turno activo para barista.

> **ADVERTENCIA**
>
> **Advertencia.** Crear un pago por separado **no** recalcula automáticamente el total del pedido ni valida la suma global. Use `POST /pedidos` para el flujo completo o agregue la lógica de conciliación en el servicio.

19.7 Facturas

Listar

```
GET /api/v1/factura-ventas?per_page=20
```

Solo admin. El cliente ve la factura de sus pedidos a través de `GET /pedidos` (`factura_venta`).

Crear

```
{
  "pedido_id": 101,
  "subtotal": 23000,
  "impuestos": 2500,
  "total": 25500,
  "fecha_emision": "2026-10-20"
}
```

El `numero_factura` **no se envía**; lo genera `FacturaService`. Si intenta enviarlo, se ignora.

19.8 Errores frecuentes

Tabla 113. *Errores frecuentes de pedidos*

HTTP	Código/mensaje	Causa
403	`TURNO_INACTIVO`	Barista sin turno
403	`TOKEN_SIN_ABILITY`	Token sin `pedidos:manage`
422	`CAJA_CERRADA`	Caja no abierta
422	`CAJA_AJENA`	Caja de otro barista
422	`Stock insuficiente...`	Receta sin stock
422	`La suma de pagos no coincide...`	Total inconsistente
422	`El comprobante es obligatorio...`	Medio virtual sin URL
422	`Un pedido pagado debe tener...`	Estado pagado sin pagos

19.9 Ejemplo `curl`

```
TOKEN=$(curl -s -X POST http://127.0.0.1:8000/api/v1/login \
  -H 'Content-Type: application/json' \
  -d '{"correo":"cliente@kaffa.com","password":"cliente123"}' | jq -r .access_token)

curl -s -X POST http://127.0.0.1:8000/api/v1/pedidos \
  -H "Authorization: Bearer $TOKEN" \
  -H 'Content-Type: application/json' \
  -d '{
    "total": 17000,
    "detalles": [
      {"producto_id": 1, "cantidad": 2, "precio_unitario": 8500, "subtotal": 17000}
    ],
    "pagos": [
      {"medio_pago_id": 1, "monto": 17000}
    ]
  }' | jq
```

El cliente no envía `cliente_id`; el backend lo fuerza.

19.10 Recomendaciones de integración

- Muestre el total calculado por el backend, no solo el del carrito.
- Para medios virtuales, suba el comprobante antes de crear el pedido.
- Tras un timeout, consulte `GET /pedidos` antes de reintentar.
- No use `DELETE` para “corregir” una venta; registre el ajuste correspondiente.
- Si implementa pagos en línea, añada idempotencia y conciliación.

20. API de caja y movimientos

La caja es el control de efectivo por sesión. El backend calcula el arqueo y bloquea la eliminación de cajas con movimientos.

20.1 Endpoints

Tabla 114. *Endpoints de caja y movimientos*

Método	Endpoint	Roles	Notas
GET	`/cajas`	Admin/barista	Barista ve todas; la SPA filtra por `abierta_por`
POST	`/cajas`	Admin/barista	`turno.activo`, throttle 10/min, ability
GET	`/cajas/{id}`	Admin/barista	Incluye movimientos
PUT	`/cajas/{id}`	Admin/barista	Cierre/actualización
DELETE	`/cajas/{id}`	Admin/barista	422 si hay movimientos
GET	`/movimiento-cajas`	Admin/barista	Lectura
POST/PUT/DELETE	`/movimiento-cajas`	Admin	Escritura manual
GET	`/turno-activo`	Autenticados	Turno actual

20.2 Apertura

Request:

```
{
  "monto_apertura_fisico": 100000,
  "monto_apertura_efectivo": 80000,
  "monto_apertura_digital": 20000
}
```

Response 201:

```
{
  "data": {
    "id": 12,
    "abierta_por": 2,
    "turno_id": 4,
    "estado": "abierta",
    "monto_apertura_sistema": "100000.00",
    "monto_apertura_fisico": "100000.00",
    "monto_apertura_efectivo": "80000.00",
    "monto_apertura_digital": "20000.00",
    "fecha_apertura": "2026-10-20T08:00:00-05:00"
  }
}
```

Campos que el cliente **no** envía: `abierta_por`, `turno_id`, `estado`, `fecha_apertura`, `monto_apertura_sistema`.

20.3 Cierre

Request:

```
{
  "estado": "cerrada",
  "monto_cierre_fisico": 150000,
  "monto_cierre_efectivo": 120000,
  "monto_cierre_digital": 30000
}
```

El servidor calcula:

```
monto_cierre_sistema = apertura + ingresos - egresos
```

Response:

```
{
  "data": {
    "id": 12,
    "estado": "cerrada",
    "cerrada_por": 2,
    "monto_cierre_sistema": "150000.00",
    "monto_cierre_fisico": "150000.00",
    "fecha_cierre": "2026-10-20T18:05:00-05:00"
  }
}
```

Si `monto_cierre_fisico` no coincide, la diferencia es faltante/sobrante; el backend no la corrige.

20.4 Movimientos

Tipos:

- `ingreso`: pagos de pedidos.
- `egreso`: pagos de compras/gastos.

Campos:

Tabla 115. *Campos de un movimiento de caja*

Campo	Uso
`caja_id`	Caja asociada
`tipo`	`ingreso` / `egreso`
`monto`	Importe
`pedido_pago_id`	Pago de pedido (si aplica)
`compra_pago_id`	Pago de compra
`gasto_pago_id`	Pago de gasto
`descripcion`	Texto libre

Los movimientos operativos los crean los servicios. El barista **no** debe crearlos manualmente; solo el admin puede escribir en `/movimiento-cajas`.

20.5 Reglas

- Una caja se abre con monto físico; el sistema toma ese valor como apertura.
- Solo se puede operar una caja abierta.
- Un barista solo opera su propia caja.
- No se elimina una caja con movimientos.
- El cierre exige turno activo para barista.
- Admin puede abrir/cerrar sin turno; `turno_id` queda `null`.

20.6 Arqueo

```
sistema = apertura + ingresos - egresos
diferencia = fisico - sistema
```

Tabla 116. *Interpretación de la diferencia de arqueo*

Diferencia	Interpretación
0	Perfecto
< 0	Faltante
> 0	Sobrante

La SPA muestra el resultado. El backend guarda los montos, no una bandera de “cuadre”.

20.7 Errores

Tabla 117. *Errores frecuentes de caja*

HTTP	Causa
403	Barista sin turno
422	Caja con movimientos al eliminar
422	Datos de cierre inválidos
422	Barista intenta usar caja ajena (en pedido/compra/gasto)
404	Caja inexistente

20.8 Ejemplo `curl`

```
TOKEN=$(curl -s -X POST http://127.0.0.1:8000/api/v1/login \
  -H 'Content-Type: application/json' \
  -d '{"correo":"barista@kaffa.com","password":"barista123"}' | jq -r .access_token)

curl -s -X POST http://127.0.0.1:8000/api/v1/cajas \
  -H "Authorization: Bearer $TOKEN" \
  -H 'Content-Type: application/json' \
```

```
  -d '{"monto_apertura_fisico":100000}' | jq

curl -s -X PUT http://127.0.0.1:8000/api/v1/cajas/12 \
  -H "Authorization: Bearer $TOKEN" \
  -H 'Content-Type: application/json' \
  -d '{"estado":"cerrada","monto_cierre_fisico":150000}' | jq
```

ADVERTENCIA

Advertencia. Cerrar una caja con un `monto_cierre_fisico` incorrecto no falla: se registra el faltante/sobrante. La validación es humana; el sistema solo calcula.

21. API de compras, proveedores, gastos y mermas

Estos recursos operan inventario y egresos. Todos exigen `role:admin,barista` y `turno.activo`; las escrituras tienen throttle 10/min y ability específica.

21.1 Endpoints

Tabla 118. *Endpoints de compras, proveedores, gastos y mermas*

Recurso	Métodos	Ability	Efecto
`/compras`	CRUD	`compras:manage`	Aumenta stock y egresa caja
`/detalle-compras`	CRUD	`compras:manage`	Líneas de compra
`/compra-pagos`	CRUD	`compras:manage`	Pagos de compra
`/proveedores`	CRUD	`proveedores:manage` (solo admin para escritura)	Catálogo de proveedores
`/gastos`	CRUD	`gastos:manage`	Egreso operativo
`/gasto-pagos`	CRUD	`gastos:manage`	Pagos de gasto
`/mermas`	CRUD	`mermas:manage`	Reduce stock

21.2 Compras

Crear

```
{
  "proveedor_id": 1,
  "numero_factura_proveedor": "FV-2026-001",
  "fecha_compra": "2026-10-20",
  "total_compra": 250000,
  "detalles": [
    { "insumo_id": 3, "cantidad": 10, "precio_costo": 12000 },
    { "insumo_id": 4, "cantidad": 5, "precio_costo": 26000 }
  ],
  "pagos": [
    { "medio_pago_id": 4, "monto": 250000, "comprobante_url": "https://..." }
  ]
}
```

Reglas:

- `detalles` obligatorio, al menos 1.
- `cantidad` ≥ 0.01, `precio_costo` ≥ 0.
- `total_compra` ≥ 0.
- Σ pagos = total (±0.01).
- Comprobante si medio virtual.
- Incrementa stock por cada detalle.

- Cada pago genera egreso de caja si se usa caja.
- Si no se envía `caja_id`, no hay movimiento de caja.

Eliminar

Revierte stock. Si el stock quedaría negativo, responde 422 y no elimina.

21.3 Proveedores

```
{
  "nit": "900123456-7",
  "nombre": "Distribuidora Andina",
  "telefono": "+57 300 000 0000",
  "correo": "ventas@andina.example",
  "direccion": "Popayán",
  "activo": true
}
```

- `nit` único.
- Lectura admin/barista; escritura solo admin (`proveedores:manage`).
- Soft delete.

21.4 Gastos

```
{
  "descripcion": "Servicios públicos",
  "categoria": "Servicios",
  "pagos": [
    { "medio_pago_id": 1, "monto": 180000 }
  ]
}
```

Reglas:

- `usuario_id` forzado al autenticado.
- El monto vive en `gasto__pagos`.
- Cada pago genera egreso de caja si se usa caja.
- Al eliminar, se borran pagos y movimientos.

21.5 Mermas

```
{
  "insumo_id": 3,
  "cantidad": 1.5,
  "motivo": "Derrame en preparación"
}
```

Reglas:

- `usuario_id` forzado.
- `cantidad` no puede superar el stock.

- Reduce `stock_actual`.
- Al eliminar, restaura stock.
- No genera movimiento de caja.

21.6 Stock: reglas de oro

Tabla 119. *Reglas de stock por acción*

Acción	Stock
Compra	+ cantidad
Venta	− consumo por receta
Merma	− cantidad
Eliminar compra	− cantidad (revierte)
Eliminar merma	+ cantidad (restaura)
Editar compra	**No** ajusta stock automáticamente

> ADVERTENCIA
>
> **Advertencia.** `PUT /compras/{id}` actualiza la cabecera, pero no recalcula stock por cambios en detalles. Si necesita corregir una compra, elimínela y créela de nuevo, o implemente un ajuste explícito.

21.7 Caja en compras/gastos

Si se envía `caja_id`:

- Debe existir y estar abierta.
- Un barista solo puede usar su caja.
- Se genera un movimiento `egreso` por cada pago.

Si no se envía, el egreso no afecta ninguna caja y el arqueo no lo verá.

21.8 Errores

Tabla 120. *Errores frecuentes de compras y mermas*

HTTP	Causa
403	Barista sin turno
403	Token sin ability
422	Σ pagos ≠ total
422	Comprobante faltante
422	Stock insuficiente al revertir
422	Caja ajena/cerrada
422	Merma mayor al stock

21.9 Ejemplo `curl`

```
TOKEN=$(curl -s -X POST http://127.0.0.1:8000/api/v1/login \
  -H 'Content-Type: application/json' \
  -d '{"correo":"barista@kaffa.com","password":"barista123"}' | jq -r .access_token)

curl -s -X POST http://127.0.0.1:8000/api/v1/compras \
  -H "Authorization: Bearer $TOKEN" \
  -H 'Content-Type: application/json' \
  -d '{
    "numero_factura_proveedor":"FV-2026-001",
    "total_compra":120000,
    "detalles":[{"insumo_id":3,"cantidad":10,"precio_costo":12000}],
    "pagos":[{"medio_pago_id":1,"monto":120000}]
  }' | jq
```

21.10 Reportería

Para calcular gastos totales, agregue `gasto__pagos`, no `gastos` (no tiene monto). Para compras, use `compras.total_compra` o sume `detalle__compras`. Documente qué fuente usa cada reporte para evitar cifras inconsistentes.

22. API de turnos

Los turnos definen quién trabaja y cuándo. El backend usa `America/Bogota` y aplica el middleware `turno.activo` a las operaciones de barista.

22.1 Endpoints

Tabla 121. *Endpoints de turnos*

Método	Endpoint	Roles	Descripción
GET	`/turnos`	Admin	Lista con baristas
POST	`/turnos`	Admin	Crea turno y asigna baristas
GET	`/turnos/{id}`	Admin	Detalle
PUT	`/turnos/{id}`	Admin	Actualiza y reemplaza baristas
DELETE	`/turnos/{id}`	Admin	422 si tiene cajas
GET	`/mis-turnos`	Barista	Turnos propios
GET	`/turno-activo`	Autenticados	Turno actual

22.2 Crear

```
{
  "fecha": "2026-10-20",
  "fecha_fin": "2026-10-27",
  "tipo": "abierto",
  "barista_ids": [2, 5]
}
```

Reglas:

- `fecha` `Y-m-d`.
- `fecha_fin` opcional, ≥ `fecha`; si se omite, dura un día.
- `tipo` ∈ `mañana | tarde | abierto`.
- `barista_ids` ≥ 1.
- Único por `fecha + tipo` (restricción de BD).
- `abierto` tiene prioridad sobre mañana/tarde.

22.3 Horarios

Tabla 122. *Horarios por tipo de turno*

Tipo	Horario
`mañana`	07:00–13:00
`tarde`	13:00–18:00
`abierto`	05:00–22:00

Fuera de 05:00–22:00 no hay turno esperado. Un turno `abierto` cubre toda la jornada extendida.

22.4 Turno activo

`GET /turno-activo` devuelve:

```
{
  "turno_activo": true,
  "turno_info": {
    "id": 4,
    "fecha": "2026-10-20",
    "fecha_fin": "2026-10-20",
    "tipo": "mañana",
    "minutos_restantes": 95,
    "hora_inicio": "07:00",
    "hora_fin": "13:00"
  }
}
```

Para admin/cliente:

```
{ "turno_activo": false, "turno_info": null }
```

22.5 `mis-turnos`

Barista autenticado. Paginación 15, orden `fecha,tipo asc`.

```
{
  "data": [
    { "id": 4, "fecha": "2026-10-20", "tipo": "mañana", "baristas": [...] }
  ],
  "links": { ... },
  "meta": { ... }
}
```

22.6 Reglas de negocio

- Un turno no se puede eliminar si tiene cajas asociadas (422).
- El admin puede crear/editar turnos sin restricción de turno.
- El barista solo consulta; no crea turnos.
- Un turno con rango cubre cada día entre `fecha` y `fecha_fin`.
- El tipo `abierto` se busca primero; si existe, gana.

22.7 Errores

Tabla 123. *Errores frecuentes de turnos*

HTTP	Código	Causa
403	`FUERA_HORARIO`	Barista fuera de horario
403	`TURNO_INACTIVO`	Sin turno asignado
403	`TURNO_EXPIRADO`	Turno terminado
422	—	`fecha_fin` < `fecha`, tipo inválido, sin baristas
422	—	Eliminar turno con cajas

22.8 Ejemplo `curl`

```
TOKEN=$(curl -s -X POST http://127.0.0.1:8000/api/v1/login \
  -H 'Content-Type: application/json' \
  -d '{"correo":"admin@kaffa.com","password":"admin123"}' | jq -r .access_token)

curl -s -X POST http://127.0.0.1:8000/api/v1/turnos \
  -H "Authorization: Bearer $TOKEN" \
  -H 'Content-Type: application/json' \
  -d '{"fecha":"2026-10-21","tipo":"mañana","barista_ids":[2]}' | jq
```

22.9 Operación diaria

1. El admin crea turnos del día o de la semana.
2. El barista inicia sesión; `login` devuelve `turno_activo`.
3. Si no tiene turno, no puede cobrar ni abrir caja.
4. Al terminar, cierra caja.
5. El admin puede reasignar baristas si hay ausencias.

> **RECOMENDACIÓN**
>
> **Recomendación.** Cree turnos con anticipación y valide la zona horaria del servidor. Un desfase de una hora puede dejar al barista fuera de turno o permitirle operar cuando no debe.

23. API de chat y reportes

El chat conecta clientes con baristas/admin. Los reportes permiten al barista informar novedades al admin.

23.1 Chat: endpoints

Tabla 124. *Endpoints de chat*

Método	Endpoint	Descripción
GET	`/mensajes`	Bandeja de contactos o hilo con `?con={id}`
GET	`/mensajes/contactos`	Sugerencias según rol
POST	`/mensajes`	Enviar mensaje
POST	`/mensajes/leer`	Marcar leídos de un contacto

23.2 Bandeja

Sin `?con`:

```
{
  "contactos": [
    {
      "id": 2,
      "nombre": "Barista Demo",
      "ultimo_mensaje": "Tu pedido está listo",
      "ultima_hora": "2026-10-20T15:30:00-05:00",
      "no_leidos": 2
    }
  ]
}
```

Con `?con=2`:

```
{
  "contacto": { "id": 2, "nombre": "Barista Demo" },
  "data": [
    { "id": 10, "remitente_id": 2, "destinatario_id": 3, "mensaje": "Hola", "leido": true, "created_at":
"..." }
  ],
  "links": { ... },
  "meta": { ... }
}
```

El hilo se pagina de 50 en 50, descendente.

23.3 Enviar

```
{
  "destinatario_id": 2,
```

```
  "mensaje": "¿Mi pedido está listo?"
}
```

Reglas:

- `mensaje` requerido, máx. 1000.
- No auto-mensajes.
- No cliente→cliente.
- Destinatario debe existir y estar activo.
- Barista/admin pueden escribir a clientes; cliente a barista/admin.

Errores 422: destinatario inexistente/inactivo, auto-mensaje, cliente→cliente.

23.4 Marcar leídos

```
{ "con": 2 }
```

Marca como leídos todos los mensajes no leídos de ese contacto para el usuario autenticado.

23.5 Contactos sugeridos

`GET /mensajes/contactos` devuelve usuarios activos según rol:

- Cliente → baristas y admins.
- Barista/admin → clientes.

No expone correos ni datos sensibles.

23.6 Reportes

Tabla 125. *Endpoints de reportes*

Método	Endpoint	Rol
POST	`/reportes`	Admin/barista
GET	`/reportes`	Admin
GET	`/reportes/{id}`	Admin
PUT/PATCH	`/reportes/{id}`	Admin
DELETE	`/reportes/{id}`	Admin

Crear:

```
{
  "mensaje": "Falta leche entera",
  "prioridad": "alta"
}
```

Reglas:

- `barista_id` forzado al autenticado.

- `prioridad` ∈ `baja | media | alta`.
- `leido` default `false`.
- Admin puede editar mensaje, prioridad y leído.

23.7 Sondeo

La SPA usa:

- Chat/no leídos: 5 s.
- Reportes: según vista.

Respete el throttle global. No sondee el hilo completo si solo necesita no leídos; use `GET /mensajes` (bandeja).

23.8 Errores

Tabla 126. *Errores frecuentes de chat y reportes*

HTTP	Causa
401	Sin token
403	Cliente intenta gestionar reportes
422	Mensaje vacío o >1000
422	Auto-mensaje o cliente→cliente
422	Destinatario inactivo

23.9 Ejemplo `curl`

```
TOKEN=$(curl -s -X POST http://127.0.0.1:8000/api/v1/login \
  -H 'Content-Type: application/json' \
  -d '{"correo":"cliente@kaffa.com","password":"cliente123"}' | jq -r .access_token)

curl -s -X POST http://127.0.0.1:8000/api/v1/mensajes \
  -H "Authorization: Bearer $TOKEN" \
  -H 'Content-Type: application/json' \
  -d '{"destinatario_id":2,"mensaje":"¿Mi pedido está listo?"}' | jq

curl -s "http://127.0.0.1:8000/api/v1/mensajes?con=2" \
  -H "Authorization: Bearer $TOKEN" | jq
```

RECOMENDACIÓN

Recomendación. Para soporte, muestre primero la bandeja y solo cargue el hilo al abrir la conversación. Evita consultas innecesarias y mejora la privacidad.

24. API de usuarios y roles

La gestión de usuarios es exclusiva del admin. Un usuario puede tener varios roles.

24.1 Endpoints

Tabla 127. *Endpoints de usuarios y roles*

Método	Endpoint	Descripción
GET	`/usuarios`	Lista; filtro `?rol=`
POST	`/usuarios`	Crea usuario verificado
GET	`/usuarios/{id}`	Detalle
PUT	`/usuarios/{id}`	Actualiza
DELETE	`/usuarios/{id}`	Soft delete
GET/POST/PUT/DELETE	`/roles`	CRUD de roles
GET/POST/PUT/DELETE	`/usuario-roles`	Pivote

Todas exigen `role:admin` + `ability:usuarios:manage` / `roles:manage`.

24.2 Crear usuario

```
{
  "nombre": "Nuevo Barista",
  "correo": "barista2@kaffa.com",
  "password": "ClaveSegura#2026",
  "foto": "https://res.cloudinary.com/...",
  "activo": true,
  "rol_ids": [2]
}
```

Reglas:

- `nombre` máx. 100.
- `correo` único, máx. 120.
- `password` fuerte.
- `foto` URL Cloudinary opcional.
- `rol_ids` ≥ 1.
- Se acepta `roles[]` como alias de `rol_ids[]` (app móvil).
- El usuario nace **verificado** (`email_verified_at=now`).

24.3 Actualizar

```
{
  "nombre": "Barista Actualizado",
  "correo": "barista2@kaffa.com",
  "password": "OtraClave#2026",
  "activo": true,
```

```
  "rol_ids": [2, 3]
}
```

`password` es opcional; si se envía, se rehashea. `rol_ids` sincroniza la pivote.

24.4 Filtrar por rol

```
GET /api/v1/usuarios?rol=barista
```

El filtro se aplica sobre la relación `roles`. Útil para selectores de turnos.

24.5 Roles

```
{ "nombre": "supervisor" }
```

Los roles sembrados son `admin`, `barista`, `cliente`. Crear roles nuevos no otorga abilities automáticamente: `AuthController::abilitiesFor` solo reconoce los tres roles. Un rol personalizado sin abilities no podrá escribir.

> **ADVERTENCIA**
>
> **Advertencia.** Si añade un rol, actualice `abilitiesFor`, la matriz RBAC, las policies y las pruebas. De lo contrario, el rol existirá pero no servirá para operar.

24.6 Pivote usuario-rol

```
{ "usuario_id": 5, "rol_id": 2 }
```

Permite asignar roles manualmente. La forma recomendada es `rol_ids` en `POST/PUT /usuarios`, que sincroniza en una transacción.

24.7 Eliminar

Soft delete. El usuario deja de aparecer en listados y no puede autenticarse. Los pedidos y movimientos históricos conservan su `usuario_id`.

24.8 Seguridad

- Solo admin.
- No se puede eliminar el propio usuario si se queda sin admin (no hay validación explícita; opere con cuidado).
- Cambiar el correo de un usuario no reenvía verificación; el admin asume la responsabilidad.
- No exponga `password` ni `remember_token`.

24.9 Errores

Tabla 128. *Errores frecuentes de usuarios*

HTTP	Causa
403	No admin
422	Correo duplicado
422	Contraseña débil
422	`rol_ids` vacío o inexistente
404	Usuario/rol inexistente

24.10 Ejemplo `curl`

```
TOKEN=$(curl -s -X POST http://127.0.0.1:8000/api/v1/login \
  -H 'Content-Type: application/json' \
  -d '{"correo":"admin@kaffa.com","password":"admin123"}' | jq -r .access_token)

curl -s -X POST http://127.0.0.1:8000/api/v1/usuarios \
  -H "Authorization: Bearer $TOKEN" \
  -H 'Content-Type: application/json' \
  -d '{
    "nombre":"Nuevo Barista",
    "correo":"barista2@kaffa.com",
    "password":"ClaveSegura#2026",
    "rol_ids":[2]
  }' | jq
```

RECOMENDACIÓN

Recomendación. Use roles múltiples en lugar de crear cuentas duplicadas. Un barista que también compra como cliente puede tener ambos roles y alternar según el contexto.

25. Asistente KAFFA

El Asistente KAFFA es un chatbot **local** disponible para todos los usuarios autenticados. No usa un proveedor externo: interpreta la pregunta por palabras clave y responde con consultas reales a la base, adaptando el contenido al rol.

25.1 Endpoints

Tabla 129. *Endpoints del Asistente KAFFA*

Método	Endpoint	Roles	Throttle
GET	`/asistente/mensajes`	Todos los autenticados	30/min
POST	`/asistente/chat`	Todos los autenticados	30/min
DELETE	`/asistente/mensajes`	Todos los autenticados	30/min

No exige ability específica. La separación de datos se hace por rol dentro del servicio.

25.2 Enviar mensaje

```
{ "mensaje": "¿Cómo van las ventas de esta semana?" }
```

Response 200:

```
{
  "respuesta": "**Ventas de los últimos 7 días**...",
  "mensaje_id": 42,
  "created_at": "2026-10-20T15:47:46-05:00"
}
```

Reglas:

- `mensaje` requerido, máx. 2000.
- Se persiste el turno `user` y el turno `assistant`.
- El historial se aísla por `usuario_id`.
- Sin token ⇒ 401.
- Throttle 30/min ⇒ 429.

25.3 Historial

```
GET /api/v1/asistente/mensajes?per_page=15
```

Response:

```
{
  "data": [
```

```
      { "id": 42, "rol": "assistant", "contenido": "...", "created_at": "..." },
      { "id": 41, "rol": "user", "contenido": "...", "created_at": "..." }
    ],
    "links": { ... },
    "meta": { ... }
  }
```

Orden: más reciente primero. `per_page` 1–100, por defecto 15.

25.4 Reiniciar

```
DELETE /api/v1/asistente/mensajes
```

Borra el hilo del usuario autenticado. Response:

```
{ "message": "Conversación eliminada" }
```

No afecta a otros usuarios.

25.5 Capacidades por rol

Admin

- Ventas hoy/7 días/mes.
- Productos más vendidos.
- Inventario por reponer.
- Pedidos por estado.
- Gastos y margen.
- Predicción de ventas por promedio móvil 7 días.
- Recomendaciones de promoción.

Barista

- Ventas.
- Productos más vendidos.
- Inventario.
- Pedidos del día.

No recibe gastos, predicciones ni marketing.

Cliente

- Menú y precios.
- Medios de pago.
- Cómo hacer un pedido.
- Eventos próximos.
- Sede/ubicación/horario.

Nunca recibe ventas, gastos, inventario ni predicciones.

25.6 Cómo detecta la intención

`AsistenteService` normaliza la pregunta (minúsculas, sin tildes) y busca palabras clave:

Tabla 130. *Intenciones y ejemplos de pregunta*

Intención	Ejemplos
Saludo	“hola”, “buenas tardes”
Ayuda	“qué puedes hacer”, “funciones”
Ventas	“ventas”, “ingresos”, “cómo va”
Productos	“más vendido”, “top”, “ranking”
Inventario	“stock”, “reponer”, “insumos”
Pedidos	“pedidos pendientes”, “cola”
Gastos	“gastos”, “margen”, “utilidad” (admin)
Predicción	“predice”, “próximos 7 días” (admin)
Promoción	“marketing”, “descuento”, “oferta” (admin)
Menú (cliente)	“productos”, “precio”, “carta”
Pagos (cliente)	“medios de pago”, “tarjeta”, “nequi”
Pedido (cliente)	“cómo comprar”, “carrito”
Eventos (cliente)	“eventos”, “taller”
Sede (cliente)	“dónde queda”, “horario”

Si no entiende, devuelve una guía de opciones.

25.7 Privacidad

- El servicio es local; no envía datos a Gemini/OpenAI.
- Las claves `GEMINI_*` están reservadas y no se usan.
- El cliente no recibe métricas internas.
- El historial se limita al usuario autenticado.
- Si se conecta un LLM, envíe solo datos agregados y anonimizados.

25.8 Limitaciones

- No es un modelo generativo: respuestas basadas en reglas.
- No mantiene contexto profundo entre mensajes; cada pregunta se interpreta por separado.
- La predicción es un promedio móvil simple, no un modelo estadístico.
- No ejecuta acciones (no crea pedidos ni cambia precios).
- No responde preguntas fuera del dominio de la cafetería.

25.9 Ejemplos `curl`

```
TOKEN=$(curl -s -X POST http://127.0.0.1:8000/api/v1/login \
  -H 'Content-Type: application/json' \
  -d '{"correo":"admin@kaffa.com","password":"admin123"}' | jq -r .access_token)

curl -s -X POST http://127.0.0.1:8000/api/v1/asistente/chat \
  -H "Authorization: Bearer $TOKEN" \
  -H 'Content-Type: application/json' \
  -d '{"mensaje":"¿Cuáles son los productos más vendidos?"}' | jq

curl -s http://127.0.0.1:8000/api/v1/asistente/mensajes \
  -H "Authorization: Bearer $TOKEN" | jq

curl -s -X DELETE http://127.0.0.1:8000/api/v1/asistente/mensajes \
  -H "Authorization: Bearer $TOKEN" | jq
```

25.10 Errores

Tabla 131. *Errores frecuentes del asistente*

HTTP	Causa
401	Sin token
422	Mensaje vacío o >2000
429	Más de 30 mensajes/min

RECOMENDACIÓN

Recomendación. Use el Asistente como ayuda operativa, no como fuente contable oficial. Para cifras financieras, consulte los reportes y la base; el Asistente agrega datos con fines informativos.

26. Matriz completa de endpoints

Esta matriz se genera automáticamente a partir de `php artisan route:list` y se contrasta con `openapi.json`. Una misma ruta puede aparecer una vez por operación; `PUT/PATCH` se muestra junto porque Laravel acepta ambos métodos. La columna **Throttle** indica el límite específico de la ruta; todas las rutas heredan además el límite global de 60 peticiones por minuto por usuario o IP.

> **NOTA**
>
> **Nota.** `route:list` reporta 142 entradas porque separa lecturas y escrituras de los `apiResource` y añade el prefijo `HEAD` a los `GET`. Al normalizar métodos y rutas quedan las operaciones funcionales de esta matriz; OpenAPI documenta 69 paths porque agrupa operaciones por recurso.

26.1 Resumen por módulo

Tabla 132. *Resumen de operaciones por módulo*

Módulo	Operaciones	Rutas con auth	Rutas con rol	Rutas con ability
Autenticación y sesión	13	7	0	0
Sitio público y eventos	8	5	5	5
Catálogo	25	21	15	15
Pedidos, pagos y facturas	15	15	13	11
Caja y movimientos	11	11	10	8
Compras y proveedores	20	20	20	18
Gastos y mermas	15	15	15	15
Turnos	6	6	6	5
Chat y reportes	9	9	5	0
Usuarios y roles	15	15	15	15
Asistente KAFFA	3	3	0	0
Total	**140**	**127**	**104**	**92**

26.2 Detalle por módulo

Autenticación y sesión

Tabla 133. *Matriz de endpoints: autenticación y sesión*

Método	Endpoint	Auth	Roles	Ability	Turno	Throttle	Operación
`POST`	`/api/v1/auth/logout-all`	Sí	Todos los autenticados	–	—	60/min global	Revocar TODOS los tokens del usuario autenticado (cierre de sesión en todos los dispositivos).
`POST`	`/api/v1/auth/refresh`	Sí	Todos los autenticados	–	—	60/min global	Rotar el token actual: revoca el actual y emite uno nuevo con un nuevo expires_at. Útil para extender sesión activa sin re-login.

Método	Endpoint	Auth	Roles	Ability	Turno	Throttle	Operación
POST	/api/v1/email/verification-notification	No	Público	–	—	5/min	Reenvía el código de verificación de un alta pendiente.
POST	/api/v1/email/verify-code	No	Público	–	—	10/min	Verifica la cuenta con el código recibido por correo. Al confirmar, materializa el usuario verificado.
POST	/api/v1/forgot-password	No	Público	–	—	5/min	Envía un código de restablecimiento al correo indicado.
GET	/api/v1/health	Sí	Todos los autenticados	–	—	60/min global	Comprueba que el token es válido.
POST	/api/v1/login	No	Público	–	—	5/min	Autenticar un usuario y retornar el token de acceso.
POST	/api/v1/logout	Sí	Todos los autenticados	–	—	60/min global	Cerrar sesión (revocar el token actual).
GET	/api/v1/me	Sí	Todos los autenticados	–	—	60/min global	Retornar el perfil del usuario autenticado.
PUT	/api/v1/perfil	Sí	Todos los autenticados	–	—	60/min global	Editar el perfil del usuario autenticado (nombre/correo/contraseña).
POST	/api/v1/registro	No	Público	–	—	10/min	Registro público de clientes (rol 'cliente').
POST	/api/v1/reset-password	No	Público	–	—	5/min	Restablece la contraseña usando el código recibido por correo.
GET	/api/v1/user	Sí	Todos los autenticados	–	—	60/min global	Alias de /me: perfil del usuario autenticado.

Sitio público y eventos

Tabla 134. *Matriz de endpoints: sitio público y eventos*

Método	Endpoint	Auth	Roles	Ability	Turno	Throttle	Operación
GET	/api/v1/equipo	No	Público	–	—	60/min	Endpoint público para la landing ("Sobre Nosotros → Nuestro Equipo").
GET	/api/v1/eventos	No	Público	–	—	60/min	Listado público de eventos publicados (landing /eventos). Ordena por fecha ascendente: primero los eventos más próximos.
POST	/api/v1/eventos	Sí	admin	eventos:manage	—	30/min	Crea un evento.
GET	/api/v1/eventos/admin	Sí	admin	eventos:manage	—	30/min	Listado de gestión (admin): incluye borradores y publicados.
POST	/api/v1/eventos/imagen	Sí	admin	eventos:manage	—	30/min	Subida de una imagen de la galería desde el cliente móvil (cámara/galería).
DELETE	/api/v1/eventos/{id}	Sí	admin	eventos:manage	—	30/min	Elimina un evento.
GET	/api/v1/eventos/{id}	No	Público	–	—	60/min	Detalle público de un evento publicado.
PUT/PATCH	/api/v1/eventos/{id}	Sí	admin	eventos:manage	—	30/min	Actualiza un evento.

Catálogo

Tabla 135. *Matriz de endpoints: catálogo*

Método	Endpoint	Auth	Roles	Ability	Turno	Throttle	Operación
GET	/api/v1/categorias	No	Público	–	—	60/min	Lista categorías.
POST	/api/v1/categorias	Sí	admin	catalogo:manage	—	30/min	Crea un categoría.
DELETE	/api/v1/categorias/{id}	Sí	admin	catalogo:manage	—	30/min	Elimina un categoría.
GET	/api/v1/categorias/{id}	No	Público	–	—	60/min	Consulta un categoría por ID.
PUT/PATCH	/api/v1/categorias/{id}	Sí	admin	catalogo:manage	—	30/min	Actualiza un categoría.
GET	/api/v1/insumos	Sí	Todos los autenticados	–	—	60/min global	Lista insumos.
POST	/api/v1/insumos	Sí	admin	catalogo:manage	—	30/min	Crea un insumo.
DELETE	/api/v1/insumos/{id}	Sí	admin	catalogo:manage	—	30/min	Elimina un insumo.
GET	/api/v1/insumos/{id}	Sí	Todos los autenticados	–	—	60/min global	Consulta un insumo por ID.
PUT/PATCH	/api/v1/insumos/{id}	Sí	admin	catalogo:manage	—	30/min	Actualiza un insumo.
GET	/api/v1/medios-pago	Sí	Todos los autenticados	–	—	60/min global	Lista medio de pagos.
POST	/api/v1/medios-pago	Sí	admin	catalogo:manage	—	30/min	Crea un medio de pago.
DELETE	/api/v1/medios-pago/{id}	Sí	admin	catalogo:manage	—	30/min	Elimina un medio de pago.
GET	/api/v1/medios-pago/{id}	Sí	Todos los autenticados	–	—	60/min global	Consulta un medio de pago por ID.
PUT/PATCH	/api/v1/medios-pago/{id}	Sí	admin	catalogo:manage	—	30/min	Actualiza un medio de pago.
GET	/api/v1/productos	No	Público	–	—	60/min	Lista productos.
POST	/api/v1/productos	Sí	admin	catalogo:manage	—	30/min	Crea un producto.
DELETE	/api/v1/productos/{id}	Sí	admin	catalogo:manage	—	30/min	Elimina un producto.
GET	/api/v1/productos/{id}	No	Público	–	—	60/min	Consulta un producto por ID.
PUT/PATCH	/api/v1/productos/{id}	Sí	admin	catalogo:manage	—	30/min	Actualiza un producto.
GET	/api/v1/recetas	Sí	Todos los autenticados	–	—	60/min global	Lista recetas.
POST	/api/v1/recetas	Sí	admin	catalogo:manage	—	30/min	Crea un receta.
DELETE	/api/v1/recetas/{id}	Sí	admin	catalogo:manage	—	30/min	Elimina un receta.
GET	/api/v1/recetas/{id}	Sí	Todos los autenticados	–	—	60/min global	Consulta un receta por ID.
PUT/PATCH	/api/v1/recetas/{id}	Sí	admin	catalogo:manage	—	30/min	Actualiza un receta.

Pedidos, pagos y facturas

Tabla 136. *Matriz de endpoints: pedidos, pagos y facturas*

Método	Endpoint	Auth	Roles	Ability	Turno	Throttle	Operación
GET	/api/v1/factura-ventas	Sí	admin	facturas:manage	—	30/min	Lista factura de ventas.
POST	/api/v1/factura-ventas	Sí	admin	facturas:manage	—	30/min	Crea un factura de venta.
DELETE	/api/v1/factura-ventas/{id}	Sí	admin	facturas:manage	—	30/min	Elimina un factura de venta.
GET	/api/v1/factura-ventas/{id}	Sí	admin	facturas:manage	—	30/min	Consulta un factura de venta por ID.
PUT/PATCH	/api/v1/factura-ventas/{id}	Sí	admin	facturas:manage	—	30/min	Actualiza un factura de venta.
GET	/api/v1/pago-pedidos	Sí	admin, barista	–	—	60/min global	Lista pago de pedidos.
POST	/api/v1/pago-pedidos	Sí	admin, barista	pedidos:manage	Sí	10/min	Crea un pago de pedido.
DELETE	/api/v1/pago-pedidos/{id}	Sí	admin, barista	pedidos:manage	Sí	10/min	Elimina un pago de pedido.
GET	/api/v1/pago-pedidos/{id}	Sí	admin, barista	–	—	60/min global	Consulta un pago de pedido por ID.
PUT/PATCH	/api/v1/pago-pedidos/{id}	Sí	admin, barista	pedidos:manage	Sí	10/min	Actualiza un pago de pedido.
GET	/api/v1/pedidos	Sí	Todos los autenticados	–	—	60/min global	Lista pedidos.
POST	/api/v1/pedidos	Sí	admin, barista, cliente	pedidos:create,pedidos:manage	Sí	10/min	Crea un pedido.
DELETE	/api/v1/pedidos/{id}	Sí	admin, barista	pedidos:manage	Sí	10/min	Elimina un pedido.
GET	/api/v1/pedidos/{id}	Sí	Todos los autenticados	–	—	60/min global	Consulta un pedido por ID.
PUT/PATCH	/api/v1/pedidos/{id}	Sí	admin, barista	pedidos:manage	Sí	10/min	Actualiza un pedido.

Caja y movimientos

Tabla 137. *Matriz de endpoints: caja y movimientos*

Método	Endpoint	Auth	Roles	Ability	Turno	Throttle	Operación
GET	/api/v1/cajas	Sí	admin, barista	cajas:manage	Sí	60/min global	Lista cajas.
POST	/api/v1/cajas	Sí	admin, barista	cajas:manage	Sí	10/min	Crea un caja.
DELETE	/api/v1/cajas/{id}	Sí	admin, barista	cajas:manage	Sí	10/min	Elimina un caja.
GET	/api/v1/cajas/{id}	Sí	admin, barista	cajas:manage	Sí	60/min global	Consulta un caja por ID.
PUT/PATCH	/api/v1/cajas/{id}	Sí	admin, barista	cajas:manage	Sí	10/min	Actualiza un caja.
GET	/api/v1/movimiento-cajas	Sí	admin, barista	–	Sí	60/min global	Lista movimiento de cajas.
POST	/api/v1/movimiento-cajas	Sí	admin, barista	movimiento-cajas:manage	Sí	60/min global	Crea un movimiento de caja.
DELETE	/api/v1/movimiento-cajas/{id}	Sí	admin, barista	movimiento-cajas:manage	Sí	60/min global	Elimina un movimiento de caja.
GET	/api/v1/movimiento-cajas/{id}	Sí	admin, barista	–	Sí	60/min global	Consulta un movimiento de caja por ID.

Método	Endpoint	Auth	Roles	Ability	Turno	Throttle	Operación
PUT/PATCH	/api/v1/movimiento-cajas/{id}	Sí	admin, barista	movimiento-cajas:manage	Sí	60/min global	Actualiza un movimiento de caja.
GET	/api/v1/turno-activo	Sí	Todos los autenticados	–	—	60/min global	Lista turno activos.

Compras y proveedores

Tabla 138. *Matriz de endpoints: compras y proveedores*

Método	Endpoint	Auth	Roles	Ability	Turno	Throttle	Operación
GET	/api/v1/compra-pagos	Sí	admin, barista	compras:manage	Sí	60/min global	Lista pago de compras.
POST	/api/v1/compra-pagos	Sí	admin, barista	compras:manage	Sí	10/min	Crea un pago de compra.
DELETE	/api/v1/compra-pagos/{id}	Sí	admin, barista	compras:manage	Sí	10/min	Elimina un pago de compra.
GET	/api/v1/compra-pagos/{id}	Sí	admin, barista	compras:manage	Sí	60/min global	Consulta un pago de compra por ID.
PUT/PATCH	/api/v1/compra-pagos/{id}	Sí	admin, barista	compras:manage	Sí	10/min	Actualiza un pago de compra.
GET	/api/v1/compras	Sí	admin, barista	compras:manage	Sí	60/min global	Lista compras.
POST	/api/v1/compras	Sí	admin, barista	compras:manage	Sí	10/min	Crea un compra.
DELETE	/api/v1/compras/{id}	Sí	admin, barista	compras:manage	Sí	10/min	Elimina un compra.
GET	/api/v1/compras/{id}	Sí	admin, barista	compras:manage	Sí	60/min global	Consulta un compra por ID.
PUT/PATCH	/api/v1/compras/{id}	Sí	admin, barista	compras:manage	Sí	10/min	Actualiza un compra.
GET	/api/v1/detalle-compras	Sí	admin, barista	compras:manage	Sí	60/min global	Lista detalle de compras.
POST	/api/v1/detalle-compras	Sí	admin, barista	compras:manage	Sí	10/min	Crea un detalle de compra.
DELETE	/api/v1/detalle-compras/{id}	Sí	admin, barista	compras:manage	Sí	10/min	Elimina un detalle de compra.
GET	/api/v1/detalle-compras/{id}	Sí	admin, barista	compras:manage	Sí	60/min global	Consulta un detalle de compra por ID.
PUT/PATCH	/api/v1/detalle-compras/{id}	Sí	admin, barista	compras:manage	Sí	10/min	Actualiza un detalle de compra.
GET	/api/v1/proveedores	Sí	admin, barista	–	Sí	60/min global	Lista proveedors.
POST	/api/v1/proveedores	Sí	admin, barista	proveedores:manage	Sí	60/min global	Crea un proveedor.
DELETE	/api/v1/proveedores/{id}	Sí	admin, barista	proveedores:manage	Sí	60/min global	Elimina un proveedor.
GET	/api/v1/proveedores/{id}	Sí	admin, barista	–	Sí	60/min global	Consulta un proveedor por ID.
PUT/PATCH	/api/v1/proveedores/{id}	Sí	admin, barista	proveedores:manage	Sí	60/min global	Actualiza un proveedor.

Gastos y mermas

Tabla 139. *Matriz de endpoints: gastos y mermas*

Método	Endpoint	Auth	Roles	Ability	Turno	Throttle	Operación
GET	/api/v1/gasto-pagos	Sí	admin, barista	gastos:manage	Sí	60/min global	Lista pago de gastos.
POST	/api/v1/gasto-pagos	Sí	admin, barista	gastos:manage	Sí	10/min	Crea un pago de gasto.
DELETE	/api/v1/gasto-pagos/{id}	Sí	admin, barista	gastos:manage	Sí	10/min	Elimina un pago de gasto.
GET	/api/v1/gasto-pagos/{id}	Sí	admin, barista	gastos:manage	Sí	60/min global	Consulta un pago de gasto por ID.
PUT/PATCH	/api/v1/gasto-pagos/{id}	Sí	admin, barista	gastos:manage	Sí	10/min	Actualiza un pago de gasto.
GET	/api/v1/gastos	Sí	admin, barista	gastos:manage	Sí	60/min global	Lista gastos.
POST	/api/v1/gastos	Sí	admin, barista	gastos:manage	Sí	10/min	Crea un gasto.
DELETE	/api/v1/gastos/{id}	Sí	admin, barista	gastos:manage	Sí	10/min	Elimina un gasto.
GET	/api/v1/gastos/{id}	Sí	admin, barista	gastos:manage	Sí	60/min global	Consulta un gasto por ID.
PUT/PATCH	/api/v1/gastos/{id}	Sí	admin, barista	gastos:manage	Sí	10/min	Actualiza un gasto.
GET	/api/v1/mermas	Sí	admin, barista	mermas:manage	Sí	60/min global	Lista mermas.
POST	/api/v1/mermas	Sí	admin, barista	mermas:manage	Sí	10/min	Crea un merma.
DELETE	/api/v1/mermas/{id}	Sí	admin, barista	mermas:manage	Sí	10/min	Elimina un merma.
GET	/api/v1/mermas/{id}	Sí	admin, barista	mermas:manage	Sí	60/min global	Consulta un merma por ID.
PUT/PATCH	/api/v1/mermas/{id}	Sí	admin, barista	mermas:manage	Sí	10/min	Actualiza un merma.

Turnos

Tabla 140. *Matriz de endpoints: turnos*

Método	Endpoint	Auth	Roles	Ability	Turno	Throttle	Operación
GET	/api/v1/mis-turnos	Sí	barista	–	—	60/min global	Lista mis turnoss.
GET	/api/v1/turnos	Sí	admin	turnos:manage	—	30/min	Lista turnos.
POST	/api/v1/turnos	Sí	admin	turnos:manage	—	30/min	Crea un turno.
DELETE	/api/v1/turnos/{id}	Sí	admin	turnos:manage	—	30/min	Elimina un turno.
GET	/api/v1/turnos/{id}	Sí	admin	turnos:manage	—	30/min	Consulta un turno por ID.
PUT/PATCH	/api/v1/turnos/{id}	Sí	admin	turnos:manage	—	30/min	Actualiza un turno.

Chat y reportes

Tabla 141. *Matriz de endpoints: chat y reportes*

Método	Endpoint	Auth	Roles	Ability	Turno	Throttle	Operación
GET	/api/v1/mensajes	Sí	Todos los autenticados	–	—	60/min global	Bandeja o conversación: - Sin parámetro con: resumen de contactos (último mensaje + no leídos). - Con parámetro con: hilo completo con ese usuario.
POST	/api/v1/mensajes	Sí	Todos los autenticados	–	—	60/min global	Enviar un mensaje. Cliente → barista/admin. Barista/admin → cualquier usuario activo.

Método	Endpoint	Auth	Roles	Ability	Turno	Throttle	Operación
GET	/api/v1/mensajes/contactos	Sí	Todos los autenticados	–	—	60/min global	Contactos sugeridos según el rol del usuario autenticado.
POST	/api/v1/mensajes/leer	Sí	Todos los autenticados	–	—	60/min global	Marcar como leídos todos los mensajes recibidos de un contacto.
GET	/api/v1/reportes	Sí	admin	–	—	60/min global	Lista reportes.
POST	/api/v1/reportes	Sí	admin, barista	–	—	60/min global	Crea un reporte.
DELETE	/api/v1/reportes/{id}	Sí	admin	–	—	60/min global	Elimina un reporte.
GET	/api/v1/reportes/{id}	Sí	admin	–	—	60/min global	Consulta un reporte por ID.
PUT/PATCH	/api/v1/reportes/{id}	Sí	admin	–	—	60/min global	Actualiza un reporte.

Usuarios y roles

Tabla 142. *Matriz de endpoints: usuarios y roles*

Método	Endpoint	Auth	Roles	Ability	Turno	Throttle	Operación
GET	/api/v1/roles	Sí	admin	roles:manage	—	30/min	Lista rols.
POST	/api/v1/roles	Sí	admin	roles:manage	—	30/min	Crea un rol.
DELETE	/api/v1/roles/{id}	Sí	admin	roles:manage	—	30/min	Elimina un rol.
GET	/api/v1/roles/{id}	Sí	admin	roles:manage	—	30/min	Consulta un rol por ID.
PUT/PATCH	/api/v1/roles/{id}	Sí	admin	roles:manage	—	30/min	Actualiza un rol.
GET	/api/v1/usuario-roles	Sí	admin	usuarios:manage	—	30/min	Lista asignación usuario-rols.
POST	/api/v1/usuario-roles	Sí	admin	usuarios:manage	—	30/min	Crea un asignación usuario-rol.
DELETE	/api/v1/usuario-roles/{id}	Sí	admin	usuarios:manage	—	30/min	Elimina un asignación usuario-rol.
GET	/api/v1/usuario-roles/{id}	Sí	admin	usuarios:manage	—	30/min	Consulta un asignación usuario-rol por ID.
PUT/PATCH	/api/v1/usuario-roles/{id}	Sí	admin	usuarios:manage	—	30/min	Actualiza un asignación usuario-rol.
GET	/api/v1/usuarios	Sí	admin	usuarios:manage	—	30/min	Lista usuarios.
POST	/api/v1/usuarios	Sí	admin	usuarios:manage	—	30/min	Crea un usuario.
DELETE	/api/v1/usuarios/{id}	Sí	admin	usuarios:manage	—	30/min	Elimina un usuario.
GET	/api/v1/usuarios/{id}	Sí	admin	usuarios:manage	—	30/min	Consulta un usuario por ID.
PUT/PATCH	/api/v1/usuarios/{id}	Sí	admin	usuarios:manage	—	30/min	Actualiza un usuario.

Asistente KAFFA

Tabla 143. *Matriz de endpoints: asistente KAFFA*

Método	Endpoint	Auth	Roles	Ability	Turno	Throttle	Operación
`POST`	`/api/v1/asistente/chat`	Sí	Todos los autenticados	–	—	30/min	Envía una pregunta al asistente y devuelve su respuesta. El servicio persiste tanto la pregunta como la respuesta en el historial.
`DELETE`	`/api/v1/asistente/mensajes`	Sí	Todos los autenticados	–	—	30/min	Borra todo el historial del usuario autenticado.
`GET`	`/api/v1/asistente/mensajes`	Sí	Todos los autenticados	–	—	30/min	Historial de conversación del usuario autenticado. Se devuelve del turno más reciente al más antiguo; el frontend lo invierte para mostrarlo en orden cronológico.

26.3 Cómo leer la matriz

- **Auth**: exige `Authorization: Bearer {token}`. Sin token, la API responde 401 en JSON.
- **Roles**: el middleware `role:` exige al menos uno de los roles indicados. Admin, barista y cliente son acumulables.
- **Ability**: el token debe poseer el alcance indicado; de lo contrario responde 403 `TOKEN_SIN_ABILITY`. El token de admin usa `*`.
- **Turno**: el middleware `turno.activo` solo afecta a baristas; admin y cliente están exentos.
- **Throttle**: límite específico por usuario/IP. Las lecturas operativas se separaron de las escrituras para no agotar el cupo de 10/min al listar.

> **RECOMENDACIÓN**
>
> **Recomendación.** Para clientes nuevos, obtenga el token con `POST /api/v1/login` y consulte `GET /api/v1/me` antes de asumir roles. No deduzca permisos solo por el nombre de la ruta: la fuente de verdad es la combinación rol + ability + policy.

27. Despliegue en producción

No hay evidencia de un despliegue real en el repositorio (sin Docker, CI/CD ni configuración de servidor). Este capítulo describe una topología **recomendada** y un runbook reproducible. Está marcado como recomendación cuando no se pudo verificar contra el proyecto.

27.1 Topología recomendada

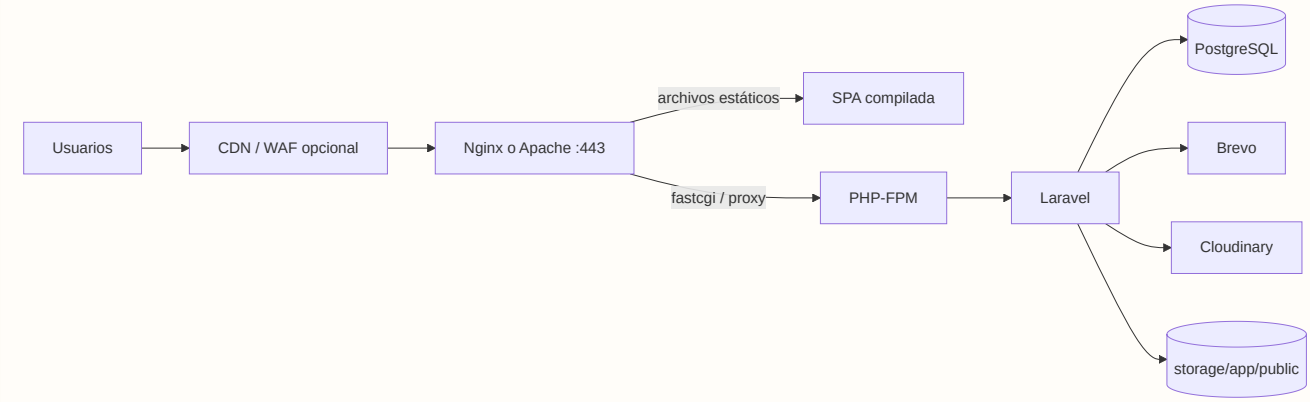


Figura 1. Topología de producción recomendada.

Tabla 144. *Componentes de la topología de producción*

Componente	Recomendación
Servidor web	Nginx 1.24+ o Apache 2.4+
PHP	PHP-FPM 8.2/8.3
Base de datos	PostgreSQL 15/16 dedicado
TLS	Let's Encrypt o certificado corporativo
Caché/colas	Redis 7 (opcional pero recomendado)
Backups	Almacenamiento externo cifrado
Monitoreo	Logs + health check + alertas

27.2 Preparar el servidor

```
sudo apt update
sudo apt install -y nginx php8.2-fpm php8.2-pgsql php8.2-mbstring \
  php8.2-xml php8.2-curl php8.2-bcmath php8.2-zip php8.2-intl \
  postgresql redis-server git unzip
```

Cree el usuario de servicio y el directorio:

```
sudo adduser --system --group --home /var/www/kaffa kaffa
sudo mkdir -p /var/www/kaffa
```

```
sudo chown -R kaffa:www-data /var/www/kaffa
```

27.3 Desplegar el backend

```
cd /var/www/kaffa
sudo -u kaffa git clone <repo> .
cd backend-kaffa
sudo -u kaffa composer install --no-dev --no-interaction --prefer-dist --optimize-autoloader
sudo -u kaffa cp .env.example .env
sudo -u kaffa php artisan key:generate
```

Edite `.env` con los valores de producción (capítulo 5). Luego:

```
sudo -u kaffa php artisan migrate --force
sudo -u kaffa php artisan storage:link
sudo -u kaffa php artisan config:cache
sudo -u kaffa php artisan route:cache
sudo -u kaffa php artisan view:cache
```

> **ADVERTENCIA**
>
> **Advertencia.** `migrate --force` omite la confirmación. Ejecútelo solo tras un backup y una prueba en staging.

27.4 Nginx

```
server {
    listen 80;
    server_name api.kaffa.example;
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl http2;
    server_name api.kaffa.example;

    root /var/www/kaffa/backend-kaffa/public;
    index index.php;

    ssl_certificate     /etc/letsencrypt/live/api.kaffa.example/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/api.kaffa.example/privkey.pem;

    client_max_body_size 12M;

    location / {
        try_files $uri $uri/ /index.php?$query_string;
    }

    location ~ \.php$ {
        include snippets/fastcgi-php.conf;
        fastcgi_pass unix:/run/php/php8.2-fpm.sock;
        fastcgi_param SCRIPT_FILENAME $realpath_root$fastcgi_script_name;
        fastcgi_param DOCUMENT_ROOT $realpath_root;
    }

    location ~ /\.(?!well-known).* {
        deny all;
    }
```

```
    location /storage/ {
        try_files $uri =404;
    }
}
```

Si la SPA se sirve desde el mismo dominio:

```
location / {
    root /var/www/kaffa/frontend/dist;
    try_files $uri $uri/ /index.html;
}

location /api/ {
    proxy_pass http://127.0.0.1:8000;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
}
```

27.5 Apache

```
<VirtualHost *:443>
    ServerName api.kaffa.example
    DocumentRoot /var/www/kaffa/backend-kaffa/public

    SSLEngine on
    SSLCertificateFile /etc/letsencrypt/live/api.kaffa.example/fullchain.pem
    SSLCertificateKeyFile /etc/letsencrypt/live/api.kaffa.example/privkey.pem

    <Directory /var/www/kaffa/backend-kaffa/public>
        AllowOverride All
        Require all granted
        Options FollowSymLinks
    </Directory>

    ErrorLog ${APACHE_LOG_DIR}/kaffa-error.log
    CustomLog ${APACHE_LOG_DIR}/kaffa-access.log combined
</VirtualHost>
```

Habilite `mod_rewrite`, `mod_ssl` y `mod_headers`.

27.6 PHP-FPM

`/etc/php/8.2/fpm/pool.d/kaffa.conf`:

```
[kaffa]
user = kaffa
group = www-data
listen = /run/php/php8.2-fpm.sock
listen.owner = www-data
listen.group = www-data
pm = dynamic
pm.max_children = 20
pm.start_servers = 4
pm.min_spare_servers = 2
pm.max_spare_servers = 8
```

```
pm.max_requests = 500
request_terminate_timeout = 60s
```

Ajuste `pm.max_children` según RAM y conexiones de PostgreSQL. Cada worker consume memoria; no lo dimensione "por si acaso".

27.7 Compilar la SPA

```
cd /var/www/kaffa/REACT_clean/REACT_kaffa
pnpm install --frozen-lockfile
pnpm run build
sudo rsync -a --delete dist/ /var/www/kaffa/frontend/dist/
```

Configure `VITE_CLOUDINARY_*` en el entorno de build. La URL de API es relativa (`/api/v1`); si usa otro dominio, ajuste `api.js`.

27.8 Runbook de despliegue

```
1. Anunciar ventana de mantenimiento (si hay migraciones).
2. Backup de base y storage.
3. git fetch && git checkout <tag>
4. composer install --no-dev
5. php artisan migrate --force
6. php artisan config:cache / route:cache / view:cache
7. pnpm build y publicar dist
8. Recargar PHP-FPM y Nginx
9. Smoke test: /health, login, catálogo, pedido de prueba
10. Monitorear logs y errores 5xx durante 15 min
```

27.9 Rollback

```
git checkout <tag-anterior>
composer install --no-dev
php artisan migrate:rollback --step=1   # solo si la migración es reversible
php artisan optimize:clear
sudo systemctl reload php8.2-fpm
```

Si la migración no es reversible, restaure el backup. Documente siempre el `down()`.

27.10 Variables de producción

Revise el checklist del capítulo 5. Como mínimo:

- `APP_ENV=production`
- `APP_DEBUG=false`
- `APP_URL` y `FRONTEND_URL` con HTTPS
- `DB_*` con usuario dedicado
- `SANCTUM_EXPIRATION=1440`
- `CORS_ALLOWED_ORIGINS` exacto

- SMTP Brevo
- `LOG_CHANNEL=daily`
- `SCRAMBLE_ENABLED=false`

27.11 Consideraciones de escalado

- La API es stateless (tokens), se puede balancear con varias instancias.
- PostgreSQL es el punto único; use réplicas de lectura solo si los reportes lo justifican.
- Redis para caché/colas si hay más de una instancia.
- El disco `storage/app/public` debe ser compartido o usar almacenamiento de objetos si hay varios nodos.
- Cloudinary evita compartir imágenes web entre nodos.

27.12 Checklist de despliegue

- [ ] Backup reciente verificado.
- [ ] `.env` de producción con secretos.
- [ ] `APP_DEBUG=false`.
- [ ] Migraciones probadas en staging.
- [ ] `storage:link` y permisos.
- [ ] HTTPS y redirección.
- [ ] CORS restringido.
- [ ] Colas/worker si aplica.
- [ ] Logs y alertas.
- [ ] Smoke test completo.
- [ ] Rollback documentado.

28. Observabilidad y operación

Operar KAFFA requiere saber si la API responde, si la base está sana, si los correos salen y si el disco de imágenes crece. Este capítulo propone un conjunto mínimo de señales y runbooks.

28.1 Health checks

Tabla 145. *Health checks por nivel*

Nivel	Qué comprobar	Cómo
Proceso	PHP-FPM/Nginx activos	`systemctl status`
HTTP	`GET /api/v1/health` con token	200 `{"status":"ok"}`
Base	`SELECT 1`	`psql` o endpoint propio
Disco	espacio y permisos	`df -h`, `ls storage`
Correo	envío de prueba	`Mail::raw`
Cloudinary	subida de prueba	consola/API

El endpoint `/health` actual **solo valida el token**. Para monitoreo real, añada un endpoint interno que compruebe base y almacenamiento, protegido por red o token.

28.2 Logs

```
tail -f storage/logs/laravel.log
tail -f /var/log/nginx/kaffa-error.log
tail -f /var/log/php8.2-fpm.log
```

Qué buscar:

- `ERROR` con stack trace.
- `No se pudo enviar el código de verificación`.
- `SQLSTATE` (conexión, deadlock, unique).
- `Token mismatch`/401 repetidos.
- `429` por throttle.

En producción use `LOG_CHANNEL=daily` y retención de 14–30 días. No registre tokens ni contraseñas.

28.3 Métricas recomendadas

Tabla 146. *Métricas recomendadas y alertas*

Métrica	Fuente	Alerta
Latencia p95 API	Nginx/APM	> 1 s
Tasa 5xx	Nginx/APM	> 1%
Tasa 401/403	Logs	Pico anómalo

Métrica	Fuente	Alerta
Tasa 429	Logs	Pico
Conexiones PostgreSQL	`pg_stat_activity`	> 80%
Espacio disco	`df`	> 80%
Tamaño `storage/app/public`	`du`	Crecimiento anómalo
Correos fallidos	Logs	> 0
Deadlocks	PostgreSQL	> 0 recurrente

28.4 Alertas mínimas

1. API no responde 2 minutos.
2. Base de datos inaccesible.
3. Disco > 85%.
4. Errores 5xx > 1% durante 5 min.
5. Fallos de correo > 3 en 10 min.
6. Backup diario no generado.

28.5 Backups y recuperación

Tabla 147. *Política de respaldos por activo*

Activo	Frecuencia	Retención	Prueba
PostgreSQL	Diaria	14–30 días	Restauración mensual
`storage/app/public/eventos`	Diaria	30 días	Acceso a una imagen
`.env`	Por cambio	Cifrado	Recuperación en staging
Código	Por release	Git	Checkout de tag
OpenAPI	Por release	Git	Comparación

Objetivos sugeridos:

- **RPO** (pérdida máxima de datos): 24 h o menos.
- **RTO** (tiempo de recuperación): 2–4 h.

Si el negocio no tolera perder un día de ventas, use WAL archiving/PITR.

28.6 Runbook: API caída

```
1. Comprobar DNS/TLS.
2. systemctl status nginx php8.2-fpm postgresql.
3. Revisar logs de Nginx/PHP/Laravel.
4. Verificar conexión a PostgreSQL.
5. Verificar espacio en disco.
6. Si hay migración reciente, evaluar rollback.
7. Restaurar servicio y smoke test.
8. Registrar incidente y causa raíz.
```

28.7 Runbook: correos no llegan

```
1. Revisar MAIL_* y que no esté MAIL_MAILER=log.
2. Probar SMTP con tinker.
3. Revisar logs de Brevo (rebotes, límites).
4. Verificar remitente y DNS/DKIM.
5. Revisar `codigos_otp` (existen, no expiraron).
6. Reenviar código y confirmar recepción.
```

28.8 Runbook: caja descuadrada

```
1. Consultar caja y movimientos.
2. Recalcular sistema = apertura + ingresos - egresos.
3. Comparar con monto_cierre_fisico.
4. Revisar pagos no registrados o movimientos duplicados.
5. No editar movimientos a mano sin autorización.
6. Registrar ajuste y documentar.
```

28.9 Runbook: stock negativo o inconsistente

```
1. Revisar insumo y recetas.
2. Consultar compras/mermas/ventas recientes.
3. Verificar que el listener DeducirInventario se ejecutó.
4. Si hay negativo, corregir con compra/merma de ajuste.
5. Revisar transacciones concurrentes/deadlocks.
```

28.10 Capacidad y crecimiento

- Revise mensualmente el tamaño de `pedidos`, `detalle__pedidos`, `movimiento__cajas` y `personal_access_tokens`.
- Archive datos históricos si las consultas se vuelven lentas.
- Depure tokens expirados:

```sql
DELETE FROM personal_access_tokens
WHERE expires_at IS NOT NULL AND expires_at < now() - interval '30 days';
```

- Depure pendientes antiguos (el servicio lo hace, pero puede forzarse).
- Monitoree la cuota de Cloudinary.

28.11 Documentación operativa

Mantenga un runbook vivo con:

- Contactos de responsables.
- Proveedores (Brevo, Cloudinary, hosting).
- Procedimiento de backup/restore.
- Credenciales en gestor de secretos.
- Historial de incidentes.

- Ventanas de mantenimiento.

29. Seguridad operativa

Este capítulo complementa el 11 con medidas de infraestructura y hábitos de operación. No sustituye un pentest.

29.1 Superficie de exposición

Tabla 148. *Superficie de exposición y riesgo*

Elemento	Exposición	Riesgo
`/api/v1/*`	Público	Fuerza bruta, abuso, enumeración
`/docs/api`	Público si Scramble activo	Revela contrato
PostgreSQL 5432	Debe ser interno	Acceso directo
`storage/app/public`	Público	Imágenes, no secretos
`.env`	Nunca público	Claves
Logs	Interno	Datos personales
Backups	Interno/cifrado	Fuga masiva

29.2 Hardening del servidor

- Actualice SO, PHP, Nginx/Apache y PostgreSQL.
- Deshabilite servicios innecesarios.
- Use firewall (UFW/nftables/security group).
- SSH solo con clave, sin root directo.
- Fail2ban o equivalente para SSH/HTTP.
- `APP_DEBUG=false`.
- `expose_php=Off`.
- Cabeceras de seguridad: `X-Content-Type-Options`, `X-Frame-Options`, `Referrer-Policy`, `Content-Security-Policy` (SPA).
- TLS moderno; deshabilite TLS 1.0/1.1.
- Rotación de logs.

29.3 Gestión de secretos

Tabla 149. *Gestión y rotación de secretos*

Secreto	Dónde vive	Rotación
`APP_KEY`	`.env` /gestor	Rara; respaldar
`DB_PASSWORD`	`.env` /gestor	90–180 días
`MAIL_PASSWORD`	`.env` /gestor	90 días
`CLOUDINARY_API_SECRET`	Backend	90–180 días
Tokens de usuario	BD	Expiración/revocación
Claves de backup	Gestor	180 días

Nunca incluya secretos en imágenes, repositorios, tickets o capturas. Si se filtran, rote de inmediato y revise accesos.

29.4 Seguridad de la base de datos

- Usuario dedicado, sin superusuario.
- `pg_hba.conf` restringido por red.
- TLS si hay red externa.
- Copias cifradas.
- Auditoría de accesos.
- No exponer 5432 a Internet.
- Revisar extensiones y permisos.

29.5 Seguridad de la API

- Tokens Bearer con expiración.
- Throttle por ruta.
- Validación de entrada con Form Requests.
- Policies y abilities.
- Respuestas JSON sin trazas.
- CORS restringido.
- No confiar en `role:` solo.
- No aceptar URLs de imágenes de dominios no permitidos.
- No permitir mass assignment: usar `validated()`.

29.6 Seguridad del frontend

- No guardar el API secret de Cloudinary.
- Sanitizar HTML del Asistente antes de renderizar.
- Evitar `dangerouslySetInnerHTML` con datos del usuario.
- CSP para mitigar XSS.

- HTTPS.
- No cachear tokens en logs del navegador.
- Cerrar sesión en equipos compartidos.

29.7 Respuesta a incidentes

```
1. Contener: revocar tokens, bloquear IP, desactivar usuario.
2. Evaluar: logs, accesos, datos afectados.
3. Erradicar: parche, rotación de secretos.
4. Recuperar: restaurar servicio, verificar integridad.
5. Notificar: responsables/usuarios según normativa.
6. Documentar: causa raíz y acciones.
```

29.8 Checklist de auditoría trimestral

- [ ] `composer audit` y `pnpm audit` sin críticos.
- [ ] Usuarios inactivos revisados.
- [ ] Tokens expirados depurados.
- [ ] Permisos de base revisados.
- [ ] Backups restaurados en staging.
- [ ] Logs sin secretos.
- [ ] CORS y TLS verificados.
- [ ] Scramble desactivado.
- [ ] Dependencias de Cloudinary/Brevo rotadas.
- [ ] Pruebas de permisos por rol.

30. Pruebas y calidad

El backend usa PHPUnit 10 sobre PostgreSQL. La suite documentada tiene **88 pruebas y 298 aserciones**; el conteo literal de métodos puede variar al añadir casos. Este capítulo explica qué cubre, qué no y cómo ampliarla.

> **VERIFICADO**
> **Verificado (2026-10-20).** `php artisan test` sobre `kaffa_test`: **88 pruebas aprobadas, 298 aserciones, 2 pruebas marcadas como deprecated** y avisos de PHP 8.5 por la constante `PDO::MYSQL_ATTR_SSL_CA` en `config/database.php`, ajenos a la lógica de negocio.

30.1 Configuración

`phpunit.xml`:

- Base `kaffa_test` (PostgreSQL).
- `BCRYPT_ROUNDS=4` para acelerar.
- Caché/cola/sesión en `array`.
- `RefreshDatabase` por prueba.

```
php artisan test
vendor/bin/phpunit
```

30.2 Inventario de pruebas

Tabla 150. *Inventario de archivos de prueba*

Archivo	Qué verifica
`ApiTest`	Login, 401 sin token, catálogo público, paginación
`EquipoTest`	`/equipo`: público, administradora, filtros, sin datos sensibles
`EmailVerificationTest`	Registro, OTP, expiración, intentos, reenvío
`PasswordResetTest`	OTP, respuesta uniforme, reseteo, revocación de tokens
`PedidoTest`	Autoría, nota, factura, stock, comprobante, pagos, scope
`CajaTest`	Apertura, turno, arqueo, movimientos, eliminación, caja ajena
`CompraMermaTest`	Stock, comprobante, reversión, merma, usuario forzado
`TurnoMiddlewareTest`	Turno activo, códigos, exenciones, `abierto`, `fecha_fin`
`ChatReporteTest`	Chat, no leídos, reportes, roles
`TokenSessionTest`	Refresh, logout, logout-all, ability
`AsistenteTest`	Multi-rol, respuesta local, historial, 401
`EventoTest`	Público/publicado, admin, galería, subida, 403

30.3 Qué cubre la suite

- Autenticación y sesión.
- Verificación por OTP y reseteo.
- Catálogo público sin sesión.
- Pedidos: autoría, stock, pagos, factura, scope.
- Caja: apertura, cierre, arqueo, movimientos.
- Compras/mermas: stock y reversión.
- Turnos: `abierto`, rango, exenciones.
- Chat/reportes.
- Tokens: rotación, revocación, ability.
- Asistente: roles e historial.
- Eventos: publicación, galería, imagen.

30.4 Gaps

Tabla 151. *Brechas de cobertura conocidas*

Área	Estado
CRUD completo de catálogo	`[GAP]`
Gestión de usuarios/roles/turnos	`[GAP]`
Gastos con caja	`[GAP]`
Subida a Cloudinary	`[GAP]` (frontend)
Filtros avanzados	`[GAP]`
Throttle por ruta	`[GAP]`
Cobertura de código	No medida
Frontend	Sin tests

30.5 Cómo añadir una prueba

1. Cree el archivo en `tests/Feature`.
2. Use `RefreshDatabase`.
3. Cree usuarios con `User::factory()` o manualmente.
4. Obtenga token con `postJson('/api/v1/login', ...)`.
5. Ejecute la acción.
6. Afirme status, estructura y efectos en base.
7. Limpie el guard entre usuarios: `$this->app['auth']->forgetGuards()`.

Ejemplo:

```
public function test_cliente_solo_ve_sus_pedidos(): void
{
    $cliente = User::factory()->create();
    $otro = User::factory()->create();
```

```
        // crear pedidos...
        $token = $this->postJson('/api/v1/login', [
            'correo' => $cliente->correo,
            'password' => 'password',
        ])->json('access_token');

        $response = $this->withHeader('Authorization', "Bearer $token")
            ->getJson('/api/v1/pedidos');

        $response->assertOk();
        $this->assertCount(1, $response->json('data'));
    }
```

30.6 Pruebas manuales de aceptación

Tabla 152. *Pruebas manuales de aceptación*

#	Flujo	Resultado
1	Login 3 roles	Redirección/roles correctos
2	Registro → OTP → login	Usuario verificado
3	Pedido cliente con pago virtual sin comprobante	422
4	Pedido con comprobante	201 y stock descontado
5	Barista fuera de turno	403 `FUERA_HORARIO` / `TURNO_INACTIVO`
6	Barista abre/cierra caja	Arqueo correcto
7	Compra sube stock	Verificar insumo
8	Merma baja stock	Verificar insumo
9	Eliminar merma restaura stock	Verificar
10	Admin crea turno con rango	Barista opera dentro
11	Asistente admin/barista/cliente	Respuestas por rol
12	Evento con imagen móvil	URL accesible

30.7 Calidad de código

```
vendor/bin/pint --test
vendor/bin/phpunit
```

- PSR-12.
- Sin `dd()` / `dump()` en producción.
- Sin `env()` fuera de `config`.
- Sin consultas en bucles.
- Sin secretos en código.
- Migraciones reversibles.

30.8 CI recomendado

```
# pseudocódigo
services:
  postgres:
    image: postgres:16
    env: POSTGRES_DB=kaffa_test ...
steps:
  - composer install
  - cp .env.example .env
  - php artisan key:generate
  - php artisan test
  - vendor/bin/pint --test
```

No hay CI en el repositorio; este pipeline es una recomendación.

30.9 Criterios de aceptación de un cambio

- [ ] Pruebas nuevas/actualizadas.
- [ ] Suite completa verde.
- [ ] Migración probada en limpio.
- [ ] OpenAPI regenerado.
- [ ] Matriz de endpoints actualizada.
- [ ] Sin secretos.
- [ ] Documentación sincronizada.

> **NOTA**
>
> **Evaluación de calidad.** Las métricas de este proceso se comparan con ISO/IEC 29119 en el capítulo 32 (§32.6).

31. Mantenimiento y contribución

Este capítulo explica cómo evolucionar el backend sin romper la documentación ni la operación.

31.1 Añadir un endpoint

1. Defina la ruta en `routes/api.php` con middleware y nombre.
2. Cree/actualice el Form Request.
3. Implemente el controlador (delgado).
4. Si hay reglas complejas, cree un servicio.
5. Añada policy/gate si aplica.
6. Actualice `openapi.json`.
7. Actualice la matriz del capítulo 26.
8. Añada prueba.
9. Documente en el capítulo correspondiente.

Checklist:

- [ ] ¿Requiere auth?
- [ ] ¿Qué roles?
- [ ] ¿Qué ability?
- [ ] ¿Requiere turno?
- [ ] ¿Qué throttle?
- [ ] ¿Qué pasa si falta un dato?
- [ ] ¿Qué pasa en concurrencia?
- [ ] ¿Afecta stock/caja/factura?
- [ ] ¿Es idempotente?

31.2 Añadir una tabla o columna

1. Cree la migración.
2. Actualice el modelo (`fillable`, `casts`, relaciones).
3. Actualice requests y validaciones.
4. Actualice policies si cambia el ownership.
5. Actualice el diccionario del capítulo 8.
6. Añada índices si se filtra/ordena.
7. Pruebe `migrate:fresh --seed`.
8. Documente el impacto en reportes.

31.3 Regenerar OpenAPI

```
cd backend-kaffa
php artisan serve
curl -s http://localhost:8000/docs/api.json -o ../docs/openapi.json
```

Luego:

```
python3 manual-tecnico/build/gen_endpoints.py
```

Y reconstruya el PDF.

31.4 Mantener el manual

```
cd docs/manual-tecnico
node build/build-all.mjs
```

El script:

1. Genera HTML.
2. Imprime portada/cuerpo/contraportada.
3. Calcula páginas del índice.
4. Reimprime si cambió.
5. Une el PDF final.

Revise:

- Que el índice tenga páginas correctas.
- Que no haya tablas cortadas.
- Que los diagramas rendericen.
- Que las tildes se vean bien.
- Que el PDF final esté en `docs/`.

31.5 Convenciones de documentación

- Español formal.
- Código y rutas en `mono`.
- Tablas con encabezado.
- Figuras con caption.
- Marcas `[NO VERIFICADO]`, `[GAP]`, **Recomendación**.
- Sin secretos reales.
- Sin números de línea; citar archivo/símbolo.

31.6 Deuda técnica conocida

Tabla 153. *Deuda técnica conocida*

Tema	Impacto	Sugerencia
Sin idempotencia	Duplicados por reintento	`Idempotency-Key`
Sin tests de frontend	Regresiones visuales	Playwright/Vitest
Sin CI/CD	Despliegues manuales	GitHub Actions
Cloudinary unsigned	Abuso de cuota	Subida firmada
Scramble en dev	Exposición si se olvida	Bloquear en prod
`QUEUE_CONNECTION=sync`	Bloqueo por SMTP	Redis + worker
Sin facturación fiscal	Incumplimiento	Proveedor autorizado
Sin métricas	Incidentes tardíos	APM/alertas

31.7 Roadmap sugerido

1. Endurecer Cloudinary (firma).
2. Idempotencia en pedidos/pagos.
3. CI/CD con pruebas y migraciones.
4. Observabilidad (Sentry/Prometheus).
5. Facturación electrónica.
6. App móvil con manejo seguro de tokens.
7. Reportes exportables y conciliación.

31.8 Contacto y soporte

Documente en el repositorio:

- Responsable funcional.
- Responsable técnico.
- Proveedor de hosting/DB.
- Cuentas Brevo/Cloudinary.
- Procedimiento de escalamiento.

RECOMENDACIÓN

Recomendación. Trate este manual como artefacto vivo: cada cambio de código debe actualizar la sección correspondiente y regenerar el PDF. La documentación que no se ejecuta se desactualiza en semanas.

NOTA

Evaluación de calidad. La gestión de cambios y el mantenimiento se evalúan frente a ISO/IEC/IEEE 12207 en el capítulo 32 (§32.7).

32. Calidad del sistema frente a normas ISO

Este capítulo responde a una pregunta distinta de las anteriores: no describe cómo funciona KAFFA, sino **cómo se midió su calidad** y qué tan cerca está de las normas internacionales que definen qué es un sistema de calidad. La evaluación se hizo sobre el backend y sus interfaces, con evidencia verificable en el repositorio, y se documenta con el mismo criterio que el resto del manual: lo que no se pudo medir se marca como brecha, no se presume.

> **NOTA**
>
> **Alcance de la afirmación.** Este capítulo es una **evaluación interna basada en evidencia**, no una certificación. Ninguna de las normas citadas fue auditada por un organismo certificador. Cuando se dice que una práctica "se alinea" con una cláusula, significa que existe evidencia documental o técnica coherente con ella; no implica conformidad formal.

32.1 Método de evaluación

La evaluación sigue el proceso de la familia **ISO/IEC 25040** (evaluación de la calidad de producto de software): establecer los requisitos de evaluación, especificar la evaluación, diseñar el plan, ejecutar las mediciones y concluir. Para cada característica se definió un indicador, una forma de cálculo y un nivel en una escala de madurez de cinco valores.

Tabla 154. *Escala de madurez usada en la evaluación*

Nivel	Nombre	Criterio
0	Inexistente	No hay práctica ni evidencia
1	Inicial	Hay una práctica aislada, sin medición ni documentación
2	Definido	La práctica está implementada y documentada, pero no se mide
3	Medido	La práctica está implementada, probada y existe una métrica
4	Optimizado	Existe umbral, seguimiento y mejora continua sobre la métrica

El puntaje de cada característica es el promedio simple de sus subcaracterísticas aplicables; el índice global es el promedio simple de las nueve características de ISO/IEC 25010:2023. Los valores no aplicables se excluyen y se justifican.

Fuentes de evidencia. Código del backend en el commit `1ea1e76`, suite de pruebas sobre PostgreSQL `kaffa_test`, `php artisan route:list`, `docs/openapi.json`, migraciones, configuración, análisis estático del texto del manual y las herramientas del propio ecosistema PHP.

Tabla 155. *Mediciones ejecutadas y comando reproducible*

Medición	Resultado	Comando
Pruebas automatizadas	88 aprobadas, 2 marcadas como deprecated, 298 aserciones, 2,78 s	`php artisan test`
Archivos y líneas del backend	141 archivos PHP, 7.311 líneas en `app/`	`find app -name "*.php"`
Operaciones de API	140 operaciones, 127 con autenticación, 104 con rol, 92 con ability	matriz del capítulo 26

Medición	Resultado	Comando
Estructura	30 controladores, 10 servicios, 18 policies, 26 Form Requests, 29 modelos, 44 migraciones	`ls app/...`
Índices declarados	30 declaraciones `index` / `unique`, 12 índices de rendimiento	`grep` en `database/migrations`
Transacciones y bloqueos	25 usos de transacción, 14 `lockForUpdate`	`grep` en `app/`
Estilo de código	120 de 228 archivos con desviaciones, 24 reglas	`vendor/bin/pint --test`
Vulnerabilidades de dependencias	39 avisos en 10 paquetes de producción; 43 en 11 con desarrollo	`composer audit --no-dev`
Cobertura de pruebas	No medida: no hay `xdebug` ni `pcov` instalados	`php -m`
Análisis estático	No configurado: no hay PHPStan, Psalm ni PHPMD	`composer.json`

NOTA

Nota. Las cifras corresponden al 2 de noviembre de 2026. El manual base fue auditado en el commit `01fb00d`; el repositorio avanzó hasta `1ea1e76` con dos commits de despliegue (dockerización y credenciales desde `.env`). Esa diferencia se registra en las limitaciones de la sección 32.11.

32.2 Correspondencia entre las normas y el manual

Tabla 156. *Normas ISO consideradas y lugar donde se aplican*

Norma	Objeto	Dónde se aplica en KAFFA	Sección de este manual
ISO 9001:2015	Sistema de gestión de la calidad	Control documental, enfoque a procesos, mejora	32.5; preliminares; capítulo 31
ISO/IEC 25000 (SQuaRE)	Marco de evaluación de calidad	Método, escala y conclusiones	32.1, 32.3, 32.9
ISO/IEC 25010:2023	Modelo de calidad de producto	Las nueve características medidas	32.3; capítulos 2, 9, 11, 14 y 30
ISO/IEC 27001:2022	Sistema de gestión de seguridad	Controles organizativos y tecnológicos	32.4; capítulos 11 y 29
ISO/IEC 27002:2022	Guía de controles de seguridad	Prácticas operativas y de acceso	32.4; capítulo 29
ISO/IEC 29119-1:2022	Conceptos y procesos de prueba	Estrategia, niveles y métricas de prueba	32.6; capítulo 30
ISO/IEC/IEEE 12207:2017	Procesos del ciclo de vida	Mantenimiento, configuración y cambios	32.7; capítulo 31
ISO/IEC/IEEE 42010:2011	Descripción de arquitectura	Vistas, interesados y decisiones	32.7; capítulo 2
ISO 8601-1:2019	Fechas y horas	Formato de fechas de la API	32.8; sección 15.2
ISO 14289-1:2014 (PDF/UA)	Accesibilidad de documentos PDF	PDF etiquetado y texto extraíble	32.8; este manual
ISO 31000:2018	Gestión del riesgo	Identificación y mitigación de riesgos	32.4 y 32.10; sección 2.9

32.3 ISO/IEC 25010:2023 — calidad de producto

ISO/IEC 25010:2023 define nueve características de calidad de producto. Cada subsección presenta el indicador usado, el resultado y el nivel asignado. Las métricas se obtuvieron del repositorio y de la suite; las que no pudieron medirse se declaran como brecha.

32.3.1 Adecuación funcional

Tabla 157. *Medición de la adecuación funcional*

Subcaracterística	Indicador	Resultado	Nivel
Completitud funcional	Operaciones documentadas / operaciones de la matriz	140/140 = 100 %	3
Corrección funcional	Pruebas aprobadas / pruebas ejecutadas	88/88 = 100 %	3
Pertinencia funcional	Escrituras con validación dedicada	26 Form Requests; 25 rutas con throttle	3

Puntaje: 3,0. La API está completamente documentada y la suite pasa sin fallos. La limitación es que la completitud se mide sobre la documentación, no sobre una prueba por endpoint: solo 28 rutas aparecen referenciadas de forma explícita en los archivos de prueba.

32.3.2 Eficiencia de desempeño

Tabla 158. *Medición de la eficiencia de desempeño*

Subcaracterística	Indicador	Resultado	Nivel
Comportamiento temporal	Prueba de carga con umbral de latencia	No existe	1
Utilización de recursos	Índices y bloqueos controlados	30 índices; 14 bloqueos puntuales	2
Capacidad	Prueba de esfuerzo o estimación de concurrencia	No existe; solo límites de throttle	1

Puntaje: 1,3. Hay trabajo real de optimización (índices por tabla, bloqueos acotados, `GREATEST` para evitar negativos), pero **no hay ninguna medición de tiempo de respuesta ni de concurrencia**. Sin esa evidencia no se puede afirmar eficiencia; se afirma únicamente que el diseño la favorece.

32.3.3 Compatibilidad

Tabla 159. *Medición de la compatibilidad*

Subcaracterística	Indicador	Resultado	Nivel
Coexistencia	Clientes y servicios conviviendo en el mismo entorno	SPA, app móvil, Brevo, Cloudinary y SMTP	3
Interoperabilidad	Contrato de intercambio documentado	JSON, OpenAPI con 69 paths, fechas ISO 8601	3

Puntaje: 3,0. El backend expone un contrato estable y versionado en `/api/v1`; los clientes son desacoplados y las integraciones externas están documentadas con sus modos de fallo.

32.3.4 Capacidad de interacción

Tabla 160. *Medición de la capacidad de interacción*

Subcaracterística	Indicador	Resultado	Nivel
Reconocibilidad	Catálogo de errores y mensajes accionables	9 códigos de negocio y respuestas uniformes	3
Aprendizaje	Documentación de uso de la API	Manual, OpenAPI y ejemplos `curl`	3
Operabilidad	Paginación, filtros y envelope consistentes	Documentado en el capítulo 15	3
Protección ante errores de usuario	Validación de entrada	26 Form Requests	3
Inclusividad	Accesibilidad de la documentación	PDF etiquetado y texto extraíble; sin auditoría PDF/UA	2

Puntaje: 2,8. La API es predecible y está bien documentada. La accesibilidad del PDF mejoró al eliminar las ligaduras que rompían el copiado, pero no se ejecutó una validación formal con un lector de pantalla.

32.3.5 Fiabilidad

Tabla 161. *Medición de la fiabilidad*

Subcaracterística	Indicador	Resultado	Nivel
Ausencia de fallos	Pruebas verdes y defectos abiertos conocidos	88/88; sin registro de defectos	2
Disponibilidad	Health checks y monitoreo	Health checks documentados; sin SLA medido	2
Tolerancia a fallos	Operaciones críticas en transacción	25 transacciones; rollback en pagos y stock	3
Recuperabilidad	Respaldo y restauración probados	Respaldo documentado; restauración no probada	2

Puntaje: 2,3. La tolerancia a fallos está bien resuelta con transacciones y bloqueos, pero no hay evidencia de disponibilidad medida ni de una prueba real de restauración.

32.3.6 Seguridad

Tabla 162. *Medición de la seguridad*

Subcaracterística	Indicador	Resultado	Nivel
Confidencialidad	Hash de contraseñas, secretos y dependencias	bcrypt con 12 rondas; `.env` fuera de Git; 39 avisos de dependencias	2
Integridad	Validación y transacciones	Form Requests y transacciones en escrituras críticas	3
No repudio	Registro de auditoría inmutable	No existe tabla de auditoría	1
Responsabilidad	Trazabilidad de autoría	`usuario_id`, `abierto_por`, `cerrada_por`, timestamps	3
Autenticidad	Autenticación y verificación	Sanctum con abilities y OTP de un solo uso	3

Puntaje: 2,4. El control de acceso es sólido, pero la gestión de vulnerabilidades y el no repudio son brechas concretas. Los 39 avisos de `composer audit` en dependencias de producción son la evidencia más contundente de que falta un proceso de actualización.

32.3.7 Mantenibilidad

Tabla 163. *Medición de la mantenibilidad*

Subcaracterística	Indicador	Resultado	Nivel
Modularidad	Separación de responsabilidades	30 controladores, 10 servicios, 18 policies, 29 modelos	3
Reusabilidad	Componentes compartidos	`BasePolicy`, `HasBasicQueries`, `TransactionService`	3
Analizabilidad	Análisis estático y cobertura	Sin PHPStan/Psalm/PHPMD; sin cobertura	1
Modificabilidad	Deuda técnica y estilo	120/228 archivos fuera de Pint; Laravel 10 sin soporte de seguridad	1
Capacidad de prueba	Suite automatizada	14 archivos, 88 pruebas, 298 aserciones, 2,78 s	3

Puntaje: 2,2. La arquitectura es entendible y la suite es rápida, pero la ausencia de análisis estático y el volumen de desviaciones de estilo hacen que el cambio sea más riesgoso de lo necesario.

32.3.8 Flexibilidad

Tabla 164. *Medición de la flexibilidad*

Subcaracterística	Indicador	Resultado	Nivel
Adaptabilidad	Configuración por entorno	Variables en `.env` y `config/`	3
Escalabilidad	Procesamiento asíncrono y escala horizontal	`QUEUE_CONNECTION=sync`; sin evidencia de escala	1
Instalabilidad	Procedimiento reproducible	Manual de instalación; dockerización posterior al commit auditado	2
Reemplazabilidad	Abstracciones de servicios externos	Integraciones directas, sin interfaces de proveedor	1

Puntaje: 1,8. La configuración es flexible, pero el acoplamiento directo a Cloudinary y Brevo, y la cola síncrona, limitan la sustitución y la escala.

32.3.9 Seguridad funcional (safety)

Tabla 165. *Medición de la seguridad funcional*

Subcaracterística	Indicador	Resultado	Nivel
Mitigación de fallos operativos	Reglas que evitan daño al negocio	Stock no negativo, caja con arqueo, factura única	3
Identificación de riesgos	Registro y tratamiento de riesgos	Tabla de riesgos arquitectónicos sin revisión periódica	2

Puntaje: 2,5. El dominio está protegido contra los errores más costosos (sobreventa, caja descuadrada, factura duplicada); lo que falta es un proceso formal de gestión de riesgos.

32.4 ISO/IEC 27001:2022 — controles de seguridad

La norma organiza los controles en cuatro temas. La tabla siguiente no afirma conformidad: indica qué controles tienen evidencia en el proyecto y cuáles son brechas.

Tabla 166. *Cobertura de controles de ISO/IEC 27001:2022*

Control	Tema	Estado	Evidencia o brecha
A.5.1 Políticas de seguridad	Organizativo	Parcial	Capítulo 11 y `docs/05-seguridad.md`; sin aprobación formal
A.5.15 Control de acceso	Organizativo	Implementado	RBAC con roles, abilities y policies
A.5.23 Seguridad en servicios cloud	Organizativo	Parcial	Brevo y Cloudinary documentados; sin contrato de nivel de servicio
A.6.3 Concienciación y formación	Personas	Brecha	No hay plan de formación
A.7.4 Seguridad física	Físico	No aplica	Infraestructura de terceros
A.8.2 Acceso privilegiado	Tecnológico	Implementado	Rol `admin` separado y abilities específicas
A.8.5 Autenticación segura	Tecnológico	Implementado	Sanctum, OTP y hash bcrypt
A.8.7 Protección contra malware	Tecnológico	No aplica	Backend sin ejecución de archivos de usuario
A.8.8 Gestión de vulnerabilidades	Tecnológico	Brecha	39 avisos de dependencias; sin proceso de actualización
A.8.9 Gestión de configuración	Tecnológico	Parcial	`.env.example` documentado; sin inventario de configuración
A.8.12 Prevención de fuga de datos	Tecnológico	Parcial	Respuestas filtradas por rol; sin DLP
A.8.13 Respaldo de información	Tecnológico	Parcial	Respaldo documentado; restauración no probada
A.8.15 Registro de eventos	Tecnológico	Parcial	Logs de Laravel; sin registro de auditoría de negocio
A.8.16 Monitoreo	Tecnológico	Parcial	Health checks y métricas recomendadas; sin alertas activas
A.8.24 Uso de criptografía	Tecnológico	Implementado	bcrypt, tokens y TLS recomendado
A.8.25 Ciclo de vida de desarrollo seguro	Tecnológico	Parcial	Pruebas y revisión; sin análisis estático
A.8.28 Codificación segura	Tecnológico	Parcial	Form Requests y ORM; 120/228 archivos fuera de Pint
A.8.32 Gestión de cambios	Tecnológico	Implementado	Git, CHANGELOG y control de versiones del manual
A.8.34 Protección durante auditoría	Tecnológico	No aplica	Sin auditorías externas

Lectura del resultado. De 19 controles evaluados: 6 implementados, 9 parciales, 2 brechas y 2 no aplican. Las brechas críticas son la **gestión de vulnerabilidades** (A.8.8) y la **prueba de restauración** (A.8.13), porque ambas afectan la continuidad del negocio.

32.5 ISO 9001:2015 — gestión de la calidad

ISO 9001 no evalúa software: evalúa la capacidad de una organización para cumplir requisitos y mejorar. El proyecto se alinea con sus principios en cuatro puntos verificables.

Tabla 167. *Alineación con ISO 9001:2015*

Cláusula	Práctica en KAFFA	Evidencia
4.4 Enfoque a procesos	Flujos con entradas, salidas y responsables	Capítulo 9
7.5 Información documentada	Control documental, versión y código	Preliminares: `KAFFA-MT-BE-001`
8.5 Provisión del servicio	Despliegue, operación y runbook	Capítulos 27 y 28
9.1 Seguimiento y medición	Suite de pruebas y métricas de calidad	Este capítulo y capítulo 30
10.2 No conformidad y acción correctiva	Registro de cambios y brechas	`CHANGELOG.md`; sección 32.10
10.3 Mejora continua	Mantenimiento y deuda técnica	Capítulo 31

Puntaje: 2,5. La documentación y el control de cambios son comparables a los de un proceso definido; lo que falta es un ciclo formal de auditoría interna y revisión por la dirección.

32.6 ISO/IEC 29119 — proceso de pruebas

Tabla 168. *Métricas del proceso de prueba*

Métrica	Resultado	Lectura
Casos automatizados	88 pruebas en 14 archivos	Cobertura funcional amplia de los flujos críticos
Aserciones	298	Verificación de comportamiento, no solo de código HTTP
Tiempo de ejecución	2,78 s	Apto para integración continua
Niveles de prueba	Predominio de pruebas de integración (`Feature`)	Falta una capa unitaria significativa
Trazabilidad	Requisito → prueba en el anexo F	Parcial: 28 rutas referenciadas de forma explícita
Cobertura de código	No medida	Brecha principal del proceso

Puntaje: 2,3. El proceso existe, es rápido y cubre los flujos de dinero, stock y acceso. No cumple el nivel "medido" de ISO/IEC 29119 porque no hay cobertura ni criterios de salida cuantificados por módulo.

32.7 ISO/IEC/IEEE 12207 y 42010 — ciclo de vida y arquitectura

Tabla 169. *Alineación con los procesos de ciclo de vida y arquitectura*

Norma	Proceso	Evidencia	Nivel
ISO/IEC/IEEE 12207	Mantenimiento y gestión de configuración	Git, `CHANGELOG.md`, capítulo 31	3
ISO/IEC/IEEE 12207	Gestión de cambios	Commits convencionales y control de versiones del manual	2
ISO/IEC/IEEE 42010	Descripción de arquitectura	Vistas de contexto, capas, despliegue y decisiones (capítulo 2)	3
ISO/IEC/IEEE 42010	Interesados y preocupaciones	Actores y riesgos (secciones 1.2 y 2.9)	2

32.8 ISO 8601 y PDF/UA — formatos y accesibilidad

Fechas (ISO 8601). La API devuelve fechas en formato ISO 8601 con desplazamiento horario (`2026-10-20T15:47:46-05:00`) y el backend opera en `America/Bogota`. La convención está documentada en la sección 15.2 y se aplica en todos los `timestamps` de Eloquent. **Nivel: 4.**

Accesibilidad del PDF (ISO 14289-1, PDF/UA). El PDF se genera etiquetado (`generateTaggedPDF`) y con esquema de documento; las imágenes llevan texto alternativo y el texto es extraíble. La corrección de las ligaduras tipográficas eliminó un defecto que impedía buscar y copiar palabras con `fi` y `fl`. No se ejecutó una validación formal con un validador PDF/UA, por lo que el nivel es **2 (definido)**, no 4.

32.9 Resultado global

Tabla 170. *Resumen de la evaluación ISO/IEC 25010:2023*

Característica	Puntaje (0–4)	Nivel
Adecuación funcional	3,0	Medido
Eficiencia de desempeño	1,3	Inicial
Compatibilidad	3,0	Medido
Capacidad de interacción	2,8	Definido
Fiabilidad	2,3	Definido
Seguridad	2,4	Definido
Mantenibilidad	2,2	Definido
Flexibilidad	1,8	Inicial
Seguridad funcional	2,5	Definido
Índice global	**2,4 / 4**	**Definido**

Interpretación. KAFFA es un sistema **funcionalmente completo y seguro en su control de acceso**, con una arquitectura mantenible y una suite de pruebas rápida. Su debilidad no está en lo que hace, sino en **cómo se demuestra**: no hay cobertura, no hay pruebas de carga, no hay análisis estático y las dependencias acumulan avisos de seguridad. El índice de 2,4 sobre 4 es coherente con esa realidad: un sistema bien construido, con un proceso de medición todavía inmaduro.

32.10 Brechas y plan de mejora

Tabla 171. *Brechas priorizadas y acciones de mejora*

#	Brecha	Norma	Impacto	Acción	Prioridad
1	39 avisos de seguridad en dependencias de producción	27001 A.8.8	Alto	Actualizar Laravel y dependencias; fijar política de parches	Alta
2	Cobertura de pruebas no medida	25010 / 29119	Alto	Instalar `xdebug` o `pcov` y publicar cobertura por módulo	Alta
3	Restauración de respaldos no probada	27001 A.8.13	Alto	Ejecutar un simulacro trimestral de restauración	Alta
4	Sin registro de auditoría de negocio	27001 A.8.15	Medio	Agregar tabla de auditoría para operaciones sensibles	Media

#	Brecha	Norma	Impacto	Acción	Prioridad
5	Sin análisis estático	25010 mantenibilidad	Medio	Incorporar PHPStan nivel 6 y ejecutarlo en CI	Media
6	120/228 archivos fuera de Pint	25010 mantenibilidad	Medio	Aplicar Pint y bloquear el merge si falla	Media
7	Sin pruebas de carga	25010 eficiencia	Medio	Medir latencia y concurrencia de los diez endpoints críticos	Media
8	Cola síncrona y acoplamiento a proveedores	25010 flexibilidad	Bajo	Introducir colas y adaptadores de proveedor	Baja
9	Documentación desincronizada con la dockerización	9001 7.5	Medio	Documentar el despliegue con Docker en el capítulo 27	Media

32.11 Limitaciones de la evaluación

- **No es una certificación.** No hubo auditoría externa ni revisión por pares independiente.
- **Alcance backend.** La calidad del frontend, de la app móvil y de la infraestructura no se midió; solo se evaluaron sus interfaces.
- **Commit de referencia.** Las métricas se tomaron sobre `1ea1e76`, posterior al commit auditado del manual (`01fb00d`). La diferencia incluye dockerización y lectura de credenciales desde `.env`, que no alteran las métricas de código pero sí la evaluación de instalabilidad.
- **Datos no verificados.** No se midió cobertura, latencia, disponibilidad ni satisfacción de usuario. Cualquier afirmación sobre esas dimensiones queda fuera de este capítulo.
- **Puntajes interpretativos.** La escala 0–4 es una herramienta de gestión, no un estándar certificable. Su valor está en la evidencia que la acompaña y en el plan de mejora, no en el número.

RECOMENDACIÓN

Recomendación. Repetir esta evaluación en cada release mayor, con la misma escala y las mismas fuentes, para que el índice global se convierta en una serie histórica. Un puntaje aislado no demuestra mejora; una serie sí.

90. Anexos

A. Plantilla de entorno comentada

```
# ── Aplicación ──────────────────────────────────────────
APP_NAME=KAFFA
APP_ENV=production
APP_KEY=base64:...
APP_DEBUG=false
APP_URL=https://api.kaffa.example
FRONTEND_URL=https://app.kaffa.example

# ── Base de datos PostgreSQL ────────────────────────────
DB_CONNECTION=pgsql
DB_HOST=127.0.0.1
DB_PORT=5432
DB_DATABASE=kaffa
DB_USERNAME=kaffa_app
DB_PASSWORD=<secreto>

# ── Sanctum / sesión ────────────────────────────────────
SANCTUM_EXPIRATION=1440
SESSION_DRIVER=redis
SESSION_LIFETIME=120

# ── CORS ────────────────────────────────────────────────
CORS_ALLOWED_ORIGINS=https://app.kaffa.example

# ── Caché / colas ───────────────────────────────────────
CACHE_DRIVER=redis
QUEUE_CONNECTION=redis
FILESYSTEM_DISK=local

# ── OTP ─────────────────────────────────────────────────
OTP_LENGTH=6
OTP_EXPIRA_MINUTOS=15
OTP_MAX_INTENTOS=5
OTP_REENVIO_SEGUNDOS=60

# ── Correo (Brevo) ──────────────────────────────────────
MAIL_MAILER=smtp
MAIL_HOST=smtp-relay.sendinblue.com
MAIL_PORT=587
MAIL_ENCRYPTION=tls
MAIL_USERNAME=<login>@smtp-brevo.com
MAIL_PASSWORD=<smtp-key>
MAIL_FROM_ADDRESS=<remitente-verificado>
MAIL_FROM_NAME=KAFFA
MAIL_TIMEOUT=10
MAIL_VERIFY_PEER=true

# ── Cloudinary ──────────────────────────────────────────
CLOUDINARY_CLOUD_NAME=roffd6et
CLOUDINARY_UPLOAD_PRESET=KAFFACCYS
CLOUDINARY_FOLDER=samples/ecommerce
CLOUDINARY_API_KEY=
CLOUDINARY_API_SECRET=

# ── Logs / docs ─────────────────────────────────────────
LOG_CHANNEL=daily
```

```
LOG_LEVEL=warning
SCRAMBLE_ENABLED=false
```

B. Catálogo OpenAPI

`docs/openapi.json` documenta **69 paths**. Los más relevantes:

Tabla 172. *Catálogo OpenAPI por grupo de rutas*

Grupo	Paths
Auth	`/login`, `/registro`, `/forgot-password`, `/reset-password`, `/email/verify-code`, `/email/verification-notification`, `/me`, `/user`, `/perfil`, `/logout`, `/auth/refresh`, `/auth/logout-all`, `/health`
Público	`/equipo`, `/categorias`, `/productos`, `/eventos`
Catálogo	`/insumos`, `/recetas`, `/medios-pago`
Pedidos	`/pedidos`, `/pago-pedidos`, `/factura-ventas`
Caja	`/cajas`, `/movimiento-cajas`, `/turno-activo`
Compras	`/compras`, `/detalle-compras`, `/compra-pagos`, `/proveedores`
Gastos	`/gastos`, `/gasto-pagos`, `/mermas`
Turnos	`/turnos`, `/mis-turnos`
Chat	`/mensajes`, `/mensajes/contactos`, `/mensajes/leer`
Reportes	`/reportes`
Usuarios	`/usuarios`, `/usuario-roles`, `/roles`
Eventos admin	`/eventos/admin`, `/eventos/imagen`
Asistente	`/asistente/mensajes`, `/asistente/chat`

Para la lista completa con métodos y middleware, use el capítulo 26.

C. SQL útil

C.1 Crear base y usuario

```
CREATE ROLE kaffa_app WITH LOGIN PASSWORD '<secreto>';
CREATE DATABASE kaffa OWNER kaffa_app ENCODING 'UTF8' TEMPLATE template0;
CREATE DATABASE kaffa_test OWNER kaffa_app ENCODING 'UTF8' TEMPLATE template0;
```

C.2 Respaldo/restauración

```
pg_dump -h 127.0.0.1 -U kaffa_app -d kaffa -Fc -f kaffa.dump
pg_restore -h 127.0.0.1 -U kaffa_app -d kaffa_restore --clean --if-exists kaffa.dump
```

C.3 Ventas y stock

```
-- Ventas de hoy
SELECT count(*) AS pedidos, coalesce(sum(total),0) AS total
```

```
FROM pedidos
WHERE deleted_at IS NULL
  AND estado IN ('pagado','entregado')
  AND created_at >= CURRENT_DATE;

-- Stock bajo
SELECT nombre, stock_actual, stock_minimo
FROM insumos
WHERE deleted_at IS NULL AND stock_actual <= stock_minimo
ORDER BY stock_actual - stock_minimo;

-- Arqueo
SELECT c.id,
       c.monto_apertura_sistema
       + coalesce(sum(CASE WHEN m.tipo='ingreso' THEN m.monto ELSE 0 END),0)
       - coalesce(sum(CASE WHEN m.tipo='egreso' THEN m.monto ELSE 0 END),0) AS sistema
FROM cajas c
LEFT JOIN movimiento__cajas m ON m.caja_id = c.id
WHERE c.id = :caja_id
GROUP BY c.id;
```

C.4 Tokens y pendientes

```
DELETE FROM personal_access_tokens
WHERE expires_at IS NOT NULL AND expires_at < now() - interval '30 days';

SELECT correo, nombre, created_at
FROM registros_pendientes
WHERE created_at < now() - interval '24 hours';
```

D. Configuraciones de servidor

Los ejemplos completos de Nginx, Apache y PHP-FPM están en el capítulo 27. Puntos clave:

- `DocumentRoot` debe apuntar a `backend-kaffa/public`, nunca a la raíz del proyecto.
- `client_max_body_size` ≥ 12 MB para imágenes de evento.
- `try_files` para Laravel.
- `fastcgi_pass` al socket de PHP-FPM.
- HTTPS obligatorio.
- `storage/` servido como archivo estático, sin ejecución PHP.

E. Catálogo de errores

Tabla 173. *Catálogo de errores de negocio*

Código	HTTP	Acción
`EMAIL_NOT_VERIFIED`	403	Verificar correo
`FUERA_HORARIO`	403	Esperar turno
`TURNO_INACTIVO`	403	Asignar turno
`TURNO_EXPIRADO`	403	Cerrar caja
`PERMISO_DENEGADO`	403	Revisar rol
`CAJA_NO_EXISTE`	422	Corregir caja

Código	HTTP	Acción
`CAJA_CERRADA`	422	Abrir caja
`CAJA_AJENA`	422	Usar caja propia
`TOKEN_SIN_ABILITY`	403	Rotar token
—	401	Enviar token
—	404	Revisar ruta/ID
—	422	Revisar `errors`
—	429	Backoff
—	500	Revisar logs

F. Trazabilidad de pruebas

Tabla 174. *Trazabilidad entre requisitos y pruebas*

Requisito	Prueba
Login/token	`ApiTest`, `TokenSessionTest`
Registro/OTP	`EmailVerificationTest`
Reset	`PasswordResetTest`
Catálogo público	`ApiTest`
Equipo	`EquipoTest`
Pedidos	`PedidoTest`
Caja	`CajaTest`
Compras/mermas	`CompraMermaTest`
Turnos	`TurnoMiddlewareTest`
Chat/reportes	`ChatReporteTest`
Asistente	`AsistenteTest`
Eventos	`EventoTest`

G. Correspondencia documento-código

Tabla 175. *Correspondencia entre documentos y código*

Documento	Código
`01-arquitectura.md`	`routes/api.php`, `app/Providers`, `config`
`02-base-datos.md`	`database/migrations`, `app/Models`
`03-api.md`	`routes/api.php`, `app/Http/Controllers`
`04-modulos.md`	`app/Services`, `app/Models`
`05-seguridad.md`	`app/Http/Middleware`, `app/Policies`
`06-instalacion.md`	`.env.example`, `composer.json`

Documento	Código
`07-despliegue.md`	`config`, `.env.example`
`08-pruebas.md`	`tests/Feature`
`10-requisitos.md`	Rutas + pruebas
`openapi.json`	Scramble

H. Glosario

Tabla 176. *Glosario de términos*

Término	Definición
Ability	Permiso acuñado en un token Sanctum (`pedidos:manage`, `catalogo:read`, etc.).
Arqueo	Conciliación de caja: apertura + ingresos − egresos.
Barista	Rol operativo; requiere turno activo.
Comprobante	Imagen que acredita un pago virtual.
Medio virtual	Medio con `es_virtual=true`; exige comprobante.
Merma	Pérdida de insumo; reduce stock.
Pivote	Tabla N:M (`usuario__rols`, `recetas`, `barista_turno`).
POS	Punto de venta.
Receta	Consumo de insumo por producto.
RBAC	Control de acceso por roles.
Sanctum	Paquete de tokens Bearer.
Scope	Filtro de datos por rol (`scopeByUser`).
Soft delete	Borrado lógico con `deleted_at`.
Throttle	Límite de peticiones.
Turno	Bloque de trabajo con horario y fecha.
OTP	Código de un solo uso.

91. Referencias y control de versiones

91.1 Referencias técnicas

- LARAVEL. *Laravel 10.x Documentation*. https://laravel.com/docs/10.x
- LARAVEL. *Laravel Sanctum*. https://laravel.com/docs/10.x/sanctum
- THE POSTGRESQL GLOBAL DEVELOPMENT GROUP. *PostgreSQL Documentation*. https://www.postgresql.org/docs/
- PHP GROUP. *PHP Manual*. https://www.php.net/manual/
- DEDOC. *Scramble — OpenAPI generator for Laravel*. https://scramble.dedoc.co
- PHPUNIT PROJECT. *PHPUnit 10 Documentation*. https://phpunit.de
- CLOUDINARY. *Upload API*. https://cloudinary.com/documentation/upload_images
- BREVO. *SMTP Relay*. https://developers.brevo.com/docs/smtp-relay
- REACT. *React Documentation*. https://react.dev
- VITE. *Vite Documentation*. https://vite.dev
- ISO. *ISO 9001:2015 — Sistemas de gestión de la calidad. Requisitos*. https://www.iso.org
- ISO/IEC. *ISO/IEC 25010:2023 — Modelo de calidad de producto (SQuaRE)*. https://www.iso.org
- ISO/IEC. *ISO/IEC 27001:2022 — Sistemas de gestión de la seguridad de la información*. https://www.iso.org
- ISO/IEC. *ISO/IEC 27002:2022 — Controles de seguridad de la información*. https://www.iso.org
- ISO/IEC. *ISO/IEC 29119-1:2022 — Conceptos generales de pruebas de software*. https://www.iso.org
- ISO/IEC/IEEE. *ISO/IEC/IEEE 12207:2017 — Procesos del ciclo de vida del software*. https://www.iso.org
- ISO/IEC/IEEE. *ISO/IEC/IEEE 42010:2011 — Descripción de arquitectura*. https://www.iso.org
- ISO. *ISO 8601-1:2019 — Representación de fechas y horas*. https://www.iso.org
- ISO. *ISO 14289-1:2014 — PDF/UA: accesibilidad de documentos PDF*. https://www.iso.org
- ISO. *ISO 31000:2018 — Gestión del riesgo. Directrices*. https://www.iso.org

91.2 Documentos internos

Tabla 177. *Documentos internos del proyecto*

Documento	Ruta
Arquitectura	`docs/01-arquitectura.md`
Base de datos	`docs/02-base-datos.md`
API	`docs/03-api.md`
Módulos	`docs/04-modulos.md`
Seguridad	`docs/05-seguridad.md`
Instalación	`docs/06-instalacion.md`
Despliegue	`docs/07-despliegue.md`
Pruebas	`docs/08-pruebas.md`
Requisitos	`docs/10-requisitos.md`

Documento	Ruta
Frontend	`docs/11-frontend.md`
OpenAPI	`docs/openapi.json`
Glosario	`docs/glosario.md`
Cambios	`docs/CHANGELOG.md`
Contribución	`docs/CONTRIBUCION.md`

91.3 Control de versiones del manual

Tabla 178. *Control de versiones del manual*

Versión	Fecha	Cambio
1.0	Octubre 2026	Creación del manual consolidado del backend
1.0 (rev. 2)	Noviembre 2026	Corrección de diagramas y paginación, índice de tablas y capítulo 32 de calidad frente a normas ISO

Commit backend auditado: `01fb00d` (`fix(turnos): fecha_fin opcional`). **Documentación base:** `docs/` sincronizada al 16 de octubre de 2026. **Entorno de auditoría:** PHP 8.5.11, PostgreSQL 18.6, Composer 2.10.3, Node 24.19.

91.4 Cómo mantenerlo

1. Actualice el capítulo afectado.
2. Regenere `openapi.json` si cambian rutas.
3. Ejecute `python3 build/gen_endpoints.py` si cambia la matriz.
4. Ejecute `node build/build-all.mjs`.
5. Revise el PDF y el índice.
6. Registre el cambio en este control de versiones.

> **TRAZABILIDAD**
>
> **Nota.** Este manual se generó a partir del código y de la documentación existente. Las afirmaciones no verificadas se marcaron como **Recomendación**, `[NO VERIFICADO]` o `[GAP]`. Si encuentra una discrepancia, prevalezca el código y actualice el manual.



Fin del documento

Manual verificado contra el código fuente del backend.

Para actualizarlo, consulte **docs/manual-tecnico/README.md**
y regenere el PDF con el script de construcción.